

DSH-Pet

源代码全文 PDF

逐文件收录当前工作区代码，带文件名与行号

生成日期：2026-08-24

范围：pet / tests / scripts / tools 及根目录代码配置

说明：行号和文件标题是 PDF 排版辅助信息；代码正文按文件内容输出。视频、图片、模型和临时运行产物不属于代码，未写入本 PDF。

文件目录

序号	文件	行数
1	.gitignore	36
2	README.md	910
3	pet/__init__.py	4
4	pet/__main__.py	8
5	pet/action_dialog.py	61
6	pet/app.py	338
7	pet/autostart.py	135
8	pet/catalog.py	492
9	pet/config.py	87
10	pet/hotkeys.py	197
11	pet/interaction.py	49
12	pet/library.py	362
13	pet/performance.py	47
14	pet/sound.py	106
15	pet/webm_clip.py	502
16	pet/window.py	1440
17	run-mac.command	9
18	scripts/convert_to_gif.py	71
19	tests/diagnose.py	76
20	tests/grab_screen.py	37
21	tests/pos_diag.py	40
22	tests/smoke.py	138
23	tests/test_media_runtime.py	252
24	tests/test_runtime.py	77

序号	文件	行数
25	tests/win_diag.py	61
26	tools/build_hq_webm.py	330
27	tools/build_project_pdf.py	253
28	tools/build_runtime_webm_assets.py	167
29	tools/build_source_code_pdf.py	161
30	tools/superres_webm_assets.py	207

01. .gitignore (36 lines)

```
1 | # Python
2 | __pycache__/
3 | *.py[cod]
4 | .pytest_cache/
5 | .venv/
6 | .pyinstaller/
7 |
8 | # 运行时产物
9 | tmp_appdata/
10 | tests/_tmp_*/
11 | backups/
12 |
13 | # 素材: 只提交 assets/characters (角色动画), 其他 assets 内容不入库
14 | assets/*
15 | !assets/characters/
16 | !assets/characters/**
17 | !assets/app-icon.png
18 | !assets/app-icon.icns
19 | assets/thumb/
20 | assets/videos/
21 | assets/animations/
22 | assets/prompt/
23 |
24 | # macOS Qt 运行时插件别名 (用于当前 PySide6 wheel 的插件发现兼容)
25 | qt-plugins/
26 |
27 | # PyInstaller 构建产物
28 | build/
29 | dist/
30 | *.spec
31 | releases/
32 |
33 | # 本地开发文档 (不随仓库发布)
34 | LOG.md
35 | LOG-INDEX.md
36 | SPEC.md
```

02. README.md (910 lines)

```
1 | # dsh-pet-indesktop
2 |
3 | ## 下载
4 |
5 | 无需从源码构建, 直接到 [Releases] (https://github.com/MerZiin/dsh-pet-indesktop/releases)
6 | 页面下载对应系统的安装包:
7 |
8 | | 你的系统 | 安装包 | 说明 |
9 | |-----|-----|-----|
10 | | Windows (WebM 版) | `dsh-pet-standalone-webm.exe` | 包体小、画质高; 首次启动解压较慢, 播放加速效果不明显 |
11 | | Windows (GIF 版) | `dsh-pet-standalone-gif.exe` | 启动快、播放加速效果好; 缺点是包体较大 |
12 | | macOS (Apple Silicon / M 系列) | `DSH-Pet-macos-arm64-1440p-master-adaptive.zip` | 解压得 `DSH-Pet.app`, 首次打开需放行 (见下方「macOS」章节) |
13 | | macOS (Intel) | — | 暂无安装包, 请按「macOS」章节源码运行 |
14 |
15 | > 文件名以 Release 页面实际发布为准。
16 |
17 | ## 本地 macOS 成品
18 |
19 | 当前签名后的 Apple Silicon 成品安装在 `~/Applications/DSH-Pet.app`, 压缩包位于
20 | `releases/DSH-Pet-macos-arm64-1440p-master-adaptive.zip`, 2560×1440 母版素材包为
21 | `releases/dsh-pet-superres-1440p-transparent-webm-91-actions.zip`, 应用使用白鲸 +
22 | 深蓝渐变圆角图标, 双击 `.app` 即可运行。
23 |
24 | 透明视频默认清理 VP9 常见的 Alpha=1 底噪, 不改动真实半透明边缘; 可在桌宠右键菜单或 macOS
25 | 状态栏菜单的「画面调整 → 清理透明底噪 (Alpha=1)」中随时关闭, 设置会自动保存。
26 |
27 | 高画质素材链包含完整 91 段 2560×1440 透明 WebM (24fps), 由原作 640×360
28 | Alpha 素材使用 Real-ESRGAN AnimeVideo-v3 4×超分生成。2560×1440 作为母版
29 | 独立保留; App 使用由母版 Lanczos 下采样的 1280×720 透明运行代理, 默认 462px
30 | 档再按 Retina 需求解码到 922×520。屏幕可见细节不减少, 同时避免每帧解码完整
31 | 1440p。原始素材与旧 1080p 版本也保存在独立备份目录, 不会被覆盖。
32 |
33 | 超分素材可用下列命令做逐文件复核 (只验证, 不生成):
34 |
```

```

35 | ```sh
36 | ./venv/bin/python tools/superres_webm_assets.py \
37 |   --input-root backups/shenshen-webm-original-640-91 \
38 |   --output-root backups/shenshen-webm-superres-1440-master-91 \
39 |   --scale 4 --verify-only
40 | ```
41 |
42 | 运行代理复核:
43 |
44 | ```sh
45 | ./venv/bin/python tools/build_runtime_webm_assets.py \
46 |   --input-root backups/shenshen-webm-superres-1440-master-91 \
47 |   --output-root assets/characters/shenshen/videos --verify-only
48 | ```
49 |
50 | > **声明与致谢**: 本项目改自、源于 [dsh-pet] (https://github.com/PC2005-cloud/dsh-pet)。
51 | > 桌宠的动画素材、动画链行为模型、交互设计均来自原项目，特此声明并感谢原作者的贡献。
52 |
53 | 把 [dsh-pet] (https://github.com/PC2005-cloud/dsh-pet) 插件里的桌宠，改造成一个
54 | **跨平台的独立桌面宠物**软件（支持 **Windows** 与 **macOS**）—— 不依赖 DSH 运行时。
55 | 用 Python + PySide6 实现，双击即跑，内置 91 段母版派生的 1280×720 透明运行动画（24fps）。
56 |
57 | 当前提供两个 Windows 版本:
58 |
59 | - **WebM 版**: 使用 2560×1440 母版派生的透明运行代理，并按实际显示尺寸有界解码。
60 | - **GIF 版**: 使用 GIF/QMovie 播放，启动快、播放加速效果好，缺点是包体较大。
61 |
62 | 两个版本都支持多角色、外部扩展、切换角色、播放速率、鼠标穿透、拖动物理等功能。
63 |
64 | Windows 用户见下方「快速开始 / 打包为 exe」，macOS 用户见「[macOS] 章节」。
65 |
66 | ## 版本对比
67 |
68 | | 版本 | 启动速度 | 播放加速效果 | 包体大小 | 适用场景 |
69 | |-----|-----|-----|-----|-----|
70 | | WebM 版 | 较快 | 不明显 | App 约 460MB / ZIP 约 381MB | 追求画质、透明边缘 |
71 | | GIF 版 | 快 | 明显 | 约 450MB | 追求启动快、播放加速效果明显 |
72 |
73 | >
74 | > **注意**: GIF 版当前**不建议**切换到 webm 素材角色**，切换后可能出现卡死。
75 | > GIF 版请使用 GIF 素材；需要播放 webm 角色请使用 WebM 版。
76 |
77 | ## 近期优化
78 |
79 | - 2560×1440 透明超分母版 + 1280×720 运行代理，默认档仅解码到 922×520
80 | - 解码队列限制为 2 帧并采用背压，不丢帧、不让 ffmpeg 提前解完整段动画
81 | - 媒体库先建立轻量动作索引，播放器按需创建；启动只预热移动动作，新增角色/动作无需改播放图层
82 | - 系统持续高负载时降低待机速度、主动动作频率和命中 mask 刷新率；画面帧率不变
83 | - 从托盘隐藏桌宠时暂停媒体解码，重新显示时自动恢复
84 | - 性能策略：素材事实来源和媒体清理拆分，角色切换/退出统一停止后台 reader
85 | - 支持 GIF / WebM 混合素材，GIF 版也能切换并播放 webm 角色
86 | - 多开桌宠时不再互相清理缓存导致卡住
87 | - 拖拽切换动画不再卡顿
88 | - 点击 Q 弹反馈，连续点击可打断动画
89 | - 播放速率调节（1.0x ~ 2.0x）
90 | - 鼠标穿透开关
91 | - 拖动物理效果（惯性、离心、抛出、重力、反弹衰减、地面摩擦）
92 | - 转向动画开始不再出现突兀镜像
93 |
94 | ## 相关优化项目
95 |
96 | [ianlike-ui/dsh-pet-standalone] (https://github.com/ianlike-ui/dsh-pet-standalone)
97 |
98 | 这是其他开发者基于本项目做的优化实现，可能在播放性能、打包体积、多角色支持或使用体验等方面进行了改进。
99 | 如果你希望体验社区优化版，可以前往该仓库查看说明和最新成果。
100 |
101 | ## 特性
102 |
103 | - **高画质透明播放**: 内置 1440p 母版派生的 VP9 Alpha 运行代理，保留透明边缘
104 | - **动画链**: 每个动画播完按概率选下一个 —— 30% 待机 / 10% 转向 / 40% 随机动作 / 20% 移动，永不停止
105 | - **多形象支持**: 支持用户通过外部目录添加自定义角色
106 | - **角色热切换**: 右键桌宠或托盘菜单可随时切换形象，无需重启
107 | - **屏幕漫游**: 朝面向方向行走，先检查屏幕空间、不走出屏幕（移动动画前后各 2s 准备/收尾，位置由代码驱动）
108 | - **左右朝向**: 转向动画播完翻转朝向，所有动画支持水平镜像
109 | - **点击回应**: 点击宠物随机播放当前角色配置的回应动画（链上非待机动画播放中不打断）
110 | - **点击 Q 弹**: 点击时立即产生“变矮再复原”的挤压反弹反馈；连续点击可打断当前动画并重复触发 Q 弹
111 | - **多级点击回应**: 单击、双击、长按、快速连续点击会分流到不同回应动作：双击/长按/连点可触发尖叫鸭音效
112 | - **鼠标靠近反馈**: 光标进入角色附近时，角色会自动朝向鼠标
113 | - **主动问候**: 默认每隔一段时间随机播放一次挥手问候，可从「互动反馈」关闭
114 | - **拖拽**: 按住拖动超过 5px 判定为拖拽，宠物播放“悬空反馈”动画跟手，松手停在原地

```

```

114 | - **边缘反馈**: 拖到屏幕边缘会短暂压扁并反弹离开; 当前角色没有独立“趴下”素材, 因此先用 Q 弹代替
115 | - **透明穿透**: 窗口逐帧按人物 alpha 生成 mask, 透明区域鼠标直接穿透到下层窗口
116 | - **右键菜单**: 手动播放待机/转向/移动/点击回应/随机动作、切换角色、回到右下角、窗口置顶、不移动、开机自启、4 档大小、退出
117 | - **系统托盘**: 显示/隐藏、切换角色、开机自启、退出; 位置/朝向/大小/置顶自动持久化
118 | - **开机自启**: Windows 写 HKCU 注册表 Run 键 / macOS 写 LaunchAgents (均无需管理员权限), 可随时开关
119 | - **播放速率调节**: 右键菜单可调 1.0x ~ 2.0x 动画播放速度
120 | - **鼠标穿透**: 托盘菜单可开启鼠标穿透, 开启后鼠标点击会穿透桌宠到下层窗口
121 | - **拖动物理**: 可开关的拖拽物理效果, 松手会抛出、带重力与反弹衰减; 拖拽过程中有惯性/离心感
122 | - **动作收藏夹 / 播放列表**: 91 段动画可收藏、编辑播放列表, 并选择循环或随机播放
123 | - **性格模式**: 安静、活泼、调皮会改变待机、动作、移动和主动问候概率
124 | - **全局快捷键**: `H/P/R/M/D` 分别控制显示隐藏、暂停、随机动作、鼠标穿透和尖叫鸭
125 |
126 | ## 自定义角色教程
127 |
128 | 你可以通过两种方式使用自定义形象:
129 |
130 | 1. **随 exe 打包**: 把形象放到项目源码的 `assets/characters/<角色ID>/videos/`, 重新打包 exe。
131 | 2. **用户本地外部扩展**: 不需要重新打包, 直接在 exe 同目录或用户数据目录放置形象文件夹。
132 |
133 | ### 方法一: 随 exe 打包
134 |
135 | 在项目源码中创建:
136 |
137 | ```text
138 | assets/characters/<角色ID>/videos/
139 | ├── idle/
140 | ├── turn/
141 | ├── move/
142 | ├── click/
143 | ├── drag/
144 | └── random/
145 | ```
146 |
147 | 把对应分类的 `.webm` 放进去, 然后重新执行打包命令即可。
148 |
149 | ### 方法二: exe 同目录外部扩展 (推荐给最终用户)
150 |
151 | 在 exe 同目录下创建:
152 |
153 | ```text
154 | <exe 所在目录>/
155 | ├── dsh-pet-standalone-webm.exe
156 | └── characters/
157 |     ├── <角色ID>/
158 |         ├── videos/
159 |             ├── idle/
160 |             ├── turn/
161 |             ├── move/
162 |             ├── click/
163 |             ├── drag/
164 |             └── random/
165 |         ...
166 |     ...
167 |
168 | 也支持用户数据目录:
169 |
170 | ```text
171 | Windows: %APPDATA%\dsh-pet-standalone/characters/<角色ID>/videos/
172 | macOS: ~/Library/Application Support/dsh-pet-standalone/characters/<角色ID>/videos/
173 | ```
174 |
175 | 程序启动或切换角色时会自动检测:
176 |
177 | - 外部目录存在 → 优先使用外部形象。
178 | - 外部目录不存在 → 回退到 exe 内置形象, 不会报错。
179 |
180 | ### 如何准备自定义形象的动画 (参考项目绘制方法)
181 |
182 | 推荐直接参考上游项目 [dsh-pet] (https://github.com/PC2005-cloud/dsh-pet):
183 |
184 | 1. 克隆或下载参考项目:
185 | ```sh
186 | git clone --depth 1 https://github.com/PC2005-cloud/dsh-pet.git
187 | ```
188 |
189 | 2. 参考项目中的透明动画位于:
190 | ```text
191 | dsh-pet/dsh-pet/assets/thumb/*.webm
192 | ```
193 |
194 | 这些是 640×360 透明 webm (VP9 + 8-bit alpha)。
195 |
196 | 3. 你可以:

```

```

193 | - 直接复制这些 webm 作为基础形象；
194 | - 或参考它们的动作分类，生成自己角色的同尺寸透明 webm；
195 | - 或使用 ffmpeg / 图像生成工具制作新的透明动画，建议至少覆盖你的最大显示尺寸。
196 |
197 | 4. 将制作好的 webm 按分类放入对应子目录：
198 |
199 | ```text
200 | videos/
201 | |--- idle/      # 待机动画
202 | |--- turn/      # 转向动画
203 | |--- move/      # 移动动画
204 | |--- click/     # 点击响应动画
205 | |--- drag/      # 拖拽动画（可选）
206 | |--- random/    # 随机动作动画
207 | ```
208 |
209 | 5. 如果文件名无法通过关键词自动识别，可以添加 `manifest.json` 精确指定分类。
210 |
211 | 6. 启动后在桌宠右键菜单或托盘菜单的「切换角色」中选择你的新角色 ID。
212 |
213 | ## 使用手册
214 |
215 | ### 启动与退出
216 |
217 | - **启动**：双击 `run.bat`（或 `python -m pet` / 打包后的 exe），桌宠出现在屏幕右下角
218 | - **退出**：右键桌宠 → 「退出」，或点系统托盘图标 → 「退出」
219 |
220 | ### 鼠标交互
221 |
222 | - **单击**：随机播放一个点击响应动画，并触发 Q 弹挤压回弹效果。
223 | - **双击**：播放更强的点击回应，并按「互动反馈 → 尖叫鸭音效」设置播放尖叫鸭。
224 | - **长按**：按住约 0.52 秒触发被吓一跳/随机动作回应；拖动超过 5px 后会转为拖拽。
225 | - **快速连续点击**：三次及以上连续点击会切换到随机动作回应，并播放尖叫鸭。
226 | - **鼠标靠近**：鼠标进入角色附近约 280px 内时，角色会看向鼠标。
227 | - **拖拽**：按住拖动超过 5px 判定为拖拽，宠物播放「悬空反馈」动画跟手，松手停在原地
228 | - **穿透**：只有宠物本体（不透明区域）可点，其余透明区域鼠标直接穿透到下层窗口
229 |
230 | ### 新功能使用说明
231 |
232 | - **播放速率**：右键桌宠 → 「播放速率」→ 选择 `1.0x ~ 2.0x`，动画会立即按新速度播放。
233 | - **鼠标穿透**：右键系统托盘图标 → 勾选「鼠标穿透」，开启后鼠标点击会穿透桌宠到下层窗口；取消勾选即可恢复。
234 | - **拖动物理**：右键桌宠 → 勾选「拖动物理」，开启后：
235 |   - 拖动时会有惯性/离心感；
236 |   - 松手后桌宠会被抛出；
237 |   - 碰到屏幕边缘会反弹并逐渐衰减；
238 |   - 落地后受摩擦力影响会慢慢停下。
239 | - **互动反馈**：右键桌宠或状态栏图标 → 「互动反馈」，可开关「尖叫鸭音效」和「偶尔主动打招呼」。脚步声、休息声暂未加入。
240 | - **动作收藏夹 / 播放列表**：右键桌宠或状态栏图标 → 「动作管理」，可以编辑收藏夹和播放列表，再选择「循环播放」或「随机播放」。
241 | - **性格模式**：右键桌宠或状态栏图标 → 「性格模式」，可选「安静 / 活泼 / 调皮」。
242 | - **全局快捷键**：默认使用 `H/P/R/M/D`，加 `H/P/R/M/D`；如果系统没有成功注册全局监听，状态栏仍可正常使用全部菜单功能。
243 |
244 | ### 右键菜单（右键点击宠物本体）
245 |
246 | | 菜单项 | 功能 |
247 | |-----|-----|
248 | | 动画 • 待机 | 手动播放待机动画；如果待机目录有多个视频，会显示二级菜单 |
249 | | 动画 • 转向 | 手动播放转向动画；如果转向目录有多个视频，会显示二级菜单 |
250 | | 动画 • 移动 | 手动播放移动动画（走路姿态 + 朝面方向真实走动；「不移动」模式下这是唯一触发移动的方式） |
251 | | 动画 • 点击回应 | 手动播放点击回应动画 |
252 | | 动画 • 随机动作 | 手动播放随机动作动画 |
253 | | 切换角色 | 热切换当前形象（内置 + 外部扩展角色都会列出） |
254 | | 回到右下角 | 把宠物复位到屏幕右下角 |
255 | | 窗口置顶 | 勾选 = 始终显示在其他窗口之上，取消 = 可被其他窗口遮挡 |
256 | | 不移动 | 勾选 = 只播放原地动画（待机/转向/随机动作），不再自动走动；取消 = 恢复正常模式 |
257 | | 开机自启 | 勾选 = 随 Windows 登录自动启动（见下方说明） |
258 | | 大小 | 4 档缩放：320px / 462px / 544px / 640px（默认 462px） |
259 | | 播放速率 | 调节动画播放速度：1.0x / 1.1x / ... / 2.0x |
260 | | 拖动物理 | 开启后拖动桌宠有惯性/离心感，松手会抛出并带重力反弹衰减 |
261 | | 退出 | 关闭桌宠 |
262 |
263 | ### 系统托盘图标
264 |
265 | | 操作 | 功能 |
266 | |-----|-----|
267 | | 双击托盘图标 | 显示 / 隐藏宠物 |
268 | | 右键托盘图标 → 显示/隐藏 | 同上 |
269 | | 右键托盘图标 → 切换角色 | 热切换当前形象 |
270 | | 右键托盘图标 → 鼠标穿透 | 开启后鼠标点击穿透到下层窗口 |
271 | | 右键托盘图标 → 开机自启 | 与右键菜单的开机自启同步 |

```

```

272 | | 右键托盘图标 → 退出          | 关闭桌宠          |
273 |
274 | ### 自动行为（无需任何操作）
275 |
276 | 桌宠会自己“生活”，挂机即可观赏：
277 |
278 | - **动画链**：每个动画播完按概率抽下一个 —— 30% 待机 / 10% 转向 / 40% 随机动作 / 20% 移动，永不停止
279 | - **屏幕漫游**：朝面向方向行走，先检查屏幕空间、不走出屏幕
280 | - **朝向翻转**：播完「东张西望」后左右翻转朝向，所有动画随之镜像
281 | - **状态记忆**：位置、朝向、大小、置顶、不移动、开机自启、当前角色均自动保存（`%APPDATA%\dsh-pet-standalone/config.json`），下次启动自动恢复
282 |
283 | ### 开机自启
284 |
285 | 勾选「开机自启」后，桌宠随系统登录自动启动：取消勾选即移除。
286 |
287 | - **Windows**：写入注册表 `HKCU\Software\Microsoft\Windows\CurrentVersion\Run`（无需管理员权限）：
288 |   源码运行时指向 `pythonw -m pet`；打包成 exe 后会先用 `start /D` 切到 exe 所在目录再启动 exe，
289 |   避免开机时默认工作目录不可写导致 onefile 解压失败（旧版自启命令会在下次启动时自动升级）
290 | - **macOS**：写入 LaunchAgents（`~/Library/LaunchAgents/`），登录后自动启动
291 |
292 | ## 实现方式
293 |
294 | ### 技术栈
295 |
296 | - **Python 3.10+ / PySide6**（Qt for Python, LGPL 许可）
297 | - **imageio-ffmpeg** 解码 1280×720 VP9 Alpha 运行代理，并按当前显示档位输出有界 RGBA
298 |
299 | ### 动画链状态机（1:1 移植原插件 `client.js`）
300 |
301 | 原插件是一个“链式”状态机：每个动画一次性播放，播完按概率抽下一个
302 | （30% 待机 / 10% 转向 / 40% 动作 / 20% 移动）。本项目的 `pet/window.py`
303 | 将这套行为完整移植为 Python：
304 |
305 | - `_pick_next()` 按概率抽下一个动画
306 | - 转向动画播完翻转朝向（`facing=right` 时水平镜像，等效原版 `scaleX(-1)`）
307 | - 移动动画只提供“走路姿态”，位置由 `QTimer` 驱动：开头/结尾各 2s 原地不动，
308 |   中间按播放进度插值位移
309 | - 点击回应/拖拽动画播完先回“待机缓冲”，待机播完再回到随机链
310 | - 点击只在待机时响应，5px 阈值区分点击与拖拽（等效原版命中层设计）
311 |
312 | ### 透明窗口与鼠标穿透
313 |
314 | 窗口用 Qt 的 `Tool` 无边框置顶窗口 + `WA_TranslucentBackground` 实现透明背景。
315 | 每帧按人物 alpha 通道生成 `QBitmap` mask，透明区域鼠标直接穿透到下层窗口，
316 | 实现“只有宠物本体可点击”（等效原版 HIT_BOX 命中层）。
317 |
318 | ### 素材播放（webm 主路线）
319 |
320 | 本项目保存 2560×1440 透明 **webm** 母版（VP9 + 8-bit alpha），App 播放其
321 | 1280×720 透明运行代理，并由 `imageio-ffmpeg` 在解码进程内按当前显示档位缩放。
322 | 逻辑画布固定为 640×360，因此素材分辨率不会改变桌宠大小或移动参数。
323 |
324 | 关键实现：
325 |
326 | - 解码命令核心参数：
327 |   ```python
328 |   imageio_ffmpeg.read_frames(
329 |       path,
330 |       pix_fmt="rgba",
331 |       bits_per_pixel=32,
332 |       input_params=["-c:v", "libvpx-vp9"],
333 |       output_params=["-vf", "scale=...:flags=lanczos"],
334 |   )
335 |   ```
336 | - `c:v libvpx-vp9` 必须放在输入之前，否则原生 vp9 解码器会丢弃 alpha。
337 | - 播放架构：
338 |   - 后台 reader 线程只负责把 RGBA 帧放入 2 帧有界队列：队列满时背压解码器。
339 |   - 主线程 `QTimer` 按视频 fps 逐帧从队列取帧。
340 |   - 每次只取最早的一帧，**不跳帧、不追帧**，避免动画快速。
341 |   - 所有 `QImage/QPixmap` 和窗口 mask 更新都在主线程完成。
342 |   - Windows 下 `imageio-ffmpeg` 内部使用 `STARTUPINFO` 隐藏 ffmpeg 控制台窗口，
343 |     避免旧 ffmpeg 子进程方案导致的“窗口反复出现/消失”。
344 |
345 | ### 多形象支持
346 |
347 | 项目支持多角色形象，每个角色一个独立目录：
348 |
349 | ```text
350 | assets/characters/<character_id>/videos/

```



```

351 | |--- idle/      # 待机
352 | |--- turn/     # 转向
353 | |--- move/     # 移动
354 | |--- click/    # 点击回应
355 | |--- drag/     # 拖拽 (可选)
356 | |--- random/   # 随机动作
357 | |---
358 |
359 | 当前内置角色:
360 |
361 | ```text
362 | shenshen (内置) + 用户通过外部目录添加的角色
363 | ```
364 |
365 | - 默认形象为 `shenshen`, 当前动画放在 `assets/characters/shenshen/videos/`。
366 | - 不同角色可以有**不同的动作集**: 程序会递归扫描 `videos/` 下的子目录,
367 |   按目录自动区分“待机 / 转向 / 移动 / 点击回应 / 拖拽 / 随机动作”。
368 | - 内置角色会随 exe 一起打包。
369 | - 同时支持用户本地外部扩展:
370 |   - exe 同目录或当前工作目录下的 `characters/<id>/videos/`
371 |   - 用户数据目录下的 `dsh-pet-standalone/characters/<id>/videos/`
372 |   - 运行时自动检测, 存在则优先使用, 不存在则回退内置, 不会报错。
373 | - 右键桌宠或托盘菜单中的「切换角色」可热切换形象。
374 | - 右键菜单中「动画 • 待机」和「动画 • 转向」已拆分为两个独立按钮。
375 |
376 | ### 动作分类规则
377 |
378 | 程序按以下优先级区分“待机 / 转向 / 移动 / 点击回应 / 拖拽 / 随机动作”:
379 |
380 | 1. **优先按 `videos/` 下的子目录分类**:
381 |   - `idle/` → 待机
382 |   - `turn/` → 转向
383 |   - `move/` → 移动
384 |   - `click/` → 点击回应
385 |   - `drag/` → 拖拽 (可选)
386 |   - `random/` → 随机动作
387 |   - 放在这些子目录之外的 webm 会进入随机动作池。
388 |   - 兼容旧结构: 如果存在 `idle_turn/`, 程序仍会尝试按文件名关键词拆分待机和转向。
389 |
390 | 2. **可选 `manifest.json` 补充/覆盖**:
391 |   - 查找位置:
392 |     - `<角色目录>/videos/manifest.json`
393 |     - `<角色目录>/manifest.json`
394 |   - 示例:
395 |     ```json
396 |     {
397 |       "idle": "我的待机.webm",
398 |       "turn": "转身动画.webm",
399 |       "moves": ["走路1.webm", "走路2.webm"],
400 |       "clicks": ["点击回应.webm"],
401 |       "drag": "拖拽动画.webm"
402 |     },
403 |     ```
404 |   - 当子目录无法满足精确分类时, 可以用 manifest 指定。
405 |
406 | 3. **关键词兜底**:
407 |   - 待机: 包含 `待机`、`idle`、`呼吸`
408 |   - 转向: 包含 `转向`、`转身`、`东张西望`、`回头`、`turn`
409 |   - 移动: 包含 `走`、`跑`、`移动`、`move`、`walk`、`run`、`踏步`、`奔跑`
410 |   - 点击回应: 包含 `点击`、`回应`、`click`、`response`
411 |   - 拖拽: 包含 `拖拽`、`拖`、`悬空`、`drag`、`抓`
412 |
413 | 4. 如果某个角色没有可用的“待机”, 程序会安全回退到该角色第一个动画, 避免启动崩溃。
414 |
415 | > 建议: 新角色推荐直接使用 `idle/ turn/ move/ click/ drag/ random/` 子目录结构,
416 | > 这样不需要 manifest 也能正确分类。
417 |
418 | ## 快速开始
419 |
420 | ### 1. 安装依赖
421 |
422 | ```sh
423 | pip install PySide6 imageio-ffmpeg
424 | ```
425 |
426 | ### 2. 准备素材
427 |
428 | 请从上游 [dsh-pet](https://github.com/PC2005-cloud/dsh-pet) 仓库获取
429 | `dsh-pet/assets/thumb/*.webm` (91 个 640×360 透明 webm), 按分类放到本项目的

```

```

430 | `assets/characters/shenshen/videos/` 下对应子目录:
431 |
432 | ```text
433 | assets/characters/shenshen/videos/
434 | |___ idle/
435 | |___ turn/
436 | |___ move/
437 | |___ click/
438 | |___ drag/
439 | |___ random/
440 | ```
441 |
442 | 无需转码即可直接运行。
443 |
444 | ### 3. 运行
445 |
446 | 双击 `run.bat`, 或命令行 `python -m pet`。
447 |
448 | ## 打包为 exe (可选)
449 |
450 | ```bat
451 | python -m PyInstaller --noconfirm --clean --onefile --windowed --noupx ^
452 |     --name dsh-pet-standalone-webm ^
453 |     --collect-all imageio_ffmpeg ^
454 |     --add-data "assets/characters;assets/characters" ^
455 |     packaging/pet_entry.py
456 | ```
457 |
458 | > 打包入口必须用 `packaging/pet_entry.py` (绝对导入); 直接用 `pet/__main__.py`
459 | > 会因相对导入在冻结模式下失效。onefile 模式启动时会先解压素材 (约 5~15 秒)。
460 |
461 | ##### GIF 版打包 (可选)
462 |
463 | 如果需要打包旧 GIF 素材版本, 使用:
464 |
465 | ```bat
466 | python -m PyInstaller --noconfirm --clean --onefile --windowed --noupx ^
467 |     --name dsh-pet-standalone-gif ^
468 |     --collect-all imageio_ffmpeg ^
469 |     --add-data "assets/characters_gif;assets/characters_gif" ^
470 |     packaging/pet_entry.py
471 | ```
472 |
473 | 程序会按素材目录自动识别:
474 |
475 | - 目录里有 `*.webm` → 使用 webm 播放
476 | - 目录里只有 `*.gif` → 使用 GIF/QMovie 播放
477 |
478 | ## macOS
479 |
480 | ### 安装包 (.app, GitHub Actions 自动构建)
481 |
482 | 无需本地 Mac, 项目通过 GitHub Actions 自动打包 macOS 版本:
483 |
484 | 1. 打开仓库的 **Actions** 页 → 左侧选「Build macOS App」→ 右侧 **Run workflow** → 确认
485 | 2. 等待构建完成 (约 10 分钟), 在构建详情页底部下载 artifact:
486 |    `dsh-pet-indesktop-macos-arm64.zip` (Apple Silicon / M 系列芯片)
487 | 3. 解压得到 `dsh-pet-indesktop.app`, 拖入「应用程序」文件夹即可
488 |
489 | > **Intel Mac 用户**: 当前仅提供 Apple Silicon (arm64) 安装包, Intel 芯片的 Mac
490 | > 请用下方「源码运行」方式使用。
491 |
492 | > **未签名提示**: 目前为免费版 (ad-hoc 签名, 未经 Apple 公证), 首次打开会被 macOS
493 | > Gatekeeper 拦截。放行方法 (任选其一, <你的app路径> 改成实际位置):
494 | >
495 | > 1. 右键 app → 「打开」→ 再点「打开」; 若无「打开」选项, 走第 2 条
496 | > 2. 系统设置 → 隐私与安全性 → 下滑找到「已阻止 `dsh-pet-indesktop`」→ 点「仍然打开」
497 | > 3. 终端清除隔离标记后双击 (最常用):
498 | > ```sh
499 | > sudo xattr -d com.apple.quarantine "<你的app路径>/dsh-pet-indesktop.app"
500 | > ```
501 | > 4. 若第 3 条仍被拦, 再补 ad-hoc 签名后双击:
502 | > ```sh
503 | > xattr -cr "<你的app路径>/dsh-pet-indesktop.app"
504 | > codesign --force --deep --sign - "<你的app路径>/dsh-pet-indesktop.app"
505 | > ```
506 |
507 | ### 源码运行
508 |

```

```

509 | ``sh
510 | pip install PySide6 imageio-ffmpeg
511 | # 准备素材：同「快速开始」第 2 步，把 webm 放到 assets/characters/shenshen/videos/
512 | python -m pet
513 | ``
514 |
515 | ### macOS 已知差异
516 |
517 | - **开机自启**：macOS 通过 LaunchAgents（~/Library/LaunchAgents/）实现，与 Windows 注册表等价
518 | - **透明穿透**：Qt 的窗口 mask 鼠标穿透在 macOS 上行为与 Windows 有差异，透明区域点击穿透**可能不完全生效**，需真机验证反馈
519 | - **配置目录**：macOS 下配置保存在 ~/Library/Application Support/dsh-pet-standalone/
520 |
521 | ## 目录结构
522 |
523 | ```
524 | | pet/          # 核心代码
525 | | | catalog.py  # 动画目录、多形象常量、分类、几何/概率常量
526 | | | library.py  # 素材索引 + WebM/GIF 播放器按需加载
527 | | | webm_clip.py # imageio-ffmpeg 解码 webm 的播放器
528 | | | performance.py # 系统负载采样与省资源模式迟滞策略
529 | | | window.py   # 桌面窗口：状态机 + 动画链 + 移动驱动 + 交互
530 | | | config.py   # 配置持久化（跨平台：APPDATA / Application Support / .config）
531 | | | autostart.py # 开机自启（跨平台：Windows 注册表 / macOS LaunchAgents）
532 | | | app.py      # 入口 + 系统托盘
533 | | assets/characters/ # 多形象动画（每个角色一个子目录）
534 | | | <character_id>/videos/*.webm
535 | | packaging/      # PyInstaller 打包入口
536 | | tools/           # 超分、透明 WebM 运行代理生成与逐文件验收
537 | | .github/workflows/ # GitHub Actions（macOS 自动打包）
538 | | tests/           # 冒烟测试 / 诊断工具
539 | | run.bat          # Windows 一键启动
540 | | requirements.txt # PySide6 + imageio-ffmpeg
541 | ```
542 |
543 | ## 已知说明
544 |
545 | **媒体分层**：`MovieLibrary` 先建立 `MediaSource` 轻量索引，窗口首次切换到动作时才创建
546 | 播放器；退出和角色切换由窗口统一关闭媒体库，避免后台 reader 残留。
547 |
548 | **webm 有界解码**：内置 1440p 母版派生的 1280×720 VP9 Alpha 运行代理；
549 | 默认 462px 档解码到 922×520，最大 640px 档才使用完整代理，覆盖 2× Retina。
550 | GIF 只作为单独导出包，不放进应用。
551 |
552 | ## 开发经验与教训
553 |
554 | > 记录本项目开发与打包过程中踩过的坑，供后续维护者参考。
555 |
556 | ### 打包与分发
557 |
558 | - **CI 打包成功 ≠ 能运行**：macOS 的 GitHub Actions 构建曾“绿色成功”，但产物缺 `pet`
559 | 模块（运行时才报 `ModuleNotFoundError`）。原因：`pyinstaller` 命令不会把当前目录加进
560 | 模块搜索路径，从 `packaging/pet_entry.py` 入口分析时找不到项目根的 `pet` 包。必须用
561 | `python -m PyInstaller --paths .`。教训：打包后在 CI 里加验证步骤（检查 warn 文件 /
562 | 解压检查权限），别只看绿灯。
563 | - **zip 会丢 macOS 可执行权限**：`zip -r` 打包 .app 后解压，二进制丢失 +x 权限，双击
564 | 无反应、终端 `permission denied`。改用 macOS 原生 `ditto -c -k --keepParent` 打包。
565 | - **未签名 app 必被 Gatekeeper 拦**：免费版加 ad-hoc 签名
566 | （`codesign --force --deep --sign -`）后，「右键打开 / 系统设置放行」可用；彻底免
567 | 拦截需 Apple 开发者账号公证（$99/年）。放行方法见上文 [macOS] 章节。
568 | - **打包前先关掉正在运行的桌宠进程**：Windows 打包时若旧 exe 进程存活，PyInstaller
569 | 覆盖产物会报 `PermissionError: 拒绝访问`；webm 版 exe 约 340MB，但仍需结束进程后重试。
570 |
571 | ### macOS 平台特性
572 |
573 | - **Tool 窗口置顶** `WA_MacAlwaysShowToolWindow`：macOS 上 Qt 的
574 | `WindowStaysOnTopHint` 对 `Tool` 窗口不可靠（Qt 官方已知问题 QTBUG-38580），正确
575 | 做法是设置 `Qt.WidgetAttribute.WA_MacAlwaysShowToolWindow`；原生 `NSWindow.setLevel`
576 | 需等窗口重建完成后（`QTimer.singleShot(0)`）再调，否则被 Qt 覆盖。
577 | - **ctypes 调 ObjC 必须显式声明 restype**：`sel_registerName` 返回 64 位 SEL 指针，
578 | 不设 `restype = c_void_p` 会被 ctypes 默认按 32 位截断，损坏的 SEL 使 ObjC runtime
579 | 段错误（SIGSEGV，try/except 拦不住）。任何返回指针的 C 函数都要显式声明返回类型。
580 | - **屏幕坐标比例要减 `availableGeometry` 的 left/top**：macOS 上
581 | `availableGeometry().top()` 等于菜单栏高度（≠0），按「窗口坐标 ÷ 可用区宽高」存比例
582 | 会偏一个菜单栏高度；正确做法是 `(坐标 - avail.left()/top()) / avail.width()/height()`。
583 |
584 | ### 素材与播放
585 |
586 | - **ffmpeg 解码透明 webm 时 `-c:v libvp9-vp9` 必须放在 `-i` 之前**：ffmpeg 原生 vp9
587 | 解码器会丢弃 WebM alpha 通道（上游 DESIGN.md 踩坑记录第 3 条，已实测复现）。

```

```
588 | - **webm 播放不能“清空队列只取最新帧”**：如果每次刷新都丢弃中间帧，动画会像快进一样。
589 | 正确做法是按视频 fps 逐帧取最早的一帧。
590 | - **播放结束标记不能误停新动画**：最后一帧触发窗口层切换动画后，旧的结束标记不应再
591 | 停止新动画的定时器；否则会出现“播完一个动画后卡住不动”。
592 | - **Windows 下 ffmpeg 子进程要隐藏控制台**：使用 `imageio_ffmpeg` 自带的
593 | `STARTUPINFO` 或显式 `CREATE_NO_WINDOW`，避免窗口反复出现/消失。
594 |
595 | ### 验证
596 |
597 | - **真机验证不可替代**：macOS 专属代码路径（ctypes/ObjC、窗口置顶）在 Windows 上编译
598 | 与冒烟测试都覆盖不到，必须真机验证；诊断日志（恢复位置/回到右下角时记录
599 | availableGeometry 与 DPR）就是为此加的。
600 |
601 | ## 附录：旧版 GIF/QMovie 路线（已归档）
602 |
603 | > 当前版本已改为 **webm 直解路线**，以下为旧版 GIF/QMovie 路线的完整说明，仅作参考存档，不再使用。
604 |
605 | <details>
606 | <summary>点击查看旧版 GIF 版本说明</summary>
607 |
608 | ### 已移除/变更记录
609 |
610 | - 内置角色 `guga`、`dada`、`suansuan`、`dudu`、`mimi` 已移除（动画未能正常绘制）。当前内置角色仅保留 `shenshen`。
611 | - 自定义角色仍可通过 exe 同目录或用户数据目录下的 `characters/<id>/videos/` 添加，并在「切换角色」菜单中热切换。
612 |
613 | ——
614 |
615 | # dsh-pet-indesktop
616 |
617 | ## 下载
618 |
619 | 无需从源码构建，直接到 [Releases](https://github.com/MerZlin/dsh-pet-indesktop/releases)
620 | 页面下载对应系统的安装包：
621 |
622 | | 你的系统 | 安装包 | 说明 |
623 | |-----|-----|-----|
624 | | Windows | `dsh-pet-standalone.exe` | 双击即跑，首次启动解压需几秒 |
625 | | macOS (Apple Silicon / M 系列) | `dsh-pet-indesktop-macos-arm64.zip` | 解压得 `dsh-pet-indesktop.app`，首次打开需放行（见下方「macOS」章节） |
626 | | macOS (Intel) | — | 暂无安装包，请按「macOS」章节源码运行 |
627 |
628 | > 文件名以 Release 页面实际发布为准。
629 |
630 | > **声明与致谢**：本项目改自、源于 [dsh-pet](https://github.com/PC2005-cloud/dsh-pet)。
631 | > 桌宠的动画素材、动画链行为模型、交互设计均来自原项目，特此声明并感谢原作者的贡献。
632 |
633 | 把 [dsh-pet](https://github.com/PC2005-cloud/dsh-pet) 插件里的桌宠，改造成一个
634 | **跨平台的独立桌面宠物**软件（支持 **Windows** 与 **macOS**）—— 不依赖 DSH 运行时，
635 | 用 Python + PySide6 实现，双击即跑，内置 91 段原作 640×360 手工透明动画（24fps）。
636 |
637 | Windows 用户见下方「快速开始 / 打包为 exe」，macOS 用户见「macOS」章节。
638 |
639 | ## 特性
640 |
641 | - **webm 原作播放**：直接运行时解码 640×360 透明 webm（VP9 + 8-bit alpha），默认显示无需放大
642 | - **动画链**：每个动画播完按概率选下一个 —— 30% 待机 / 10% 转向 / 40% 随机动作 / 20% 移动，永不停止
643 | - **多形象支持**：内置多个角色，并支持用户通过外部目录添加自定义角色
644 | - **角色热切换**：右键桌宠或托盘菜单可随时切换形象，无需重启
645 | - **屏幕漫游**：朝面向方向行走，先检查屏幕空间、不走出屏幕（移动动画前后各 2s 准备/收尾，位置由代码驱动）
646 | - **左右朝向**：转向动画播完翻转朝向，所有动画支持水平镜像
647 | - **点击回应**：点击宠物随机播放当前角色配置的回应动画（链上非待机动画播放中不打断）
648 | - **拖拽**：按住拖动超过 5px 判定为拖拽，宠物播放「悬空反馈」动画跟手，松手停在原地
649 | - **透明穿透**：窗口逐帧按人物 alpha 生成 mask，透明区域鼠标直接穿透到下层窗口
650 | - **右键菜单**：手动播放待机/转向/移动/点击回应/随机动作、切换角色、回到右下角、窗口置顶、不移动、开机自启、4 档大小、退出
651 | - **系统托盘**：显示/隐藏、切换角色、开机自启、退出；位置/朝向/大小/置顶自动持久化
652 | - **开机自启**：Windows 写 HKCU 注册表 Run 键 / macOS 写 LaunchAgents（均无需管理员权限），可随时开关
653 |
654 | ## 使用手册
655 |
656 | ### 启动与退出
657 |
658 | - **启动**：双击 `run.bat`（或 `python -m pet` / 打包后的 exe），桌宠出现在屏幕右下角
659 | - **退出**：右键桌宠 → 「退出」，或点系统托盘图标 → 「退出」
660 |
661 | ### 鼠标交互
662 |
663 | - **点击**：单击宠物本体，随机播放当前角色配置的回应动画之一。
664 | 链上非待机动画播放中点击不会打断
665 | - **拖拽**：按住拖动超过 5px 判定为拖拽，宠物播放「悬空反馈」动画跟手，松手停在原地
666 | - **穿透**：只有宠物本体（不透明区域）可点，其余透明区域鼠标直接穿透到下层窗口
```

```

667 |
668 | ### 右键菜单（右键点击宠物本体）
669 |
670 | 菜单项 | 功能
671 | -----|-----
672 | 动画 · 待机 | 手动播放待机动画：如果待机目录有多个视频，会显示二级菜单
673 | 动画 · 转向 | 手动播放转向动画：如果转向目录有多个视频，会显示二级菜单
674 | 动画 · 移动 | 手动播放移动动画（走路姿态 + 朝面向方向真实走动；「不移动」模式下这是唯一触发移动的方式）
675 | 动画 · 点击回应 | 手动播放点击回应动画
676 | 动画 · 随机动作 | 手动播放随机动作动画
677 | 切换角色 | 热切换当前形象（内置 + 外部扩展角色都会列出）
678 | 回到右下角 | 把宠物复位到屏幕右下角
679 | 窗口置顶 | 勾选 = 始终显示在其他窗口之上，取消 = 可被其他窗口遮挡
680 | 不移动 | 勾选 = 只播放原地动画（待机/转向/随机动作），不再自动走动；取消 = 恢复正常模式
681 | 开机自启 | 勾选 = 随 Windows 登录自动启动（见下方说明）
682 | 大小 | 4 档缩放：320px / 462px / 544px / 640px（默认 462px）
683 | 退出 | 关闭桌宠
684 |
685 | ### 系统托盘图标
686 |
687 | 操作 | 功能
688 | -----|-----
689 | 双击托盘图标 | 显示 / 隐藏宠物
690 | 右键托盘图标 → 显示/隐藏 | 同上
691 | 右键托盘图标 → 切换角色 | 热切换当前形象
692 | 右键托盘图标 → 开机自启 | 与右键菜单的开机自启同步
693 | 右键托盘图标 → 退出 | 关闭桌宠
694 |
695 | ### 自动行为（无需任何操作）
696 |
697 | 桌宠会自己“生活”，挂机即可观赏：
698 |
699 | - **动画链**：每个动画播完按概率抽下一个 —— 30% 待机 / 10% 转向 / 40% 随机动作 / 20% 移动，永不停止
700 | - **屏幕漫游**：朝面向方向行走，先检查屏幕空间、不走出屏幕
701 | - **朝向翻转**：播完「东张西望」后左右翻转朝向，所有动画随之镜像
702 | - **状态记忆**：位置、朝向、大小、置顶、不移动、开机自启、当前角色均自动保存（`%APPDATA%\dsh-pet-standalone/config.json`），下次启动自动恢复
703 |
704 | ### 开机自启
705 |
706 | 勾选「开机自启」后，桌宠随系统登录自动启动；取消勾选即移除。
707 |
708 | - **Windows**：写入注册表 `HKCU\Software\Microsoft\Windows\CurrentVersion\Run`（无需管理员权限）；
709 | 源码运行时指向 `pythonw -m pet`；打包成 exe 后会先用 `start /D` 切到 exe 所在目录再启动 exe，
710 | 避免开机时默认工作目录不可写导致 onefile 解压失败（旧版自启命令会在下次启动时自动升级）
711 | - **macOS**：写入 LaunchAgents（`~/Library/LaunchAgents/`），登录后自动启动
712 |
713 | ## 实现方式
714 |
715 | ### 技术栈
716 |
717 | - **Python 3.10+ / PySide6**（Qt for Python, LGPL 许可）
718 | - **QMovie** 播放 GIF，运行时零外部依赖（无需 ffmpeg）
719 |
720 | ### 动画链状态机（1:1 移植原插件 `client.js`）
721 |
722 | 原插件是一个“链式”状态机：每个动画一次性播放，播完按概率抽下一个
723 | （30% 待机 / 10% 转向 / 40% 动作 / 20% 移动）。本项目的 `pet/window.py`
724 | 将这套行为完整移植为 Python：
725 |
726 | - `pick_next()` 按概率抽下一个动画
727 | - 转向动画播完翻转朝向（`facing=right` 时水平镜像，等效原版 `scaleX(-1)`）
728 | - 移动动画只提供“走路姿态”，位置由 `QTimer` 驱动：开头/结尾各 2s 原地不动，
729 | 中间按播放进度插值位移
730 | - 点击回应/拖拽动画播完先回“待机缓冲”，待机播完再回到随机链
731 | - 点击只在待机时响应，5px 阈值区分点击与拖拽（等效原版命中层设计）
732 |
733 | ### 透明窗口与鼠标穿透
734 |
735 | 窗口用 Qt 的 `QTool` 无边框置顶窗口 + `WGL_TranslucentBackground` 实现透明背景。
736 | 每帧按人物 alpha 通道生成 `QBitmap` mask，透明区域鼠标直接穿透到下层窗口，
737 | 实现“只有宠物本体可点击”（等效原版 HIT_BOX 命中层）。
738 |
739 | ### 素材转码（webm → GIF）
740 |
741 | 原项目的播放资源是 640×360 透明 **webm**（VP9 + 8-bit alpha）：原始 1280×720
742 | MP4 是非透明素材。本项目可把 WebM 另行导出为同尺寸透明 **GIF**，但应用默认仍使用 WebM。
743 | `scripts/convert.py` 的关键点是 `-c:v libvpx-vp9` 必须放在 `-i` 之前
744 | ffmpeg 原生 vp9 解码器会丢弃 alpha（原项目 DESIGN.md 踩坑记录第 3 条，已实测复现）。
745 |

```

```

746 | ## 快速开始
747 |
748 | ### 1. 安装依赖
749 |
750 | ```sh
751 | pip install PySide6
752 | ```
753 |
754 | ### 2. 准备素材
755 |
756 | 素材体积较大（GIF 392MB，未随仓库分发）。请从上游
757 | [dsh-pet](https://github.com/PC2005-cloud/dsh-pet) 仓库获取
758 | `dsh-pet/assets/thumb/*.webm`（91 个 640×360 透明 webm），放到本项目的
759 | `assets/videos/` 目录，然后转码：
760 |
761 | ```sh
762 | pip install imageio-ffmpeg pillow
763 | python scripts/convert.py # 默认读 assets/videos/，输出 assets/animations/
764 | # 或指定源目录：python scripts/convert.py --src <你的webm目录>
765 | ```
766 |
767 | ### 3. 运行
768 |
769 | 双击 `run.bat`，或命令行 `python -m pet`。
770 |
771 | ## 打包为 exe（可选）
772 |
773 | ```bat
774 | python -m PyInstaller --noconfirm --clean --onefile --windowed ^
775 |     --name dsh-pet-indesktop ^
776 |     --add-data "assets/animations;assets/animations" ^
777 |     packaging/pet_entry.py
778 | ```
779 |
780 | > 打包入口必须用 `packaging/pet_entry.py`（绝对导入）：直接用 `pet/_main_.py`
781 | > 会因相对导入在冻结模式下失效。onefile 模式启动时会先解压素材（约 5~15 秒）。
782 | >
783 | > `--runtime-tmpdir "."` 是按“进程当前工作目录”解析的，不是 exe 所在目录。
784 | > 程序内部的开机自启会先切到 exe 所在目录，因此解压目录会生成在 exe 同目录；
785 | > 若 exe 位于系统保护目录（如 `Program Files`）且无写权限，请把 exe 放到用户可写目录。
786 |
787 | ## macOS
788 |
789 | ### 安装包（.app, GitHub Actions 自动构建）
790 |
791 | 无需本地 Mac，项目通过 GitHub Actions 自动打包 macOS 版本：
792 |
793 | 1. 打开仓库的 **Actions** 页 → 左侧选「Build macOS App」→ 右侧 **Run workflow** → 确认
794 | 2. 等待构建完成（约 10 分钟），在构建详情页面底部下载 artifact：
795 | `dsh-pet-indesktop-macos-arm64.zip`（Apple Silicon / M 系列芯片）
796 | 3. 解压得到 `dsh-pet-indesktop.app`，拖入「应用程序」文件夹即可
797 |
798 | > **Intel Mac 用户**：当前仅提供 Apple Silicon（arm64）安装包，Intel 芯片的 Mac
799 | > 请用下方「源码运行」方式使用。
800 |
801 | > **未签名提示**：目前为免费版（ad-hoc 签名，未经 Apple 公证），首次打开会被 macOS
802 | > Gatekeeper 拦截。放行方法（任选其一，`<你的app路径>` 改成实际位置）：
803 | >
804 | > 1. 右键 app → 「打开」→ 再点「打开」：若无「打开」选项，走第 2 条
805 | > 2. 系统设置 → 隐私与安全性 → 下滑找到「已阻止 `dsh-pet-indesktop`」→ 点「仍然打开」
806 | > 3. 终端清除隔离标记后双击（最常用）：
807 | > ```sh
808 | > sudo xattr -d com.apple.quarantine "<你的app路径>/dsh-pet-indesktop.app"
809 | > ```
810 | > 4. 若第 3 条仍被拦，再补 ad-hoc 签名后双击：
811 | > ```sh
812 | > xattr -cr "<你的app路径>/dsh-pet-indesktop.app"
813 | > codesign --force --deep --sign - "<你的app路径>/dsh-pet-indesktop.app"
814 | > ```
815 |
816 | ### 源码运行
817 |
818 | ```sh
819 | pip install PySide6
820 | # 准备素材：同「快速开始」第 2 步，把 webm 放到 assets/videos/ 后转码
821 | pip install imageio-ffmpeg pillow
822 | python scripts/convert.py
823 | python -m pet
824 | ```

```

```

825 |
826 | ### macOS 已知差异
827 |
828 | - **开机自启**: macOS 通过 LaunchAgents(`~/Library/LaunchAgents/`)实现, 与 Windows 注册表等价
829 | - **透明穿透**: Qt 的窗口 mask 鼠标穿透在 macOS 上行为与 Windows 有差异, 透明区域点击穿透**可能不完全生效**, 需真机验证反馈
830 | - **配置目录**: macOS 下配置保存在 `~/Library/Application Support/dsh-pet-standalone/`
831 |
832 | ## 目录结构
833 |
834 | ```
835 | | pet/          # 核心代码
836 | | | catalog.py  # 91 段动画目录、分类、几何/概率常量
837 | | | library.py  # QMovie 素材库 (速度补偿)
838 | | | window.py   # 桌宠窗口: 状态机 + 动画链 + 移动驱动 + 交互
839 | | | config.py   # 配置持久化 (跨平台: APPDATA / Application Support / .config)
840 | | | autostart.py # 开机自启 (跨平台: Windows 注册表 / macOS LaunchAgents)
841 | | | app.py       # 入口 + 系统托盘
842 | | | scripts/convert.py # 素材转码: webm → 透明 GIF
843 | | | packaging/   # PyInstaller 打包入口
844 | | | .github/workflows/ # GitHub Actions (macOS 自动打包)
845 | | | tests/        # 冒烟测试 / 帧率实测 / 诊断工具
846 | | | run.bat        # Windows 一键启动
847 | | | requirements.txt # PySide6
848 | | ```
849 |
850 | ## 已知说明
851 |
852 | **清晰度略糊于 web 端**: web 端直接播放 640×360 透明 webm (VP9 视频, 8-bit alpha),
853 | 而本项目的 GIF 受格式本身限制 —— ① 只支持 **1-bit alpha** (每像素要么全透明要么
854 | 全不透明, 无半透明过渡, 发丝边缘略硬); ② 最多 **256 色调色板** (有损颜色量化)。
855 | 分辨率与帧率与 web 端一致 (640×360 / 24fps), 但颜色与边缘过渡略逊, 属 GIF 格式的
856 | 固有限制。若追求与 web 端完全一致的画质, 可改用运行时 ffmpeg 解码 webm 的方案。
857 |
858 | ## 开发经验与教训
859 |
860 | > 记录本项目开发与打包过程中踩过的坑, 供后续维护者参考。
861 |
862 | ### 打包与分发
863 |
864 | - **CI 打包成功 ≠ 能运行**: macOS 的 GitHub Actions 构建曾“绿色成功”, 但产物缺 `pet`
865 | 模块 (运行时才报 `ModuleNotFoundError`)。原因: `pyinstaller` 命令不会把当前目录加进
866 | 模块搜索路径, 从 `packaging/pet_entry.py` 入口分析时找不到项目根的 `pet` 包。必须用
867 | `python -m PyInstaller --paths .`。教训: 打包后在 CI 里加验证步骤 (检查 warn 文件 /
868 | 解压检查权限), 别只看绿灯。
869 | - **zip 会丢 macOS 可执行权限**: `zip -r` 打包 .app 后解压, 二进制丢失 +x 权限, 双击
870 | 无反应、终端 `permission denied`。改用 macOS 原生 `ditto -c -k --keepParent` 打包。
871 | - **未签名 app 必被 Gatekeeper 拦**: 免费版加 ad-hoc 签名
872 |   (`codesign --force --deep --sign -`) 后, [右键打开 / 系统设置放行] 可用; 彻底免
873 | 拦截需 Apple 开发者账号公证 ($99/年)。放行方法见上文 [macOS] 章节。
874 | - **打包前先关掉正在运行的桌宠进程**: Windows 打包时若旧 exe 进程存活 (或杀毒软件
875 | 正在扫描 400MB 大文件), PyInstaller 覆盖产物会报 `PermissionError: 拒绝访问`,
876 | 需结束进程并等扫描结束后重试。
877 |
878 | ### macOS 平台特性
879 |
880 | - **Tool 窗口置顶用 `WA_MacAlwaysShowToolWindow`**: macOS 上 Qt 的
881 | `WindowStaysOnTopHint` 对 `Tool` 窗口不可靠 (Qt 官方已知问题 QTBUG-38580), 正确
882 | 做法是设置 `Qt.WidgetAttribute.WA_MacAlwaysShowToolWindow`。原生 `NSWindow.setLevel`
883 | 需等窗口重建完成后 (`QTimer.singleShot(0)`) 再调, 否则被 Qt 覆盖。
884 | - **ctypes 调 ObjC 必须显式声明 restype**: `sel_registerName` 返回 64 位 SEL 指针,
885 | 不设 `restype = c_void_p` 会被 ctypes 默认按 32 位截断, 损坏的 SEL 使 ObjC runtime
886 | 段错误 (SIGSEGV, try/except 拦不住)。任何返回指针的 C 函数都要显式声明返回类型。
887 | - **屏幕坐标比例要减 `availableGeometry` 的 left/top**: macOS 上
888 | `availableGeometry().top()` 等于菜单栏高度 (≠0), 按 [窗口坐标 ÷ 可用区宽高] 存比例
889 | 会偏一个菜单栏高度; 正确做法是 `(坐标 - avail.left()/top()) / avail.width()/height()`。
890 |
891 | ### 素材与播放
892 |
893 | - **ffmpeg 转码透明 webm 时 `-c:v libvpx-vp9` 必须放在 `-i` 之前**: ffmpeg 原生 vp9
894 | 解码器会丢弃 WebM alpha 通道 (上游 DESIGN.md 踩坑记录第 3 条, 已实测复现)。
895 | - **QMovie 播放 GIF 偏慢约 20%**: QMovie 的定时器 + 解码开销使每帧比 GIF 原生时间长慢,
896 | 需 `setSpeed(120)` 校准 (见 `pet/library.py` 的 `PLAYBACK_SPEED`)。
897 |
898 | ### 验证
899 |
900 | - **真机验证不可替代**: macOS 专属代码路径 (ctypes/ObjC、窗口置顶) 在 Windows 上编译
901 | 与冒烟测试都覆盖不到, 必须真机验证: 诊断日志 (恢复位置/回到右下角时记录
902 | availableGeometry 与 DPR) 就是为此加的。
903 |

```

```

904 | </details>
905 |
906 | ## 许可与致谢
907 |
908 | - 本项目的 Python 代码为独立实现，采用 **MIT** 许可。
909 | - 动画素材版权与许可归属原项目 [dsh-pet] (https://github.com/PC2005-cloud/dsh-pet) (MIT)。
910 | - 再次感谢 [PC2005-cloud/dsh-pet] (https://github.com/PC2005-cloud/dsh-pet) 原作者。

```

03. pet/__init__.py (4 lines)

```

1 | # -*- coding: utf-8 -*-
2 | """dsh-pet 独立桌宠（不依赖 DSH 运行时的 Windows 桌面宠物）。"""
3 |
4 | __version__ = '0.1.0'

```

04. pet/__main__.py (8 lines)

```

1 | # -*- coding: utf-8 -*-
2 | """python -m pet 入口。"""
3 |
4 | import sys
5 |
6 | from .app import main
7 |
8 | sys.exit(main())

```

05. pet/action_dialog.py (61 lines)

```

1 | # -*- coding: utf-8 -*-
2 | """动作收藏夹/播放列表的轻量选择窗口。"""
3 |
4 | from __future__ import annotations
5 |
6 | from PySide6.QtCore import Qt
7 | from PySide6.QtWidgets import (
8 |     QDialog,
9 |     QDialogButtonBox,
10 |     QLabel,
11 |     QListWidget,
12 |     QListWidgetItem,
13 |     QVBoxLayout,
14 | )
15 |
16 |
17 | class ActionSetDialog(QDialog):
18 |     """用系统复选列表编辑一组动作，返回用户勾选的动作名。"""
19 |
20 |     def __init__(
21 |         self,
22 |         title: str,
23 |         names: list[str],
24 |         selected: list[str],
25 |         parent=None,
26 |     ) -> None:
27 |         super().__init__(parent)
28 |         self.setWindowTitle(title)
29 |         self.setMinimumSize(360, 480)
30 |
31 |         layout = QVBoxLayout(self)
32 |         layout.addWidget(QLabel("勾选动作后点击“完成”：播放列表会按当前顺序循环或随机播放。"))
33 |
34 |         self.list_widget = QListWidget(self)
35 |         selected_set = set(selected)
36 |         for name in names:
37 |             item = QListWidgetItem(name, self.list_widget)
38 |             item.setFlags(item.flags() | Qt.ItemFlag.ItemIsUserCheckable)
39 |             item.setCheckState(
40 |                 Qt.CheckState.Checked
41 |                 if name in selected_set
42 |                 else Qt.CheckState.Unchecked

```



```

43 |         )
44 |         layout.addWidget(self.list_widget)
45 |
46 |         buttons = QDialogButtonBox(
47 |             QDialogButtonBox.StandardButton.Ok
48 |             | QDialogButtonBox.StandardButton.Cancel,
49 |             parent=self,
50 |         )
51 |         buttons.accepted.connect(self.accept)
52 |         buttons.rejected.connect(self.reject)
53 |         layout.addWidget(buttons)
54 |
55 |         def selected_names(self) -> list[str]:
56 |             names: list[str] = []
57 |             for index in range(self.list_widget.count()):
58 |                 item = self.list_widget.item(index)
59 |                 if item.checkState() == Qt.CheckState.Checked:
60 |                     names.append(item.text())
61 |             return names

```

06. pet/app.py (338 lines)

```

1 | # -*- coding: utf-8 -*-
2 | """
3 | 应用入口 —— QApplication + 桌面窗口 + 系统托盘。
4 |
5 | 支持运行时切换角色:
6 | - 右键桌宠 → 「切换角色」
7 | - 托盘菜单 → 「切换角色」
8 | 切换后会热加载对应形象的 webm, 并保留位置/朝向等配置。
9 | """
10 |
11 | from __future__ import annotations
12 |
13 | import logging
14 | import shutil
15 | import sys
16 | import time
17 | from pathlib import Path
18 |
19 | from PySide6.QtCore import QApplication, QTimer
20 | from PySide6.QtGui import QIcon
21 | from PySide6.QtWidgets import QApplication, QMenu, QMessageBox, QSystemTrayIcon
22 |
23 | from . import autostart as autostart_mod
24 | from . import catalog
25 | from .config import Config
26 | from .hotkeys import GlobalHotkeys
27 | from .library import MovieLibrary
28 | from .window import PetWindow
29 |
30 |
31 | def _setup_logging(config: Config) -> None:
32 |     config.dir.mkdir(parents=True, exist_ok=True)
33 |     logging.basicConfig(
34 |         filename=str(config.dir / 'pet.log'),
35 |         level=logging.INFO,
36 |         format='%(asctime)s %(levelname)s %(message)s',
37 |         encoding='utf-8',
38 |     )
39 |
40 |
41 | def _show_startup_error(title: str, message: str) -> None:
42 |     QMessageBox.critical(None, title, message)
43 |
44 |
45 | def _resource_path(*parts: str) -> Path:
46 |     """返回源码运行与 PyInstaller 运行都可用的资源路径。"""
47 |     if getattr(sys, 'frozen', False):
48 |         root = Path(getattr(sys, '_MEIPASS', Path(sys.executable).resolve().parent))
49 |     else:
50 |         root = Path(__file__).resolve().parent.parent
51 |     return root.joinpath(*parts)
52 |
53 |
54 | def _configure_qt_plugins() -> None:

```

```

55 | """注册随应用分发的 Qt 插件目录，避免 macOS 找不到平台插件。"""
56 | plugin_root = _resource_path('qt-plugins')
57 | if plugin_root.is_dir():
58 |     QApplication.addLibraryPath(str(plugin_root))
59 |
60 |
61 | def _application_icon() -> QIcon:
62 |     icon_path = _resource_path('assets', 'app-icon.png')
63 |     return QIcon(str(icon_path)) if icon_path.is_file() else QIcon()
64 |
65 |
66 | def _cleanup_stale_runtime_dirs() -> None:
67 |     """清理 PyInstaller onefile 遗留的 _MEI* 临时目录。
68 |
69 |     注意：多开桌宠时，每个实例都有自己的 _MEI 目录，不能删除其他正在运行的实例目录。
70 |     这里只清理“很久没有被修改”的目录，避免误删其他桌宠的运行缓存。
71 |     """
72 |     if not getattr(sys, "frozen", False):
73 |         return
74 |     meipass = getattr(sys, "_MEIPASS", None)
75 |     if not meipass:
76 |         return
77 |     current = Path(meipass).resolve()
78 |     parent = current.parent
79 |     stale_age = 24 * 3600 # 只清理超过 24 小时未变化的目录
80 |     now = time.time()
81 |     for child in parent.glob("_MEI[0-9]*"):
82 |         if not child.is_dir() or child.resolve() == current:
83 |             continue
84 |         try:
85 |             mtime = child.stat().st_mtime
86 |         except OSError:
87 |             continue
88 |         if now - mtime < stale_age:
89 |             continue
90 |         try:
91 |             shutil.rmtree(child)
92 |             logging.info("已清理遗留缓存目录: %s", child)
93 |         except OSError:
94 |             logging.warning("清理遗留缓存目录失败 (可能被占用): %s", child)
95 |
96 |
97 | class PetApp:
98 |     """管理桌宠窗口、托盘与角色热切换。"""
99 |
100 |     def __init__(self, app: QApplication, config: Config) -> None:
101 |         self.app = app
102 |         self.config = config
103 |         self.win: PetWindow | None = None
104 |         self.tray: QSystemTrayIcon | None = None
105 |         self._quit_bound = False
106 |         self.hotkeys = GlobalHotkeys(self._handle_hotkey)
107 |
108 |     # ----- 启动
109 |     def start(self) -> None:
110 |         character_id = str(self.config.get('character', catalog.DEFAULT_CHARACTER))
111 |         logging.info('当前形象: %s', character_id)
112 |         if not self._quit_bound:
113 |             self.app.aboutToQuit.connect(self.shutdown)
114 |             self._quit_bound = True
115 |         self.hotkeys.start()
116 |         self._create_ui(character_id)
117 |
118 |     def _create_library(self, character_id: str) -> MovieLibrary:
119 |         scale = float(self.config.get('scale', catalog.DEFAULT_SCALE))
120 |         lib = MovieLibrary(
121 |             character_id=character_id,
122 |             decode_size=catalog.decode_size_for_scale(scale),
123 |             soft_edges=bool(self.config.get('soft_edges', True)),
124 |         )
125 |         logging.info('素材加载完成: %s %d 段动画', character_id, len(lib.names()))
126 |         return lib
127 |
128 |     def _create_ui(self, character_id: str) -> None:
129 |         lib = self._create_library(character_id)
130 |         win = PetWindow(lib, self.config)
131 |         win.on_switch_character = self.switch_character
132 |         win.show()
133 |

```

```

134 | tray = self._build_tray(win)
135 |
136 | # 清理旧对象（热切换时使用）
137 | old_win = self.win
138 | old_tray = self.tray
139 | self.win = win
140 | self.tray = tray
141 |
142 | if old_win is not None:
143 |     old_win.shutdown()
144 |     old_win.hide()
145 |     old_tray.hide() if old_tray is not None else None
146 |     QTimer.singleShot(0, old_win.deleteLater)
147 |     if old_tray is not None:
148 |         QTimer.singleShot(0, old_tray.deleteLater)
149 |
150 |
151 | # ----- 角色切换
152 | def switch_character(self, character_id: str) -> None:
153 |     if self.win is None:
154 |         return
155 |     current = str(self.config.get('character', catalog.DEFAULT_CHARACTER))
156 |     if character_id == current:
157 |         return
158 |
159 |     # 先保存配置，即使后续加载失败也记住用户选择
160 |     self.config.set('character', character_id)
161 |     self.config.save()
162 |
163 |     try:
164 |         # 预创建新库，失败则保留当前角色
165 |         lib = self._create_library(character_id)
166 |     except Exception as exc:
167 |         logging.exception('切换角色失败: %s', character_id)
168 |         _show_startup_error('切换角色失败', str(exc))
169 |         return
170 |
171 |     logging.info('切换角色: %s -> %s', current, character_id)
172 |
173 |     # 用新库创建新窗口/托盘，旧对象延迟销毁
174 |     win = PetWindow(lib, self.config)
175 |     win.on_switch_character = self.switch_character
176 |     win.show()
177 |
178 |     tray = self._build_tray(win)
179 |
180 |     old_win = self.win
181 |     old_tray = self.tray
182 |     self.win = win
183 |     self.tray = tray
184 |
185 |     old_win.hide()
186 |     old_win.shutdown()
187 |     if old_tray is not None:
188 |         old_tray.hide()
189 |     QTimer.singleShot(0, old_win.deleteLater)
190 |     if old_tray is not None:
191 |         QTimer.singleShot(0, old_tray.deleteLater)
192 |
193 |
194 | def shutdown(self) -> None:
195 |     """应用唯一退出口：避免每次热切换重复连接 aboutToQuit。"""
196 |     if self.tray is not None:
197 |         self.tray.hide()
198 |     if self.win is not None:
199 |         self.win._save_position()
200 |         self.win.shutdown()
201 |         self.hotkeys.stop()
202 |
203 | def _toggle_visible(self) -> None:
204 |     if self.win is None:
205 |         return
206 |     if self.win.isVisible():
207 |         self.win.suspend_animation()
208 |         self.win.hide()
209 |     else:
210 |         self.win.show()
211 |         self.win.resume_animation()
212 |

```

```

213 | def _handle_hotkey(self, action: str) -> None:
214 |     win = self.win
215 |     if win is None:
216 |         return
217 |     if action == 'toggle_visible':
218 |         self._toggle_visible()
219 |     elif action == 'toggle_pause':
220 |         win.toggle_pause()
221 |     elif action == 'random_action':
222 |         win.play_random_action()
223 |     elif action == 'toggle_mouse_through':
224 |         win.set_mouse_through(not win.mouse_through)
225 |     elif action == 'duck_sound':
226 |         win.trigger_duck_sound()
227 |
228 | # ----- 托盘
229 | def _build_tray(self, win: PetWindow) -> QSystemTrayIcon:
230 |     tray = QSystemTrayIcon(QIcon(win.icon_pixmap()))
231 |
232 |     menu = QMenu()
233 |     menu.addAction('显示 / 隐藏', self._toggle_visible)
234 |     pause = menu.addAction('暂停 / 继续')
235 |     pause.setCheckable(True)
236 |     pause.setChecked(win._paused)
237 |     pause.toggled.connect(win.set_paused)
238 |     win.pausedChanged.connect(pause.setChecked)
239 |     menu.addAction('随机动作', win.play_random_action)
240 |
241 |     m_char = menu.addMenu('切换角色')
242 |     current = str(self.config.get('character', catalog.DEFAULT_CHARACTER))
243 |     for cid in catalog.list_available_characters():
244 |         act = m_char.addAction(cid)
245 |         act.setCheckable(True)
246 |         act.setChecked(cid == current)
247 |         act.triggered.connect(lambda checked=False, cid=cid: self.switch_character(cid))
248 |
249 |     display = menu.addMenu('画面调整')
250 |     soft_edges = display.addAction('清理透明底噪 (Alpha=1)')
251 |     soft_edges.setCheckable(True)
252 |     soft_edges.setChecked(win.soft_edges)
253 |     soft_edges.toggled.connect(win.set_soft_edges)
254 |     win.softEdgesChanged.connect(soft_edges.setChecked)
255 |
256 |     interaction = menu.addMenu('互动反馈')
257 |     duck_sound = interaction.addAction('尖叫鸭音效')
258 |     duck_sound.setCheckable(True)
259 |     duck_sound.setChecked(win.duck_sound_enabled)
260 |     duck_sound.toggled.connect(win.set_duck_sound)
261 |     win.duckSoundChanged.connect(duck_sound.setChecked)
262 |     greetings = interaction.addAction('偶尔主动打招呼')
263 |     greetings.setCheckable(True)
264 |     greetings.setChecked(win.proactive_greetings)
265 |     greetings.toggled.connect(win.set_proactive_greetings)
266 |     win.proactiveGreetingsChanged.connect(greetings.setChecked)
267 |
268 |     win.add_action_menu(menu)
269 |     win.add_personality_menu(menu)
270 |
271 |     shortcuts = menu.addMenu('全局快捷键')
272 |     shortcut_prefix = ' '
273 |     shortcut_rows = (
274 |         ('H', '显示 / 隐藏'),
275 |         ('P', '暂停 / 继续'),
276 |         ('R', '随机动作'),
277 |         ('M', '鼠标穿透'),
278 |         ('D', '尖叫鸭'),
279 |     )
280 |     for key, label in shortcut_rows:
281 |         item = shortcuts.addAction(f'{shortcut_prefix}[key] {label}')
282 |         item.setEnabled(False)
283 |     status = '已启用' if self.hotkeys.enabled else '不可用 (仍可使用菜单)'
284 |     status_item = shortcuts.addAction(f'状态: {status}')
285 |     status_item.setEnabled(False)
286 |
287 |     mouse_through = menu.addAction('鼠标穿透')
288 |     mouse_through.setCheckable(True)
289 |     mouse_through.setChecked(bool(self.config.get('mouse_through', False)))
290 |     mouse_through.toggled.connect(win.set_mouse_through)
291 |

```

```

292 |         menu.addSeparator()
293 |
294 |         auto = menu.addAction(' 开机自启')
295 |         auto.setCheckable(True)
296 |         auto.setChecked(autostart_mod.is_enabled())
297 |         auto.toggled.connect(autostart_mod.set_enabled)
298 |
299 |         menu.addSeparator()
300 |         menu.addAction(' 退出', self.app.quit)
301 |
302 |         tray.setContextMenu(menu)
303 |         tray.setToolTip(' dsh-pet 独立桌宠')
304 |         tray.activated.connect(
305 |             lambda reason: self._toggle_visible()
306 |             if reason == QSystemTrayIcon.ActivationReason.DoubleClick
307 |             else None
308 |         )
309 |         tray.show()
310 |         return tray
311 |
312 |
313 | def main(argv: list[str] | None = None) -> int:
314 |     _configure_qt_plugins()
315 |     app = QApplication(argv if argv is not None else sys.argv)
316 |     app.setWindowIcon(_application_icon())
317 |     app.setApplicationName(' dsh-pet-standalone')
318 |     app.setQuitOnLastWindowClosed(False)
319 |
320 |     config = Config()
321 |     _setup_logging(config)
322 |     logging.info(' dsh-pet-standalone 启动')
323 |     _cleanup_stale_runtime_dirs()
324 |
325 |     controller = PetApp(app, config)
326 |     try:
327 |         controller.start()
328 |     except Exception as exc:
329 |         logging.exception(' 启动失败')
330 |         _show_startup_error(' dsh-pet-standalone', str(exc))
331 |         return 1
332 |
333 |     logging.info(' 进入事件循环')
334 |     return app.exec()
335 |
336 |
337 | if __name__ == '__main__':
338 |     sys.exit(main())

```

07. pet/autostart.py (135 lines)

```

1 | # -*- coding: utf-8 -*-
2 | """
3 | 开机自启动管理（跨平台）。
4 |
5 | - Windows: HKCU Run 注册表键（无需管理员权限）；
6 | - macOS: LaunchAgents plist（~/Library/LaunchAgents/）；
7 | - 其他平台: no-op（返回 False / 不操作）。
8 |
9 | 设计原则：**系统自启配置是唯一真相**。菜单勾选状态直接查它们，不与 config.json
10 | 冗余存储，避免两处状态不同步。
11 |
12 | 命令按运行形态自适应：
13 | - PyInstaller 打包（sys.frozen）：Windows 自启动先用 `start /D` 切到 exe 所在目录再启动 exe；
14 |   macOS/Linux 指向 .app 内二进制自身；
15 | - 源码运行：Windows 用 `pythonw -m pet`，macOS/Linux 用 `python -m pet`（带工作目录）。
16 | """
17 |
18 | from __future__ import annotations
19 |
20 | import sys
21 | from pathlib import Path
22 |
23 | APP_ID = "com.merzlin.dsh-pet-standalone"
24 | VALUE_NAME = "dsh-pet-standalone"
25 | RUN_KEY = r"Software\Microsoft\Windows\CurrentVersion\Run"
26 |

```

```

27 | _IS_WIN = sys.platform == "win32"
28 | _IS_MAC = sys.platform == "darwin"
29 |
30 | if _IS_WIN:
31 |     import winreg
32 |
33 |
34 | def _project_root() -> Path:
35 |     return Path(__file__).resolve().parent.parent
36 |
37 |
38 | def _plist_path() -> Path:
39 |     return Path.home() / "Library" / "LaunchAgents" / f"[APP_ID].plist"
40 |
41 |
42 | def _pythonw_path() -> str:
43 |     """Windows 源码运行时，取与 python.exe 同目录的 pythonw.exe（无控制台窗口）。"""
44 |     exe = sys.executable
45 |     if _IS_WIN and exe.lower().endswith("python.exe"):
46 |         return exe[: -len("python.exe")] + "pythonw.exe"
47 |     return exe
48 |
49 |
50 | def _win_command_is_current(command: str) -> bool:
51 |     """判断 Windows 自启命令是否已是“先切工作目录再启动”的新格式。"""
52 |     return "cmd /c start" in command.lower()
53 |
54 |
55 | def is_enabled() -> bool:
56 |     """当前是否已注册开机自启。"""
57 |     if _IS_WIN:
58 |         try:
59 |             with winreg.OpenKey(winreg.HKEY_CURRENT_USER, RUN_KEY) as key:
60 |                 command, _ = winreg.QueryValueEx(key, VALUE_NAME)
61 |                 # 兼容旧版，已开启但仍是旧命令（直接指向 exe，未切工作目录）时，
62 |                 # 自动升级为新命令，避免开机自启因 CWD 不可写而解压失败。
63 |                 if (
64 |                     getattr(sys, "frozen", False)
65 |                     and isinstance(command, str)
66 |                     and not _win_command_is_current(command)
67 |                 ):
68 |                     try:
69 |                         with winreg.OpenKey(
70 |                             winreg.HKEY_CURRENT_USER, RUN_KEY, 0, winreg.KEY_SET_VALUE
71 |                         ) as write_key:
72 |                             winreg.SetValueEx(
73 |                                 write_key, VALUE_NAME, 0, winreg.REG_SZ, _win_command()
74 |                             )
75 |                     except OSError:
76 |                         # 只读场景（如权限异常）不强求升级，仍视为已启用
77 |                         pass
78 |                     return True
79 |             except FileNotFoundError:
80 |                 return False
81 |     if _IS_MAC:
82 |         return _plist_path().exists()
83 |     return False
84 |
85 |
86 | def _win_command() -> str:
87 |     if getattr(sys, "frozen", False):
88 |         # onefile 的 runtime_tmpdir="" 是相对“当前工作目录”解析的。
89 |         # 开机自启（HKCU Run）默认工作目录可能是 System32 等不可写目录。
90 |         # 用 start 先切到 exe 所在目录再启动 exe，既保证解压目录在 exe 同目录，
91 |         # 又不会让 cmd 窗口一直等待桌宠退出。
92 |         exe = Path(sys.executable).resolve()
93 |         return f'cmd /c start "" /D "{exe.parent}" "{exe}"'
94 |     return f'cmd /c start "" /D "{_project_root()}" "{_pythonw_path()}" -m pet'
95 |
96 |
97 | def _mac_program_args() -> list[str]:
98 |     if getattr(sys, "frozen", False):
99 |         # .app 内二进制路径，直接作为 LaunchAgent 程序运行
100 |         return [str(sys.executable)]
101 |     return [sys.executable, "-m", "pet"]
102 |
103 |
104 | def enable() -> None:
105 |     if _IS_WIN:

```

```

106 |         with winreg.OpenKey(winreg.HKEY_CURRENT_USER, RUN_KEY, 0, winreg.KEY_SET_VALUE) as key:
107 |             winreg.SetValueEx(key, VALUE_NAME, 0, winreg.REG_SZ, _win_command())
108 |     elif _IS_MAC:
109 |         import plistlib
110 |
111 |         _plist_path().parent.mkdir(parents=True, exist_ok=True)
112 |         plist: dict = {
113 |             "Label": APP_ID,
114 |             "ProgramArguments": _mac_program_args(),
115 |             "RunAtLoad": True,
116 |         }
117 |         if not getattr(sys, "frozen", False):
118 |             plist["WorkingDirectory"] = str(_project_root())
119 |         with _plist_path().open("wb") as f:
120 |             plistlib.dump(plist, f)
121 |
122 |
123 | def disable() -> None:
124 |     if _IS_WIN:
125 |         try:
126 |             with winreg.OpenKey(winreg.HKEY_CURRENT_USER, RUN_KEY, 0, winreg.KEY_SET_VALUE) as key:
127 |                 winreg.DeleteValue(key, VALUE_NAME)
128 |         except FileNotFoundError:
129 |             pass
130 |     elif _IS_MAC:
131 |         _plist_path().unlink(missing_ok=True)
132 |
133 |
134 | def set_enabled(on: bool) -> None:
135 |     enable() if on else disable()

```

08. pet/catalog.py (492 lines)

```

1 | # -*- coding: utf-8 -*-
2 | """
3 | 动画目录（catalog）—— 全部动画名、文件映射、分类与几何常量的“事实来源”。
4 |
5 | 素材来源：dsh-pet 插件（https://github.com/PC2005-cloud/dsh-pet）的
6 | assets/thumb/*.webm（640×360 透明 webm，VP9 alpha）。
7 |
8 | 几何常量与原插件 client.js 完全一致：
9 | - 画布 640×360，人物脚底 y=330
10 | - 落地偏移 PAD = 360 - 330 = 30px（绘制时把帧下移 PAD，让脚踩在窗口底线）
11 | """
12 |
13 | import os
14 | import sys
15 | import json
16 | import math
17 | from pathlib import Path
18 |
19 | # ----- 画布几何
20 | # 逻辑画布尺寸。素材可以高于这个尺寸：窗口布局、移动和命中区域始终
21 | # 使用逻辑像素，避免更换 1080p/1440p 素材时连带修改交互参数。
22 | CANVAS_W = 640
23 | CANVAS_H = 360
24 |
25 | # macOS Retina 最大按 2× backing pixels 渲染。2560×1440 超分母版在备份/
26 | # 发布素材中保留。App 使用由母版下采样的 1280×720 透明运行代理；默认档位
27 | # 还会按显示尺寸继续有界解码，避免无效 RGBA 像素进入 Python。
28 | RENDER_DPR_CAP = 2.0
29 | DECODE_MAX_W = int(CANVAS_W * RENDER_DPR_CAP)
30 | DECODE_MAX_H = int(CANVAS_H * RENDER_DPR_CAP)
31 |
32 |
33 | def decode_size_for_scale(scale: float) -> tuple[int, int]:
34 |     """返回当前显示档位所需的 2× Retina 解码尺寸（偶数像素）。"""
35 |     width = min(
36 |         DECODE_MAX_W,
37 |         (math.ceil(CANVAS_W * max(0.1, scale) * RENDER_DPR_CAP) + 1) // 2 * 2,
38 |     )
39 |     height = min(
40 |         DECODE_MAX_H,
41 |         (math.ceil(CANVAS_H * max(0.1, scale) * RENDER_DPR_CAP) + 1) // 2 * 2,
42 |     )
43 |     return max(2, width), max(2, height)

```

```

44 |
45 | # 脚底在画布内的 y 坐标（与母版一致：640×360 画布脚底 y=330）
46 | FEET_Y = 330 / 360 * CANVAS_H # = 330
47 |
48 | # 落地偏移：帧下移多少让脚底恰好落在窗口底线
49 | PAD = CANVAS_H - FEET_Y # = 30
50 |
51 | # 视频帧时长（毫秒）—— 24fps → 40ms，用于时长/移动插值换算
52 | FRAME_MS = 40
53 |
54 | # 动画链概率（与 client.js 一致）：30% 待机 / 10% 转向 / 40% 动作 / 20% 移动
55 | P_IDLE = 0.30
56 | P_TURN = 0.40 # 累计阈值：<0.3 待机，<0.4 转向
57 | P_ACTS = 0.80 # 累计阈值：<0.8 动作，>=0.8 移动
58 |
59 | # 移动参数（与 client.js 一致）
60 | MOVE_MIN_PX = 60
61 | MOVE_MAX_PX = 240
62 | MOVE_MARGIN = 20 # 屏幕边缘安全边距
63 | MOVE_LEAD_SEC = 2 # 动画开头 2s 准备动作，位置不动
64 | MOVE_TAIL_SEC = 2 # 动画结尾 2s 收尾动作，位置不动
65 |
66 | # 拖拽判定阈值（像素，缩放前逻辑像素）
67 | DRAG_THRESHOLD = 5
68 |
69 | # 交互反馈参数：短延迟用于区分单击/双击，其他反馈保持低频且可打断。
70 | TAP_BURST_WINDOW_MS = 320
71 | LONG_PRESS_MS = 520
72 | CURSOR_POLL_MS = 120
73 | CURSOR_REACTION_RADIUS = 280
74 | CURSOR_DEAD_ZONE = 16
75 | PROACTIVE_GREETING_MIN_MS = 45_000
76 | PROACTIVE_GREETING_MAX_MS = 90_000
77 | EDGE_FEEDBACK_MARGIN = 18
78 | EDGE_BOUNCE_SPEED = 300.0
79 |
80 | # 行为预设：概率总和为 1；问候时间用毫秒，避免在窗口层散落魔法数字。
81 | PERSONALITY_PRESETS = {
82 |   'quiet': {
83 |     'label': '安静',
84 |     'idle': 0.70,
85 |     'turn': 0.10,
86 |     'acts': 0.20,
87 |     'move': 0.00,
88 |     'greeting_min_ms': 120_000,
89 |     'greeting_max_ms': 240_000,
90 |     'greeting_chance': 0.45,
91 |   },
92 |   'lively': {
93 |     'label': '活泼',
94 |     'idle': 0.30,
95 |     'turn': 0.10,
96 |     'acts': 0.40,
97 |     'move': 0.20,
98 |     'greeting_min_ms': 45_000,
99 |     'greeting_max_ms': 90_000,
100 |     'greeting_chance': 0.85,
101 |   },
102 |   'mischievous': {
103 |     'label': '调皮',
104 |     'idle': 0.15,
105 |     'turn': 0.15,
106 |     'acts': 0.50,
107 |     'move': 0.20,
108 |     'greeting_min_ms': 20_000,
109 |     'greeting_max_ms': 45_000,
110 |     'greeting_chance': 1.00,
111 |   },
112 | }
113 |
114 | # 默认显示缩放与右下角边距
115 | # 目标显示宽度 ≈ 462px（与 DSH web 端一致）→ 462 / 640 ≈ 0.72
116 | DEFAULT_SCALE = 0.72
117 | CORNER_MARGIN = 24 # 距屏幕右缘的默认间距
118 |
119 | # 可选的显示缩放档位（相对 640 宽：320px / 462px / 544px / 640px）
120 | SCALE_STEPS = (0.5, 0.72, 0.85, 1.0)
121 |
122 | # 系统繁忙时减少主动动作与解码频率。负载判断使用迟滞，避免频繁切换。

```



```

123 | BUSY_IDLE_PROBABILITY = 0.82
124 | BUSY_TURN_PROBABILITY = 0.88
125 | BUSY_IDLE_SPEED_FACTOR = 0.70
126 | BUSY_IDLE_PAUSE_MS = 1800
127 | BUSY_MASK_FRAME_INTERVAL = 2
128 | LOAD_SAMPLE_MS = 5000
129 |
130 | # ----- 多形象
131 | # 当前内置形象与未来扩展形象 ID (目录名建议使用稳定 ASCII)
132 | DEFAULT_CHARACTER = 'shenshen'
133 | CHARACTERS = ('shenshen',)
134 | MANIFEST_FILENAME = 'manifest.json'
135 | # videos 下的分类子目录
136 | DIR_IDLE = 'idle'
137 | DIR_TURN = 'turn'
138 | DIR_IDLE_TURN = 'idle_turn' # 兼容旧结构: 待机+转向合并目录
139 | DIR_MOVE = 'move'
140 | DIR_CLICK = 'click'
141 | DIR_DRAG = 'drag'
142 | DIR_RANDOM = 'random'
143 |
144 |
145 | # ----- 动画映射
146 | # 内置动作以实际素材目录为事实来源。这样新增原作动作时不必再手工维护
147 | # 一份容易漏项的 90+ 行文件名清单; 值保留相对 videos/ 的分类路径。
148 | _DEFAULT_VIDEO_ROOT = (
149 |     Path(__file__).resolve().parent.parent
150 |     / 'assets' / 'characters' / DEFAULT_CHARACTER / 'videos'
151 | )
152 | ANIM_FILES: dict[str, str] = {
153 |     path.stem: path.relative_to(_DEFAULT_VIDEO_ROOT).as_posix()
154 |     for path in sorted(_DEFAULT_VIDEO_ROOT.rglob('*.*webm'))
155 | }
156 |
157 | # 兼容旧字段名: webm 文件名映射
158 | WEBM_FILES: dict[str, str] = ANIM_FILES
159 |
160 | # 动画分组 (语义与 client.js 一致)
161 | IDLE = '待机呼吸休闲'
162 | TURN = '东张西望'
163 | MOVES = ['螃蟹走路', '原地漂浮踏步', '原地左转奔跑']
164 | CLICKS = [
165 |     '点击回应-开心跃动',
166 |     '点击回应-害羞惊讶',
167 |     '点击回应-傲娇生气',
168 |     '点击回应-元气挥手',
169 |     '点击回应-挠痒咯咯笑',
170 | ]
171 | DRAG = '被鼠标拖拽悬空反馈'
172 | ACTS = [n for n in ANIM_FILES if n not in (IDLE, TURN, DRAG, *MOVES, *CLICKS)]
173 |
174 | # 不在导入阶段锁定动作数量。默认形象当前是 91 段, 但后续新增动作或
175 | # 外部角色不应因为 catalog import 失败; 当前素材完整性由测试/打包验收负责。
176 |
177 |
178 | def assets_dir() -> Path:
179 |     """兼容旧调用: 默认形象 shenshen 的 webm 素材目录。"""
180 |     return webm_dir()
181 |
182 |
183 | def characters_dir() -> Path:
184 |     """内置多形象根目录 (项目根/assets/characters)。"""
185 |     return Path(__file__).resolve().parent.parent / 'assets' / 'characters'
186 |
187 |
188 | def character_video_dir(character_id: str) -> Path:
189 |     """内置某个形象的 webm 目录: assets/characters/<id>/videos。"""
190 |     return characters_dir() / character_id / 'videos'
191 |
192 |
193 | def characters_gif_dir() -> Path:
194 |     """内置 GIF 多形象根目录 (项目根/assets/characters_gif)。"""
195 |     return Path(__file__).resolve().parent.parent / 'assets' / 'characters_gif'
196 |
197 |
198 | def character_gif_video_dir(character_id: str) -> Path:
199 |     """内置某个形象的 GIF 目录: assets/characters_gif/<id>/videos。"""
200 |     return characters_gif_dir() / character_id / 'videos'
201 |

```

```

202 |
203 | def external_character_dirs() -> list[Path]:
204 |     """外部可扩展形象根目录（不存在时返回空列表，不报错）。
205 |
206 |     顺序：
207 |     1. exe 同目录 / 当前工作目录下的 characters/
208 |     2. 用户数据目录下的 dsh-pet-standalone/characters/
209 |     """
210 |     dirs: list[Path] = []
211 |     if getattr(sys, 'frozen', False):
212 |         base = Path(sys.executable).resolve().parent
213 |     else:
214 |         base = Path.cwd()
215 |     dirs.append(base / 'characters')
216 |
217 |     if sys.platform == 'win32':
218 |         data_root = Path(os.environ.get('APPDATA', Path.home())) / 'dsh-pet-standalone'
219 |     elif sys.platform == 'darwin':
220 |         data_root = Path.home() / 'Library' / 'Application Support' / 'dsh-pet-standalone'
221 |     else:
222 |         data_root = Path(os.environ.get('XDG_CONFIG_HOME', Path.home() / '.config')) / 'dsh-pet-standalone'
223 |     dirs.append(data_root / 'characters')
224 |     return dirs
225 |
226 |
227 | def resolve_character_video_dir(character_id: str) -> Path:
228 |     """按 外部 > 内置(webm) > 内置(gif) 返回形象视频目录；都不存在时回退 webm 路径，不报错。"""
229 |     for root in external_character_dirs():
230 |         candidate = root / character_id / 'videos'
231 |         if candidate.is_dir():
232 |             return candidate
233 |     webm_dir_path = character_video_dir(character_id)
234 |     if webm_dir_path.is_dir() and any(webm_dir_path.rglob('*.*webm')):
235 |         return webm_dir_path
236 |     gif_dir_path = character_gif_video_dir(character_id)
237 |     if gif_dir_path.is_dir() and any(gif_dir_path.rglob('*.*gif')):
238 |         return gif_dir_path
239 |     return webm_dir_path
240 |
241 |
242 | def list_available_characters() -> list[str]:
243 |     """返回可切换角色列表：内置角色 + 外部目录中额外检测到的角色。
244 |
245 |     外部目录不存在时静默跳过，不会报错。
246 |     """
247 |     ids: list[str] = list(CHARACTERS)
248 |     seen = set(ids)
249 |     for root in external_character_dirs():
250 |         if not root.is_dir():
251 |             continue
252 |         try:
253 |             entries = sorted(root.iterdir())
254 |         except OSError:
255 |             continue
256 |         for child in entries:
257 |             video_dir = child / 'videos'
258 |             if child.is_dir() and video_dir.is_dir() and (
259 |                 any(video_dir.rglob('*.*webm')) or any(video_dir.rglob('*.*gif'))
260 |             ):
261 |                 cid = child.name
262 |                 if cid not in seen:
263 |                     seen.add(cid)
264 |                     ids.append(cid)
265 |     return ids
266 |
267 |
268 | def webm_dir() -> Path:
269 |     """默认形象 shenshen 的 webm 素材目录（兼容旧调用）。"""
270 |     return character_video_dir(DEFAULT_CHARACTER)
271 |
272 |
273 | def legacy_assets_dir() -> Path:
274 |     """兼容旧名称：默认形象 webm 素材目录。"""
275 |     return webm_dir()
276 |
277 |
278 | def load_character_manifest(character_id: str, asset_dir: Path | str | None = None) -> dict | None:
279 |     """读取角色目录下的 manifest.json（可选）。
280 |

```

```

281 |     查找位置（按优先级）：
282 |     1. <角色目录>/videos/manifest.json
283 |     2. <角色目录>/manifest.json
284 |
285 |     不存在或解析失败时返回 None，不影响运行。
286 |     """
287 |     if asset_dir is not None:
288 |         video_dir = Path(asset_dir)
289 |     else:
290 |         video_dir = resolve_character_video_dir(character_id)
291 |     candidates = [
292 |         video_dir / MANIFEST_FILENAME,
293 |         video_dir.parent / MANIFEST_FILENAME,
294 |     ]
295 |     for path in candidates:
296 |         if path.is_file():
297 |             try:
298 |                 data = json.loads(path.read_text(encoding='utf-8'))
299 |                 if isinstance(data, dict):
300 |                     return data
301 |             except Exception:
302 |                 return None
303 |     return None
304 |
305 |
306 | def _manifest_name(value, names: set[str]) -> str | None:
307 |     """把 manifest 中的文件名/动画名解析为实际存在的动画名。"""
308 |     if not isinstance(value, str):
309 |         return None
310 |     stem = Path(value).stem
311 |     if stem in names:
312 |         return stem
313 |     if value in names:
314 |         return value
315 |     return None
316 |
317 |
318 | def _manifest_names(values, names: set[str]) -> list[str]:
319 |     if not isinstance(values, list):
320 |         values = [values]
321 |     result: list[str] = []
322 |     for v in values:
323 |         name = _manifest_name(v, names)
324 |         if name is not None and name not in result:
325 |             result.append(name)
326 |     return result
327 |
328 |
329 | def _keyword_match(name: str, keywords) -> bool:
330 |     low = name.lower()
331 |     return any(k.lower() in low for k in keywords)
332 |
333 |
334 | def build_categories(names, manifest: dict | None = None, folder_map: dict | None = None, folder_files: dict | None = None) -> dict:
335 |     """根据某个形象实际拥有的动画名，动态计算分类。
336 |
337 |     分类优先级：
338 |     1. 如果提供 folder_map（来自 videos/ 下的子目录），优先按子目录分类；
339 |     2. 如果角色目录存在 manifest.json，则按 manifest 指定分类（可补充/覆盖）；
340 |     3. 否则按内置已知名称 + 文件名关键词自动识别；
341 |     4. 未进入核心分类的动画自动归入“随机动作池”。
342 |
343 |     推荐的 videos 目录结构：
344 |     videos/
345 |     |   |-- idle/      # 待机（可多个）
346 |     |   |-- turn/     # 转向（可多个）
347 |     |   |-- move/     # 移动
348 |     |   |-- click/    # 点击回应
349 |     |   |-- drag/     # 拖拽（可选）
350 |     |   |-- random/   # 随机动作
351 |     |
352 |     """
353 |     names = set(names)
354 |     if not names:
355 |         return {
356 |             'idle': None, 'turn': None,
357 |             'idles': [], 'turns': [],
358 |             'moves': [], 'clicks': [], 'drag': None, 'acts': [],
359 |         }

```

```

360 | idles: list[str] = []
361 | turns: list[str] = []
362 | moves: list[str] = []
363 | clicks: list[str] = []
364 | drag = None
365 |
366 | if folder_files is not None:
367 |     by_folder: dict[str, list[str]] = {k: list(v) for k, v in folder_files.items()}
368 | elif folder_map:
369 |     by_folder: dict[str, list[str]] = {}
370 |     for name in names:
371 |         by_folder.setdefault(folder_map.get(name, ''), []).append(name)
372 | else:
373 |     by_folder = {}
374 |
375 | if folder_files is not None or folder_map:
376 |     idles = list(by_folder.get(DIR_IDLE, []))
377 |     turns = list(by_folder.get(DIR_TURN, []))
378 |     legacy_idle_turn = by_folder.get(DIR_IDLE_TURN, [])
379 |
380 |     if not idles and legacy_idle_turn:
381 |         idle_candidates = [
382 |             n for n in legacy_idle_turn
383 |             if n == IDLE or _keyword_match(n, ['待机', 'idle', '呼吸'])
384 |         ]
385 |         idles = idle_candidates or (legacy_idle_turn[:1] if legacy_idle_turn else [])
386 |
387 |     if not turns and legacy_idle_turn:
388 |         turn_candidates = [
389 |             n for n in legacy_idle_turn
390 |             if n == TURN or _keyword_match(n, ['转向', '转身', '东张西望', 'turn', '回头', '转'])
391 |         ]
392 |         turns = turn_candidates
393 |         if not turns and idles and len(legacy_idle_turn) > 1:
394 |             turns = [n for n in legacy_idle_turn if n != idles[0]][1:]
395 |
396 |     moves = list(by_folder.get(DIR_MOVE, []))
397 |     clicks = list(by_folder.get(DIR_CLICK, []))
398 |     drag_names = by_folder.get(DIR_DRAG, [])
399 |     if drag_names:
400 |         drag = drag_names[0]
401 |
402 | # manifest 补充/覆盖
403 | if manifest:
404 |     if not idles:
405 |         m = _manifest_name(manifest.get('idle'), names)
406 |         if m:
407 |             idles = [m]
408 |     if not turns:
409 |         m = _manifest_name(manifest.get('turn'), names)
410 |         if m:
411 |             turns = [m]
412 |     if not moves:
413 |         moves = _manifest_names(manifest.get('moves', []), names)
414 |     if not clicks:
415 |         clicks = _manifest_names(manifest.get('clicks', []), names)
416 |     if drag is None:
417 |         drag = _manifest_name(manifest.get('drag'), names)
418 |
419 | # 关键词兜底
420 | if not idles:
421 |     m = IDLE if IDLE in names else next(
422 |         (n for n in names if _keyword_match(n, ['待机', 'idle', '呼吸'])), None
423 |     )
424 |     if m:
425 |         idles = [m]
426 | if not turns:
427 |     m = TURN if TURN in names else next(
428 |         (n for n in names if _keyword_match(n, ['转向', '转身', '东张西望', 'turn', '回头', '转'])), None
429 |     )
430 |     if m:
431 |         turns = [m]
432 | if drag is None:
433 |     drag = DRAG if DRAG in names else next(
434 |         (n for n in names if _keyword_match(n, ['拖拽', '拖', '悬空', 'drag', '抓'])), None
435 |     )
436 | if not moves:
437 |     moves = [n for n in MOVES if n in names]
438 |     if not moves:

```

```

439 |         moves = [n for n in names if _keyword_match(n, ['走', '跑', '移动', 'move', 'walk', 'run', '踏步', '奔跑'])]
440 |     if not clicks:
441 |         clicks = [n for n in CLICKS if n in names]
442 |     if not clicks:
443 |         clicks = [n for n in names if _keyword_match(n, ['点击', '回应', 'click', 'response'])]
444 |
445 |     # 如果没有明确 idle, 安全回退到第一个动画, 避免启动崩溃
446 |     if not idles:
447 |         first = next(iter(names), None)
448 |         if first:
449 |             idles = [first]
450 |
451 |     core = set(idles) | set(turns) | set(moves) | set(clicks)
452 |     if drag:
453 |         core.add(drag)
454 |
455 |     # 平铺目录 (folder == '') 只是兼容旧素材结构, 不应把所有动画
456 |     # 当成“随机动作”。只有真正存在子目录时, 才按目录分类处理。
457 |     has_folder_layout = folder_files is not None and any(by_folder)
458 |     if has_folder_layout:
459 |         # 子目录模式下, random/ 和未知目录的内容都进入随机动作池;
460 |         # 允许同一文件同时出现在多个分类中 (例如测试时复制同一视频到多个文件夹)
461 |         acts = []
462 |         known = {DIR_IDLE, DIR_TURN, DIR_MOVE, DIR_CLICK, DIR_DRAG}
463 |         for folder, ns in by_folder.items():
464 |             # folder == '' 是旧版平铺素材的重复入口, 不应覆盖结构化分类。
465 |             if folder == DIR_RANDOM or (folder and folder not in known):
466 |                 acts.extend(ns)
467 |         seen_acts = set()
468 |         unique_acts = []
469 |         for n in acts:
470 |             if n not in seen_acts:
471 |                 seen_acts.add(n)
472 |                 unique_acts.append(n)
473 |         acts = unique_acts
474 |     else:
475 |         acts = [n for n in names if n not in core]
476 |     return {
477 |         'idle': idles[0] if idles else None,
478 |         'turn': turns[0] if turns else None,
479 |         'idles': idles,
480 |         'turns': turns,
481 |         'moves': moves,
482 |         'clicks': clicks,
483 |         'drag': drag,
484 |         'acts': acts,
485 |     }
486 |
487 |
488 | def resolve_asset_path(name: str, filename: str, base_dir: Path | None = None) -> Path:
489 |     """解析 webm 素材路径: 不存在时返回预期路径以便上层报错。"""
490 |     base_dir = Path(base_dir) if base_dir is not None else webm_dir()
491 |     path = base_dir / WEBM_FILES.get(name, filename)
492 |     return path

```

09. pet/config.py (87 lines)

```

1 | # -*- coding: utf-8 -*-
2 | """
3 | 配置持久化 (跨平台):
4 | - Windows: %APPDATA%\dsh-pet-standalone/config.json
5 | - macOS: ~/Library/Application Support/dsh-pet-standalone/config.json
6 | - Linux: ~/.config/dsh-pet-standalone/config.json
7 |
8 | 记录: 位置 (相对屏幕可用区的中心比例, 分辨率变化后仍正确)、
9 | 朝向、缩放、置顶开关。
10 | """
11 |
12 | from __future__ import annotations
13 |
14 | import json
15 | import os
16 | import sys
17 | from pathlib import Path
18 |
19 | from . import catalog

```

```

20 |
21 |
22 | def _default_base() -> Path:
23 |     """按平台返回配置根目录 (Windows=APPDATA, macOS=Application Support, Linux=~/.config) . """
24 |     if sys.platform == 'win32':
25 |         return Path(os.environ.get('APPDATA') or Path.home())
26 |     if sys.platform == 'darwin':
27 |         return Path.home() / 'Library' / 'Application Support'
28 |     return Path.home() / '.config'
29 |
30 |
31 | class Config:
32 |     def __init__(self, base: Path | str | None = None) -> None:
33 |         base = Path(base) if isinstance(base, str) else (base or _default_base())
34 |         self.dir = base / 'dsh-pet-standalone'
35 |         self.path = self.dir / 'config.json'
36 |         self.data: dict = {
37 |             'version': 2, # 配置结构版本; scale 语义变更时递增
38 |             'rx': None, # 窗口中心 x / 屏幕可用区宽 (None=默认右下角)
39 |             'ry': None, # 窗口中心 y / 屏幕可用区高
40 |             'facing': 'left',
41 |             'scale': catalog.DEFAULT_SCALE,
42 |             'on_top': True,
43 |             'no_move': False, # 不移动: 勾选后状态机不再自动移动, 仅手动点移动动画才走动
44 |             'character': catalog.DEFAULT_CHARACTER, # 当前形象 ID
45 |             'playback_speed': 1.0, # 动画播放速率
46 |             'mouse_through': False, # 鼠标穿透
47 |             'drag_physics': False, # 拖动物理效果
48 |             'soft_edges': True, # 兼容旧配置: 清理精确 Alpha=1 底噪
49 |             'duck_sound': True, # 双击/长按/连续点击时播放尖叫鸭
50 |             'proactive_greetings': True, # 偶尔主动播放挥手问候
51 |             'favorites': [], # 用户收藏的动画名
52 |             'playlist': [], # 播放列表动画名
53 |             'playlist_mode': 'off', # off / loop / random
54 |             'personality': 'lively', # quiet / lively / mischievous
55 |         }
56 |         self._load()
57 |
58 |     def _load(self) -> None:
59 |         try:
60 |             raw = json.loads(self.path.read_text(encoding='utf-8'))
61 |         except (OSError, ValueError):
62 |             return
63 |         if isinstance(raw, dict):
64 |             if int(raw.get('version', 1)) < 2:
65 |                 # v1 -> v2: 素材从 220×124 换成 640×360, scale 语义变化,
66 |                 # 旧 scale (如 1.0 表示 220px) 需重置为新的默认值。
67 |                 raw.pop('scale', None)
68 |                 raw['version'] = 2
69 |             for key in self.data:
70 |                 if key in raw and raw[key] is not None:
71 |                     self.data[key] = raw[key]
72 |
73 |     def get(self, key: str, default=None):
74 |         return self.data.get(key, default)
75 |
76 |     def set(self, key: str, value) -> None:
77 |         self.data[key] = value
78 |
79 |     def save(self) -> None:
80 |         try:
81 |             self.dir.mkdir(parents=True, exist_ok=True)
82 |             self.path.write_text(
83 |                 json.dumps(self.data, ensure_ascii=False, indent=2),
84 |                 encoding='utf-8',
85 |             )
86 |         except OSError:
87 |             pass # 配置写失败不致命

```

10. pet/hotkeys.py (197 lines)

```

1 | # -*- coding: utf-8 -*-
2 | """macOS Carbon 全局快捷键的最小封装。
3 |
4 | 不依赖 PyObjC: Qt 的 Cocoa 事件循环可以同时接收 Carbon hot key 事件。
5 | 其他平台安全降级为 disabled, 状态栏菜单仍然可用。

```

```

6 | """
7 |
8 | from __future__ import annotations
9 |
10 | import ctypes
11 | import ctypes.util
12 | import logging
13 | import sys
14 | from dataclasses import dataclass
15 |
16 | from PySide6.QtCore import QTimer
17 |
18 |
19 | class _EventHotKeyID(ctypes.Structure):
20 |     _fields_ = [
21 |         ('signature', ctypes.c_uint32),
22 |         ('id', ctypes.c_uint32),
23 |     ]
24 |
25 |
26 | @dataclass(frozen=True)
27 | class HotKeySpec:
28 |     name: str
29 |     key_code: int
30 |     action: str
31 |
32 |
33 | class GlobalHotkeys:
34 |     """注册一组低冲突的 Control+Option+Command 全局快捷键。"""
35 |
36 |     MOD_COMMAND = 1 << 8
37 |     MOD_SHIFT = 1 << 9
38 |     MOD_OPTION = 1 << 11
39 |     MOD_CONTROL = 1 << 12
40 |     EVENT_CLASS_KEYBOARD = 0x6B657962 # 'keyb'
41 |     EVENT_HOT_KEY_PRESSED = 5
42 |     PARAM_DIRECT_OBJECT = 0x2D2D2D2D # '----'
43 |     TYPE_EVENT_HOT_KEY_ID = 0x68686864 # 'hkid'
44 |
45 |     # Carbon virtual key codes: H=4, P=35, R=15, M=46, D=2.
46 |     DEFAULTS = (
47 |         HotKeySpec('显示 / 隐藏', 4, 'toggle_visible'),
48 |         HotKeySpec('暂停 / 继续', 35, 'toggle_pause'),
49 |         HotKeySpec('随机动作', 15, 'random_action'),
50 |         HotKeySpec('鼠标穿透', 46, 'toggle_mouse_through'),
51 |         HotKeySpec('尖叫鸭', 2, 'duck_sound'),
52 |     )
53 |
54 |     def __init__(self, callback) -> None:
55 |         self.callback = callback
56 |         self.enabled = False
57 |         self.error: str | None = None
58 |         self._carbon = None
59 |         self._handler = None
60 |         self._hotkeys: list[ctypes.c_void_p] = []
61 |         self._callback_ref = None
62 |
63 |     def start(self) -> bool:
64 |         if sys.platform != 'darwin':
65 |             self.error = '仅 macOS 支持全局快捷键'
66 |             return False
67 |         if self.enabled:
68 |             return True
69 |         try:
70 |             path = ctypes.util.find_library('Carbon') or (
71 |                 '/System/Library/Frameworks/Carbon.framework/Carbon'
72 |             )
73 |             self._carbon = ctypes.cdll.LoadLibrary(path)
74 |             carbon = self._carbon
75 |             carbon.GetApplicationEventTarget.restype = ctypes.c_void_p
76 |             carbon.InstallEventHandler.argtypes = [
77 |                 ctypes.c_void_p,
78 |                 ctypes.c_void_p,
79 |                 ctypes.c_uint32,
80 |                 ctypes.c_void_p,
81 |                 ctypes.c_void_p,
82 |                 ctypes.POINTER(ctypes.c_void_p),
83 |             ]
84 |             carbon.InstallEventHandler.restype = ctypes.c_int32

```

```

85 |         carbon.RegisterEventHotKey. argtypes = [
86 |             ctypes.c_uint32,
87 |             ctypes.c_uint32,
88 |             _EventHotKeyID,
89 |             ctypes.c_void_p,
90 |             ctypes.c_uint32,
91 |             ctypes.POINTER(ctypes.c_void_p),
92 |         ]
93 |         carbon.RegisterEventHotKey. restype = ctypes.c_int32
94 |         carbon.GetEventParameter. argtypes = [
95 |             ctypes.c_void_p,
96 |             ctypes.c_uint32,
97 |             ctypes.c_uint32,
98 |             ctypes.POINTER(ctypes.c_void_p),
99 |             ctypes.c_uint32,
100 |             ctypes.POINTER(ctypes.c_uint32),
101 |             ctypes.c_void_p,
102 |         ]
103 |         carbon.GetEventParameter. restype = ctypes.c_int32
104 |
105 |         event_target = carbon.GetApplicationEventTarget()
106 |         event_spec = (ctypes.c_uint32 * 2) (
107 |             self.EVENT_CLASS_KEYBOARD,
108 |             self.EVENT_HOT_KEY_PRESSED,
109 |         )
110 |         handler_ref = ctypes.c_void_p()
111 |
112 |         @ctypes.CFUNCTYPE(
113 |             ctypes.c_int32,
114 |             ctypes.c_void_p,
115 |             ctypes.c_void_p,
116 |             ctypes.c_void_p,
117 |         )
118 |         def handler(_next_handler, event, _user_data):
119 |             hotkey_id = (ctypes.c_uint32 * 2) ()
120 |             actual_size = ctypes.c_uint32()
121 |             status = carbon.GetEventParameter(
122 |                 event,
123 |                 self.PARAM_DIRECT_OBJECT,
124 |                 self.TYPE_EVENT_HOT_KEY_ID,
125 |                 None,
126 |                 ctypes.sizeof(hotkey_id),
127 |                 ctypes.byref(actual_size),
128 |                 ctypes.byref(hotkey_id),
129 |             )
130 |             if status == 0:
131 |                 self._dispatch(int(hotkey_id[1]))
132 |             return 0
133 |
134 |         self._callback_ref = handler
135 |         status = carbon.InstallEventHandler(
136 |             event_target,
137 |             ctypes.cast(handler, ctypes.c_void_p),
138 |             1,
139 |             ctypes.byref(event_spec),
140 |             None,
141 |             ctypes.byref(handler_ref),
142 |         )
143 |         if status != 0:
144 |             raise OSError(f'InstallEventHandler status={status}')
145 |         self._handler = handler_ref
146 |
147 |         for index, spec in enumerate(self.DEFAULTS):
148 |             hotkey_id = _EventHotKeyID(0x44534850, index + 1)
149 |             ref = ctypes.c_void_p()
150 |             status = carbon.RegisterEventHotKey(
151 |                 spec.key_code,
152 |                 self.MOD_COMMAND | self.MOD_OPTION | self.MOD_CONTROL,
153 |                 hotkey_id,
154 |                 event_target,
155 |                 0,
156 |                 ctypes.byref(ref),
157 |             )
158 |             if status != 0:
159 |                 raise OSError(
160 |                     f'RegisterEventHotKey {spec.name} status={status}'
161 |                 )
162 |             self._hotkeys.append(ref)
163 |         self.enabled = True

```



```

164 |         logging.info('全局快捷键已启用: Control+Option+Command')
165 |         return True
166 |     except (OSError, AttributeError, ctypes.ArgumentError) as exc:
167 |         self.error = str(exc)
168 |         logging.warning('全局快捷键不可用: %s', exc)
169 |         self.stop()
170 |         return False
171 |
172 |     def stop(self) -> None:
173 |         carbon = self._carbon
174 |         if carbon is not None:
175 |             unregister = getattr(carbon, 'UnregisterEventHotKey', None)
176 |             if unregister is not None:
177 |                 unregister.argtypes = [ctypes.c_void_p]
178 |                 unregister.restype = ctypes.c_int32
179 |                 for ref in self._hotkeys:
180 |                     unregister(ref)
181 |             remove = getattr(carbon, 'RemoveEventHandler', None)
182 |             if remove is not None and self._handler is not None:
183 |                 remove.argtypes = [ctypes.c_void_p]
184 |                 remove.restype = ctypes.c_int32
185 |                 remove(self._handler)
186 |             self._hotkeys.clear()
187 |             self._handler = None
188 |             self._callback_ref = None
189 |             self.enabled = False
190 |
191 |     def _dispatch(self, index: int) -> None:
192 |         if 1 <= index <= len(self.DEFAULTS):
193 |             try:
194 |                 action = self.DEFAULTS[index - 1].action
195 |                 QTimer.singleShot(0, lambda: self.callback(action))
196 |             except Exception:
197 |                 logging.exception('全局快捷键动作失败: %s', index)

```

11. pet/interaction.py (49 lines)

```

1 | # -*- coding: utf-8 -*-
2 | """桌宠输入手势与空间反馈的纯函数。"""
3 |
4 | from __future__ import annotations
5 |
6 | import math
7 |
8 |
9 | def classify_tap_burst(count: int) -> str:
10 |     """把一次快速点击序列分成单击、双击或连续点击。"""
11 |     if count <= 1:
12 |         return "single"
13 |     if count == 2:
14 |         return "double"
15 |     return "rapid"
16 |
17 |
18 | def cursor_facing(
19 |     center: tuple[float, float],
20 |     cursor: tuple[float, float],
21 |     radius: float,
22 |     dead_zone: float = 16.0,
23 | ) -> str | None:
24 |     """鼠标进入反应半径后返回朝向：在中心死区或半径外不改变朝向。"""
25 |     dx = cursor[0] - center[0]
26 |     dy = cursor[1] - center[1]
27 |     if math.hypot(dx, dy) > radius or abs(dx) < dead_zone:
28 |         return None
29 |     return "right" if dx > 0 else "left"
30 |
31 |
32 | def edge_contacts(
33 |     rect: tuple[float, float, float, float],
34 |     screen: tuple[float, float, float, float],
35 |     margin: float,
36 | ) -> frozenset[str]:
37 |     """返回窗口是否贴近屏幕四边：参数均为 x, y, width, height。"""
38 |     x, y, width, height = rect
39 |     sx, sy, sw, sh = screen

```

```

40 |     contacts: set[str] = set()
41 |     if x <= sx + margin:
42 |         contacts.add("left")
43 |     if y <= sy + margin:
44 |         contacts.add("top")
45 |     if x + width >= sx + sw - margin:
46 |         contacts.add("right")
47 |     if y + height >= sy + sh - margin:
48 |         contacts.add("bottom")
49 |     return frozenset(contacts)

```

12. pet/library.py (362 lines)

```

1 | # -*- coding: utf-8 -*-
2 | """
3 | Media library —— 多形象，自动识别 webm / gif。
4 |
5 | 支持按角色 ID 加载不同形象。
6 | - 默认从内置 assets/characters/<character_id>/videos/ 加载
7 | - 也支持外部扩展目录 (exe 同目录/用户数据目录下的 characters/<id>/videos)
8 | - 如果目录里是 *.webm 则用 WebMClip；如果是 *.gif 则用 GifClip
9 |
10 | 对外保持与窗口层一致的形状：
11 | - movie(name) -> clip object
12 | - movies() -> name -> clip mapping
13 | - frames(name) / duration(name) (秒)
14 |
15 | WebMClip 基于 imageio-ffmpeg 解码高分辨率透明 webm (RGBA)，运行时再按窗口尺寸平滑缩放。
16 | GifClip 基于 QMovie 播放透明 GIF (兼容旧 GIF 路线)。
17 | """
18 |
19 | from __future__ import annotations
20 |
21 | from collections import OrderedDict
22 | from concurrent.futures import ThreadPoolExecutor
23 | from dataclasses import dataclass
24 | import threading
25 | from pathlib import Path
26 | from typing import Mapping
27 |
28 | from PySide6.QtCore import QObject, Signal
29 | from PySide6.QtGui import QMovie, QPixmap
30 |
31 | from . import catalog
32 | from .webm_clip import (
33 |     DecodedFrame,
34 |     WebMClip,
35 |     frame_canvas_image,
36 |     trim_transparent_frame,
37 | )
38 |
39 |
40 | # QMovie 播放速度补偿 (%)：GIF 路线使用，校准 QMovie 偏慢问题
41 | PLAYBACK_SPEED = 120
42 | DEFAULT_CLIP_CACHE_SIZE = 8
43 |
44 |
45 | @dataclass(frozen=True)
46 | class MediaSource:
47 |     """一个动作的轻量索引记录。
48 |
49 |     目录扫描阶段只保留路径和格式，不创建 QMovie/Qtimer/ffmpeg reader。
50 |     这样新增角色或动作时，索引层不再被具体播放器实现绑定。
51 |     """
52 |
53 |     name: str
54 |     path: Path
55 |     media_type: str
56 |     folder: str = ''
57 |
58 |
59 | class GifClip(QObject):
60 |     """QMovie 包装：与 WebMClip 接口兼容的 GIF 播放器。"""
61 |
62 |     frameChanged = Signal(int)
63 |     finished = Signal()

```

```

64 |     errorOccurred = Signal(str)
65 |
66 |     def __init__(
67 |         self,
68 |         path: Path,
69 |         parent: QObject | None = None,
70 |         *,
71 |         soft_edges: bool = False,
72 |     ) -> None:
73 |         super().__init__(parent)
74 |         self.path = path
75 |         self._soft_edges = bool(soft_edges)
76 |         self._movie = QMovie(str(path))
77 |         self._movie.setCacheMode(QMovie.CacheMode.CacheNone)
78 |         self._movie.setSpeed(PLAYBACK_SPEED)
79 |         self._movie.frameChanged.connect(self._on_frame_changed)
80 |         self._movie.finished.connect(self.finished)
81 |         self._movie.error.connect(lambda err: self.errorOccurred.emit(str(err)))
82 |         self._frame_count = 0
83 |         self._current_frame: DecodedFrame | None = None
84 |         self.playback_speed = 1.0
85 |         self._movie.jumpToFrame(0)
86 |         self._refresh_frame()
87 |         self._frame_count = max(0, self._movie.frameCount())
88 |
89 |     def frameCount(self) -> int:
90 |         if self._frame_count <= 0:
91 |             self._frame_count = max(0, self._movie.frameCount())
92 |         return max(1, self._frame_count)
93 |
94 |     def duration(self) -> float:
95 |         return self.frameCount() * catalog.FRAME_MS / 1000.0 / self.playback_speed
96 |
97 |     def currentFrameNumber(self) -> int:
98 |         return self._movie.currentFrameNumber()
99 |
100 |     def currentTimeSeconds(self) -> float:
101 |         n = self._movie.currentFrameNumber()
102 |         frames = self.frameCount()
103 |         if frames <= 0:
104 |             return 0.0
105 |         return n * (self.duration() / frames)
106 |
107 |     def currentFrame(self) -> DecodedFrame | None:
108 |         if self._current_frame is None:
109 |             self._refresh_frame()
110 |         return self._current_frame
111 |
112 |     def currentPixmap(self):
113 |         """旧诊断接口：窗口热路径直接消费 currentFrame()。"""
114 |         frame = self.currentFrame()
115 |         return QPixmap() if frame is None else QPixmap.fromImage(frame_canvas_image(frame))
116 |
117 |     def set_playback_speed(self, speed: float) -> None:
118 |         self.playback_speed = max(0.1, float(speed))
119 |         self._movie.setSpeed(int(round(PLAYBACK_SPEED * self.playback_speed)))
120 |
121 |     def start(self) -> None:
122 |         self._movie.start()
123 |
124 |     def stop(self) -> None:
125 |         self._movie.stop()
126 |
127 |     def jumpToFrame(self, frame_index: int) -> bool:
128 |         if frame_index < 0:
129 |             frame_index = 0
130 |         total = self._movie.frameCount()
131 |         if total > 0 and frame_index >= total:
132 |             frame_index = total - 1
133 |         changed = self._movie.jumpToFrame(frame_index)
134 |         if changed:
135 |             self._refresh_frame()
136 |         return changed
137 |
138 |     def warm_meta(self) -> None:
139 |         # GIF 由 QMovie 直接管理元数据，无需额外预热
140 |         return
141 |
142 |     def _on_frame_changed(self, n: int) -> None:

```

```

143 |         self._refresh_frame()
144 |         fc = self._movie.frameCount()
145 |         if fc > 0:
146 |             self._frame_count = fc
147 |         self.frameChanged.emit(n)
148 |
149 |     def _refresh_frame(self) -> None:
150 |         image = self._movie.currentImage()
151 |         if image is not None and not image.isNull():
152 |             self._current_frame = trim_transparent_frame(
153 |                 image,
154 |                 soften_edges=self._soft_edges,
155 |             )
156 |
157 |
158 | class MovieLibrary(QObject):
159 |     """素材库：加载指定形象的 webm 或 gif 动画。"""
160 |
161 |     def __init__(
162 |         self,
163 |         parent: QObject | None = None,
164 |         *,
165 |         character_id: str | None = None,
166 |         asset_dir: Path | str | None = None,
167 |         manifest: Mapping[str, str] | None = None,
168 |         decode_size: tuple[int, int] | None = None,
169 |         cache_limit: int = DEFAULT_CLIP_CACHE_SIZE,
170 |         soft_edges: bool = False,
171 |     ) -> None:
172 |         super().__init__(parent)
173 |         self.character_id = character_id or catalog.DEFAULT_CHARACTER
174 |         if asset_dir is not None:
175 |             self._asset_dir = Path(asset_dir)
176 |         else:
177 |             self._asset_dir = catalog.resolve_character_video_dir(self.character_id)
178 |         self._manifest = None if manifest is None else dict(manifest)
179 |         self._decode_size = decode_size or catalog.decode_size_for_scale(catalog.DEFAULT_SCALE)
180 |         self._cache_limit = max(1, int(cache_limit))
181 |         self._soft_edges = bool(soft_edges)
182 |         self._active_name: str | None = None
183 |         self._manifest = catalog.load_character_manifest(self.character_id, self._asset_dir)
184 |         self._folder_map: dict[str, str] = {}
185 |         self._folder_files: dict[str, list[str]] = {}
186 |         self._sources: dict[str, MediaSource] = {}
187 |         self._movies: OrderedDict[str, object] = OrderedDict()
188 |         self._media_type: str = 'webm'
189 |
190 |         self._load_all()
191 |
192 |     def _load_all(self) -> None:
193 |         if self._manifest is None:
194 |             # 自动扫描该形象目录下的 webm 或 gif，支持不同角色有不同动作集
195 |             if not self._asset_dir.is_dir():
196 |                 raise FileNotFoundError(
197 |                     f"角色素材目录不存在: {self._asset_dir} (character_id={self.character_id})"
198 |                 )
199 |             webm_files = sorted(self._asset_dir.rglob('*.*webm'))
200 |             gif_files = sorted(self._asset_dir.rglob('*.*gif'))
201 |             files = webm_files + gif_files
202 |             if not files:
203 |                 raise FileNotFoundError(
204 |                     f"角色素材目录中没有 webm/gif 文件: {self._asset_dir}"
205 |                 )
206 |             if webm_files and gif_files:
207 |                 self._media_type = 'mixed'
208 |             elif webm_files:
209 |                 self._media_type = 'webm'
210 |             else:
211 |                 self._media_type = 'gif'
212 |             self._manifest = {}
213 |             self._folder_map = {}
214 |             self._folder_files = {}
215 |             for f in files:
216 |                 rel = f.relative_to(self._asset_dir)
217 |                 name = f.stem
218 |                 self._manifest[name] = rel.as_posix()
219 |                 folder = rel.parts[0].lower() if len(rel.parts) > 1 else ''
220 |                 self._folder_map[name] = folder
221 |                 self._folder_files.setdefault(folder, []).append(name)

```

```

222 |
223 |     missing: list[str] = []
224 |     resolved: dict[str, Path] = {}
225 |     for name, fname in self._manifest.items():
226 |         path = self._asset_dir / fname
227 |         if not path.exists():
228 |             missing.append(f"{name}: {path}")
229 |             continue
230 |         resolved[name] = path
231 |
232 |     if missing:
233 |         raise FileNotFoundError("缺少素材文件: " + ", ".join(missing))
234 |
235 |     for name, path in resolved.items():
236 |         folder = self.folder_map.get(name, '')
237 |         media_type = 'gif' if path.suffix.lower() == '.gif' else 'webm'
238 |         self._sources[name] = MediaSource(name, path, media_type, folder)
239 |
240 |     # 只预热移动动画：移动逻辑在播放前就需要 duration；其他动画的 reader
241 |     # 会在首帧回调前补齐元数据。避免 91 个动作启动时全部创建播放器并跑 ffmpeg。
242 |     move_names = self.folder_files.get(catalog.DIR_MOVE, [])
243 |     move_clips = [self.movie(name) for name in move_names if name in self._sources]
244 |     if move_clips:
245 |         threading.Thread(
246 |             target=self._warm_meta_background,
247 |             args=(move_clips,),
248 |             daemon=True,
249 |         ).start()
250 |
251 |     @staticmethod
252 |     def _warm_meta_background(clips: list[object]) -> None:
253 |         try:
254 |             workers = min(3, len(clips))
255 |             with ThreadPoolExecutor(max_workers=workers) as ex:
256 |                 list(ex.map(lambda clip: clip.warm_meta(), clips))
257 |         except Exception:
258 |             # 预热失败不致命，后续按需读取时会再尝试
259 |             pass
260 |
261 |     def movie(self, name: str):
262 |         """按需创建并缓存播放器；索引不存在时保持原有 KeyError 语义。"""
263 |         if name not in self._sources:
264 |             raise KeyError(name)
265 |         movie = self._movies.get(name)
266 |         if movie is not None:
267 |             self._movies.move_to_end(name)
268 |             return movie
269 |         source = self._sources[name]
270 |         if source.media_type == 'gif':
271 |             movie = GIFClip(
272 |                 source.path,
273 |                 parent=self,
274 |                 soft_edges=self._soft_edges,
275 |             )
276 |         else:
277 |             movie = WebMClip(
278 |                 source.path,
279 |                 parent=self,
280 |                 decode_size=self._decode_size,
281 |                 soft_edges=self._soft_edges,
282 |             )
283 |         self._movies[name] = movie
284 |         self._trim_cache()
285 |         return movie
286 |
287 |     def activate(self, name: str):
288 |         """取得并保护当前播放器，LRU 只淘汰非活动动作。"""
289 |         previous = self._active_name
290 |         self._active_name = name
291 |         try:
292 |             movie = self.movie(name)
293 |         except Exception:
294 |             self._active_name = previous
295 |             raise
296 |         self._trim_cache()
297 |         return movie
298 |
299 |     def deactivate(self, name: str | None = None) -> None:
300 |         if name is None or name == self._active_name:

```

```

301 |         self._active_name = None
302 |
303 |     def _trim_cache(self) -> None:
304 |         while len(self._movies) > self._cache_limit:
305 |             candidate = next(
306 |                 (name for name in self._movies if name != self._active_name),
307 |                 None,
308 |             )
309 |             if candidate is None:
310 |                 return
311 |             self._discard(candidate)
312 |
313 |     def _discard(self, name: str) -> None:
314 |         clip = self._movies.pop(name, None)
315 |         if clip is None:
316 |             return
317 |         clip.stop()
318 |         clip.deleteLater()
319 |
320 |     def frames(self, name: str) -> int:
321 |         return self.movie(name).frameCount()
322 |
323 |     def duration(self, name: str) -> float:
324 |         return self.movie(name).duration()
325 |
326 |     def names(self) -> list[str]:
327 |         return list(self._sources.keys())
328 |
329 |     def movies(self) -> dict[str, object]:
330 |         """返回已经创建的播放器；未播放动作不会被强制实例化。"""
331 |         return dict(self._movies)
332 |
333 |     def loaded_count(self) -> int:
334 |         """当前已创建的播放器数量，供诊断和性能回归测试使用。"""
335 |         return len(self._movies)
336 |
337 |     def stop_all(self) -> None:
338 |         """停止全部媒体 reader：角色切换和退出共用这一处清理逻辑。"""
339 |         for clip in self._movies.values():
340 |             clip.stop()
341 |
342 |     def close(self) -> None:
343 |         """关闭媒体库并释放已创建播放器；可重复调用。"""
344 |         self._active_name = None
345 |         for name in list(self._movies):
346 |             self._discard(name)
347 |
348 |     def set_decode_size(self, width: int, height: int) -> None:
349 |         """同步更新所有 WebM clip 的显示解码上限。"""
350 |         self._decode_size = (int(width), int(height))
351 |         for clip in self._movies.values():
352 |             setter = getattr(clip, 'set_decode_size', None)
353 |             if setter is not None:
354 |                 setter(*self._decode_size)
355 |
356 |     def set_soft_edges(self, enabled: bool) -> None:
357 |         """切换 Alpha=1 底噪清理：释放旧播放器，确保后续帧统一处理。"""
358 |         enabled = bool(enabled)
359 |         if enabled == self._soft_edges:
360 |             return
361 |         self.close()
362 |         self._soft_edges = enabled

```

13. pet/performance.py (47 lines)

```

1 | # -*- coding: utf-8 -*-
2 | """轻量系统负载采样与带迟滞的省资源状态判断。"""
3 |
4 | from __future__ import annotations
5 |
6 | import os
7 | from dataclasses import dataclass
8 |
9 |
10 | def system_load_ratio() -> float:
11 |     """返回 1 分钟系统负载/逻辑 CPU 数；不支持时安全回退为 0。"""

```

```

12 |     try:
13 |         load_1m = float(os.getloadavg()[0])
14 |         cores = max(1, int(os.cpu_count() or 1))
15 |         return max(0.0, load_1m / cores)
16 |     except (AttributeError, OSError, TypeError, ValueError):
17 |         return 0.0
18 |
19 |
20 | @dataclass
21 | class LoadGovernor:
22 |     """用连续样本和迟滞避免在普通/省资源模式之间频繁跳动。"""
23 |
24 |     high_threshold: float = 0.75
25 |     low_threshold: float = 0.50
26 |     enter_samples: int = 2
27 |     exit_samples: int = 3
28 |     constrained: bool = False
29 |     _high_count: int = 0
30 |     _low_count: int = 0
31 |
32 |     def observe(self, load_ratio: float) -> bool:
33 |         """记录一次采样, 返回 constrained 状态是否发生变化。"""
34 |         previous = self.constrained
35 |         if self.constrained:
36 |             self._high_count = 0
37 |             self._low_count = self._low_count + 1 if load_ratio <= self.low_threshold else 0
38 |             if self._low_count >= self.exit_samples:
39 |                 self.constrained = False
40 |                 self._low_count = 0
41 |         else:
42 |             self._low_count = 0
43 |             self._high_count = self._high_count + 1 if load_ratio >= self.high_threshold else 0
44 |             if self._high_count >= self.enter_samples:
45 |                 self.constrained = True
46 |                 self._high_count = 0
47 |         return self.constrained != previous

```

14. pet/sound.py (106 lines)

```

1 | # -*- coding: utf-8 -*-
2 | """轻量本地音效: 首次触发时生成并用系统播放器异步播放。"""
3 |
4 | from __future__ import annotations
5 |
6 | import logging
7 | import math
8 | import shutil
9 | import struct
10 | import subprocess
11 | import sys
12 | import wave
13 | from pathlib import Path
14 |
15 |
16 | class DuckScream:
17 |     """生成一段短促的卡通尖叫鸭, 不把音频解码库塞进桌宠包。"""
18 |
19 |     def __init__(self, output_dir: Path) -> None:
20 |         self.path = Path(output_dir) / "sounds" / "screaming-duck.wav"
21 |         self.enabled = True
22 |         self._process: subprocess.Popen | None = None
23 |         self._warned = False
24 |
25 |     def play(self) -> None:
26 |         if not self.enabled:
27 |             return
28 |         try:
29 |             self._ensure_wave()
30 |             if sys.platform == "darwin":
31 |                 player = shutil.which("afplay")
32 |                 if not player:
33 |                     self._warn_once("找不到 afplay, 尖叫鸭音效不可用")
34 |                 return
35 |             if self._process is not None and self._process.poll() is None:
36 |                 self._process.terminate()
37 |             self._process = subprocess.Popen(

```

```

38 |         [player, str(self.path)],
39 |         stdout=subprocess.DEVNULL,
40 |         stderr=subprocess.DEVNULL,
41 |     )
42 |     return
43 | if sys.platform == "win32":
44 |     import winsound
45 |
46 |     winsound.PlaySound(
47 |         str(self.path),
48 |         winsound.SND_FILENAME | winsound.SND_ASYNC,
49 |     )
50 |     return
51 | player = shutil.which("paplay") or shutil.which("aplay")
52 | if player:
53 |     self._process = subprocess.Popen(
54 |         [player, str(self.path)],
55 |         stdout=subprocess.DEVNULL,
56 |         stderr=subprocess.DEVNULL,
57 |     )
58 | except (OSError, RuntimeError, ValueError) as exc:
59 |     self._warn_once("尖叫鸭音效播放失败: %s", exc)
60 |
61 | def close(self) -> None:
62 |     if self._process is not None and self._process.poll() is None:
63 |         try:
64 |             self._process.terminate()
65 |         except OSError:
66 |             pass
67 |     self._process = None
68 |
69 | def _warn_once(self, message: str, *args) -> None:
70 |     if not self._warned:
71 |         logging.warning(message, *args)
72 |         self._warned = True
73 |
74 | def _ensure_wave(self) -> None:
75 |     if self.path.is_file() and self.path.stat().st_size > 44:
76 |         return
77 |     self.path.parent.mkdir(parents=True, exist_ok=True)
78 |     sample_rate = 24_000
79 |     duration = 0.46
80 |     total = int(sample_rate * duration)
81 |     phase = 0.0
82 |     frames = bytearray()
83 |     for index in range(total):
84 |         t = index / sample_rate
85 |         # 先短促“嘎”一下，再上扬成尖叫，最后快速收尾。
86 |         if t < 0.09:
87 |             frequency = 780.0 - 420.0 * (t / 0.09)
88 |         elif t < 0.36:
89 |             frequency = 360.0 + 1_050.0 * ((t - 0.09) / 0.27)
90 |         else:
91 |             frequency = 1_410.0 - 900.0 * ((t - 0.36) / 0.10)
92 |         frequency *= 1.0 + 0.08 * math.sin(2.0 * math.pi * 7.0 * t)
93 |         phase += 2.0 * math.pi * frequency / sample_rate
94 |         envelope = min(1.0, t / 0.012) * min(1.0, (duration - t) / 0.055)
95 |         tone = (
96 |             math.sin(phase)
97 |             + 0.32 * math.sin(phase * 2.01)
98 |             + 0.13 * math.sin(phase * 3.02)
99 |         )
100 |         sample = max(-1.0, min(1.0, tone * 0.24 * envelope))
101 |         frames.extend(struct.pack("<h", int(sample * 32767)))
102 |     with wave.open(str(self.path), "wb") as output:
103 |         output.setnchannels(1)
104 |         output.setsampwidth(2)
105 |         output.setframerate(sample_rate)
106 |         output.writeframes(frames)

```

15. pet/webm_clip.py (502 lines)

```

1 | # -*- coding: utf-8 -*-
2 | """
3 | WebM-backed clip library (webm 主路线)。
4 |

```



```

5 | 使用 imageio-ffmpeg 自带的静态 ffmpeg 解码透明 webm:
6 | - read_frames(..., pix_fmt='rgba', bits_per_pixel=32, input_params=['-c:v','libvpx-vp9'])
7 | 可正确保留 VP9 alpha, 输出 RGBA 原始帧。
8 | - imageio-ffmpeg 内部在 Windows 上使用 STARTUPINFO 隐藏控制台窗口,
9 | 避免旧 ffmpeg 子进程方案导致的“窗口反复出现/消失”。
10 |
11 | 线程模型:
12 | - 后台 reader 线程只负责把 RGBA 字节放入有界队列;
13 | - 主线程 QTimer 按视频 fps 从队列取帧, 只构造裁边 QImage 并发出 frameChanged;
14 | - 所有 Qt GUI 操作只发生在主线程。
15 | """
16 |
17 | from __future__ import annotations
18 |
19 | from dataclasses import dataclass
20 | import logging
21 | import queue
22 | import threading
23 |
24 | from PySide6.QtCore import QObject, QTimer, Signal
25 | from PySide6.QtGui import QPixmap, QImage, QPainter, QPixelFormat, QRegion
26 |
27 | from . import catalog
28 |
29 | logger = logging.getLogger(__name__)
30 | TRANSPARENT_CROP_PADDING = 8
31 |
32 |
33 | @dataclass(frozen=True, slots=True)
34 | class DecodedFrame:
35 |     """裁掉透明边缘后的图像, 以及它在原解码画布中的位置。"""
36 |
37 |     image: QImage
38 |     x: int
39 |     y: int
40 |     canvas_width: int
41 |     canvas_height: int
42 |
43 |     @property
44 |     def offset(self) -> tuple[int, int]:
45 |         return self.x, self.y
46 |
47 |     @property
48 |     def canvas_size(self) -> tuple[int, int]:
49 |         return self.canvas_width, self.canvas_height
50 |
51 |
52 | def clear_alpha_floor(image: QImage) -> QImage:
53 |     """清除 VP9 Alpha 常见的精确 1 阶底噪, 不改变真实半透明边缘。"""
54 |     result = image.convertToFormat(QImage.Format.Format_ARGB32_Premultiplied)
55 |     if result.isNull():
56 |         return result
57 |
58 |     # 用 C 实现的 bytes.translate 生成“仅 Alpha=1 可见”的清除遮罩,
59 |     # 再用 DestinationOut 清除这些像素。这样 Alpha=2 及以上(真实抗锯齿
60 |     # 边缘)完全原样保留, 且会同步清除透明像素里残留的 RGB。
61 |     alpha = result.convertToFormat(QImage.Format.Format_Alpha8)
62 |     floor_mask = bytes(255 if value == 1 else 0 for value in range(256))
63 |     mask_alpha = QImage(
64 |         result.width(),
65 |         result.height(),
66 |         QImage.Format.Format_Alpha8,
67 |     )
68 |     mask_alpha.bits()[0:] = bytes(alpha.bits()).translate(floor_mask)
69 |     mask = QImage(
70 |         result.width(),
71 |         result.height(),
72 |         QImage.Format.Format_ARGB32_Premultiplied,
73 |     )
74 |     mask.fill(0xFFFFFFFF)
75 |     mask.setAlphaChannel(mask_alpha)
76 |     painter = QPainter(result)
77 |     painter.setCompositionMode(
78 |         QPainter.CompositionMode.CompositionMode_DestinationOut
79 |     )
80 |     painter.drawImage(0, 0, mask)
81 |     painter.end()
82 |     return result
83 |

```

```

84 |
85 | def trim_transparent_frame(
86 |     image: QImage,
87 |     *,
88 |     padding: int = TRANSPARENT_CROP_PADDING,
89 |     soften_edges: bool = False,
90 | ) -> DecodedFrame:
91 |     """裁掉透明压缩噪声，并保留安全边与原画布坐标。"""
92 |     canvas_width, canvas_height = image.width(), image.height()
93 |     if image.isNull():
94 |         return DecodedFrame(QImage(), 0, 0, canvas_width, canvas_height)
95 |
96 |     # Alpha=1 必须在计算边界前清掉：否则它虽然不会进入最终画面，仍会把
97 |     # 每一帧的裁剪范围撑大，造成动画边缘/尺寸在播放时抖动。
98 |     working = clear_alpha_floor(image) if soften_edges else image
99 |     bounds = QRegion(QBitmap.fromImage(working.createAlphaMask())).boundingRect()
100 |     if bounds.isEmpty():
101 |         transparent = QImage(
102 |             1,
103 |             1,
104 |             QImage.Format.Format_ARGB32_Premultiplied,
105 |         )
106 |         transparent.fill(0)
107 |         return DecodedFrame(transparent, 0, 0, canvas_width, canvas_height)
108 |
109 |     padding = max(0, int(padding))
110 |     if padding:
111 |         bounds = bounds.adjusted(-padding, -padding, padding, padding).intersected(
112 |             image.rect()
113 |         )
114 |
115 |     cropped = working.copy(bounds).convertToFormat(
116 |         QImage.Format.Format_ARGB32_Premultiplied
117 |     )
118 |
119 |     return DecodedFrame(
120 |         cropped,
121 |         bounds.x(),
122 |         bounds.y(),
123 |         canvas_width,
124 |         canvas_height,
125 |     )
126 |
127 |
128 | def frame_canvas_image(frame: DecodedFrame) -> QImage:
129 |     """兼容接口需要完整画布时才还原：动画热路径不调用。"""
130 |     if frame.canvas_width <= 0 or frame.canvas_height <= 0:
131 |         return QImage()
132 |     canvas = QImage(
133 |         frame.canvas_width,
134 |         frame.canvas_height,
135 |         QImage.Format.Format_ARGB32,
136 |     )
137 |     canvas.fill(0)
138 |     painter = QPainter(canvas)
139 |     painter.drawImage(frame.x, frame.y, frame.image)
140 |     painter.end()
141 |     return canvas
142 |
143 | # 解码输出最多 1280×720, RGBA 单帧约 3.5 MiB。2 帧覆盖约 83ms (24fps),
144 | # 在保留短时调度缓冲的同时，把队列峰值控制在约 7 MiB。
145 | FRAME_QUEUE_SIZE = 2
146 |
147 | # 2560×1440 母版由 ffmpeg 在解码进程内用 Lanczos 缩到 Retina 真正需要的
148 | # 最大 backing size: 较小的外部角色不会被反向放大。
149 | def _decode_filter(width: int, height: int) -> str:
150 |     return (
151 |         "scale=w='min(%d,iw)':h='min(%d,ih)':"
152 |         "force_original_aspect_ratio=decrease:force_divisible_by=2:flags=lanczos"
153 |     ) % (width, height)
154 |
155 | # 进程内元数据缓存：避免反复切换角色时重复调用 count_frames_and_secs
156 | _META_CACHE: dict[str, tuple[int, float]] = {}
157 |
158 | try:
159 |     import imageio_ffmpeg
160 | except Exception as exc: # pragma: no cover - 依赖缺失时无法使用 webm 路线
161 |     imageio_ffmpeg = None
162 |     _IMPORT_ERROR = exc

```

```

163 | else:
164 |     _IMPORT_ERROR = None
165 |
166 |
167 | class WebMClip(QObject):
168 |     """与窗口层期望的媒体播放器接口兼容。"""
169 |
170 |     available = imageio_ffmpeg is not None
171 |
172 |     frameChanged = Signal(int)
173 |     finished = Signal()
174 |     errorOccurred = Signal(str)
175 |
176 |     def __init__(
177 |         self,
178 |         path,
179 |         parent: QObject | None = None,
180 |         *,
181 |         decode_size: tuple[int, int] | None = None,
182 |         soft_edges: bool = False,
183 |     ) -> None:
184 |         super().__init__(parent)
185 |         self.path = path
186 |         self._bpp = 4 # RGBA
187 |         self._decode_max_w, self._decode_max_h = (
188 |             decode_size or catalog.decode_size_for_scale(catalog.DEFAULT_SCALE)
189 |         )
190 |         self._decode_filter = _decode_filter(self._decode_max_w, self._decode_max_h)
191 |         self._soft_edges = bool(soft_edges)
192 |         self._source_w = catalog.CANVAS_W
193 |         self._source_h = catalog.CANVAS_H
194 |         self._frame_w = catalog.CANVAS_W
195 |         self._frame_h = catalog.CANVAS_H
196 |
197 |         # 元数据 (惰性填充, 由 MovieLibrary 并行 warm 或首次使用时读取)
198 |         self._frame_count = 0
199 |         self._duration = 0.0
200 |         self._fps = 24.0
201 |         self._playback_speed = 1.0
202 |
203 |         # 播放状态
204 |         self._queue: queue.Queue = queue.Queue(maxsize=FRAME_QUEUE_SIZE)
205 |         self._stop_evt = threading.Event()
206 |         self._thread: threading.Thread | None = None
207 |         self._timer = QTimer(self)
208 |         self._timer.setInterval(self._timer_interval())
209 |         self._timer.timeout.connect(self._poll)
210 |
211 |         self._current_frame: DecodedFrame | None = None
212 |         self._first_frame: DecodedFrame | None = None
213 |         self._frame_index = 0
214 |         self._ended_fired = False
215 |         self._running = False
216 |
217 |         # ----- metadata
218 |         def _ensure_meta(self) -> None:
219 |             if self._duration > 0 or imageio_ffmpeg is None:
220 |                 return
221 |             key = str(self.path)
222 |             cached = _META_CACHE.get(key)
223 |             if cached is not None:
224 |                 self._frame_count, self._duration = cached
225 |                 if self._frame_count > 0 and self._duration > 0:
226 |                     self._fps = self._frame_count / self._duration
227 |                 return
228 |             try:
229 |                 frames, secs = imageio_ffmpeg.count_frames_and_secs(key)
230 |                 if frames and frames > 0:
231 |                     self._frame_count = int(frames)
232 |                     if secs and secs > 0:
233 |                         self._duration = float(secs)
234 |                         if self._frame_count > 0 and self._duration > 0:
235 |                             self._fps = self._frame_count / self._duration
236 |                         _META_CACHE[key] = (self._frame_count, self._duration)
237 |             except Exception as exc:
238 |                 logger.warning('webm 元数据读取失败 %s: %s', self.path, exc)
239 |                 # 保留默认值, 后续 reader 会尝试从 read_frames 的 meta 补充
240 |
241 |         def warm_meta(self) -> None:

```

```

242 |     """预取元数据（可被线程池并行调用）。"""
243 |     self._ensure_meta()
244 |
245 |     def _timer_interval(self) -> int:
246 |         if self._fps > 0:
247 |             return max(1, int(round(1000 / (self._fps * self.playback_speed))))
248 |         return max(1, int(round(catalog.FRAME_MS / self.playback_speed)))
249 |
250 |     def frameCount(self) -> int:
251 |         if self._frame_count <= 0:
252 |             self._ensure_meta()
253 |         return max(1, self._frame_count)
254 |
255 |     def duration(self) -> float:
256 |         if self._duration <= 0:
257 |             self._ensure_meta()
258 |         return self._duration / self.playback_speed if self._duration > 0 else 0.0
259 |
260 |     def currentFrameNumber(self) -> int:
261 |         return self._frame_index
262 |
263 |     def currentTimeSeconds(self) -> float:
264 |         if self._fps <= 0:
265 |             return 0.0
266 |         return self._frame_index / (self._fps * self.playback_speed)
267 |
268 |     def currentFrame(self) -> DecodedFrame | None:
269 |         return self._current_frame
270 |
271 |     def currentPixmap(self):
272 |         """旧诊断接口：运行时窗口直接消费 currentFrame(), 不做往返转换。"""
273 |         if self._current_frame is None:
274 |             return QPixmap()
275 |         return QPixmap.fromImage(frame_canvas_image(self._current_frame))
276 |
277 |     def sourceSize(self) -> tuple[int, int]:
278 |         """编码素材尺寸：用于诊断和验收，不影响逻辑画布。"""
279 |         return self._source_w, self._source_h
280 |
281 |     def decodedSize(self) -> tuple[int, int]:
282 |         """送入 Qt 的有界 RGBA 帧尺寸。"""
283 |         return self._frame_w, self._frame_h
284 |
285 |     def set_decode_size(self, width: int, height: int) -> None:
286 |         """更新显示所需解码上限；下次播放按新尺寸重建首帧与 reader。"""
287 |         width, height = max(2, int(width)), max(2, int(height))
288 |         if (width, height) == (self._decode_max_w, self._decode_max_h):
289 |             return
290 |         self.stop()
291 |         self._decode_max_w, self._decode_max_h = width, height
292 |         self._decode_filter = _decode_filter(width, height)
293 |         self._first_frame = None
294 |         self._current_frame = None
295 |
296 |     # ----- lifecycle
297 |     def set_playback_speed(self, speed: float) -> None:
298 |         self.playback_speed = max(0.1, float(speed))
299 |         if self._timer.isActive():
300 |             self._timer.setInterval(self._timer_interval())
301 |
302 |     def start(self) -> None:
303 |         if self._running:
304 |             return
305 |         if imageio_ffmpeg is None:
306 |             self.errorOccurred.emit(str(_IMPORT_ERROR or 'imageio_ffmpeg 不可用'))
307 |             return
308 |
309 |         self._stop_evt = threading.Event()
310 |         self._queue = queue.Queue(maxsize=FRAME_QUEUE_SIZE)
311 |         self._frame_index = 0
312 |         self._ended_fired = False
313 |         self._running = True
314 |
315 |         frame_queue = self._queue
316 |         self._thread = threading.Thread(
317 |             target=self._reader,
318 |             args=(self._stop_evt, frame_queue),
319 |             daemon=True,
320 |         )

```

```

321 |         self._thread.start()
322 |         self._timer.start()
323 |
324 |     def stop(self) -> None:
325 |         self._running = False
326 |         self._timer.stop()
327 |         if self._stop_evt is not None:
328 |             self._stop_evt.set()
329 |         thread = self._thread
330 |         self._thread = None
331 |         # reader 在队列背压处最多每 100ms 检查一次 stop_evt; 短暂 join 能让
332 |         # 角色切换/退出真正收口, 同时不会把 UI 卡在 ffmpeg 子进程上。
333 |         if thread is not None and thread is not threading.current_thread() and thread.is_alive():
334 |             thread.join(timeout=0.15)
335 |
336 |     def jumpToFrame(self, frame_index: int) -> bool:
337 |         # 本项目只需要回到首帧; 完整 seek 通过重启 reader + 丢弃帧实现。
338 |         if frame_index <= 0:
339 |             self.stop()
340 |             self._frame_index = 0
341 |             if self._first_frame is not None:
342 |                 self._current_frame = self._first_frame
343 |             else:
344 |                 self._current_frame = None
345 |                 self._decode_first_frame_sync()
346 |             return True
347 |         return False
348 |
349 |     def _decode_first_frame_sync(self) -> None:
350 |         """同步解码首帧, 保证 jumpToFrame(0)/currentPixmap 在 start() 前有画面。"""
351 |         if imageio_ffmpeg is None:
352 |             return
353 |         gen = None
354 |         try:
355 |             gen = imageio_ffmpeg.read_frames(
356 |                 str(self.path),
357 |                 pix_fmt='rgba',
358 |                 bits_per_pixel=self._bpp * 8,
359 |                 input_params=['-c:v', 'libvpx-vp9'],
360 |                 output_params=['-vf', self._decode_filter],
361 |             )
362 |             meta = next(gen)
363 |             self._apply_stream_meta(meta)
364 |             frame = next(gen)
365 |             if meta.get('fps'):
366 |                 self._fps = float(meta['fps'])
367 |             if meta.get('duration'):
368 |                 self._duration = float(meta['duration'])
369 |             if self._frame_count <= 0 and self._fps > 0 and self._duration > 0:
370 |                 self._frame_count = int(round(self._fps * self._duration))
371 |             expect = self._frame_w * self._frame_h * self._bpp
372 |             if len(frame) == expect:
373 |                 img = QImage(frame, self._frame_w, self._frame_h, self._frame_w * self._bpp,
374 |                             QImage.Format.Format_RGBA8888)
375 |                 if not img.isNull():
376 |                     self._current_frame = trim_transparent_frame(
377 |                         img,
378 |                         soften_edges=self._soft_edges,
379 |                     )
380 |                     self._first_frame = self._current_frame
381 |         except Exception as exc:
382 |             logger.warning('webm 首帧预解码失败 %s: %s', self.path, exc)
383 |         finally:
384 |             if gen is not None:
385 |                 try:
386 |                     gen.close()
387 |                 except Exception:
388 |                     pass
389 |
390 |     # ----- reader
391 |     def _apply_stream_meta(self, meta: dict) -> None:
392 |         source_size = meta.get('source_size') or meta.get('size')
393 |         frame_size = meta.get('size') or source_size
394 |         if isinstance(source_size, (tuple, list)) and len(source_size) == 2:
395 |             try:
396 |                 width, height = int(source_size[0]), int(source_size[1])
397 |                 if width > 0 and height > 0:
398 |                     self._source_w, self._source_h = width, height
399 |             except (TypeError, ValueError):

```

```

400 |         pass
401 |     if isinstance(frame_size, (tuple, list)) and len(frame_size) == 2:
402 |         try:
403 |             width, height = int(frame_size[0]), int(frame_size[1])
404 |             if width > 0 and height > 0:
405 |                 self._frame_w, self._frame_h = width, height
406 |         except (TypeError, ValueError):
407 |             pass
408 |
409 | def _reader(self, stop_evt: threading.Event, frame_queue: queue.Queue) -> None:
410 |     gen = None
411 |     try:
412 |         gen = imageio_ffmpeg.read_frames(
413 |             str(self.path),
414 |             pix_fmt='rgba',
415 |             bits_per_pixel=self._bpp * 8,
416 |             input_params=['-c:v', 'libvpx-vp9'],
417 |             output_params=['-vf', self._decode_filter],
418 |         )
419 |         meta = next(gen)
420 |         self._apply_stream_meta(meta)
421 |         # 用实际流信息修正元数据
422 |         if meta.get('fps'):
423 |             self._fps = float(meta['fps'])
424 |         if meta.get('duration'):
425 |             self._duration = float(meta['duration'])
426 |         if self._frame_count <= 0 and self._fps > 0 and self._duration > 0:
427 |             self._frame_count = int(round(self._fps * self._duration))
428 |
429 |         for frame in gen:
430 |             if not self._queue_frame(frame_queue, frame, stop_evt):
431 |                 break
432 |             # 正常播完时放入结束标记
433 |             if not stop_evt.is_set():
434 |                 self._queue_frame(frame_queue, None, stop_evt)
435 |     except Exception as exc:
436 |         logger.exception('webm 解码失败: %s', self.path)
437 |         self.errorOccurred.emit(str(exc))
438 |         # 异常中断也要放入结束标记, 避免动画链卡在最后一帧
439 |         try:
440 |             frame_queue.put(None, timeout=0.2)
441 |         except Exception:
442 |             pass
443 |     finally:
444 |         if gen is not None:
445 |             try:
446 |                 gen.close()
447 |             except Exception:
448 |                 pass
449 |
450 | @staticmethod
451 | def _queue_frame(
452 |     frame_queue: queue.Queue,
453 |     item: bytes | None,
454 |     stop_evt: threading.Event,
455 | ) -> bool:
456 |     """按播放速度解压解码器, 避免队列满时继续解码并丢帧。"""
457 |     while not stop_evt.is_set():
458 |         try:
459 |             frame_queue.put(item, timeout=0.1)
460 |             return True
461 |         except queue.Full:
462 |             continue
463 |     return False
464 |
465 | def _poll(self) -> None:
466 |     """主线程按视频帧率逐帧取帧, 不跳帧、不积压追帧。
467 |
468 |     注意: 不能一次清空队列只处理最新帧, 否则会把中间帧丢弃,
469 |     导致动画视觉上“快进”。这里每次只取最早的一帧。
470 |     """
471 |     try:
472 |         item = self._queue.get_nowait()
473 |     except queue.Empty:
474 |         return
475 |
476 |     if item is None:
477 |         # 正常播完: 若在处理最后一帧时已经由窗口层启动了下一个动画,
478 |         # self._queue 已被替换, 不会走到这里。

```

```

479 |         if not self._ended_fired:
480 |             self._ended_fired = True
481 |             self._running = False
482 |             self._timer.stop()
483 |             self.finished.emit()
484 |         return
485 |
486 |     self._process_frame(item)
487 |
488 |     def _process_frame(self, data: bytes) -> None:
489 |         expect = self._frame_w * self._frame_h * self._bpp
490 |         if len(data) != expect:
491 |             logger.warning('webm 帧长度异常: got=%d expect=%d', len(data), expect)
492 |             return
493 |         img = QImage(data, self._frame_w, self._frame_h, self._frame_w * self._bpp,
494 |                       QImage.Format.Format_RGBA8888)
495 |         if img.isNull():
496 |             return
497 |         self._current_frame = trim_transparent_frame(
498 |             img,
499 |             soften_edges=self._soft_edges,
500 |         )
501 |         self._frame_index += 1
502 |         self.frameChanged.emit(self._frame_index)

```

16. pet/window.py (1440 lines)

```

1 | # -*- coding: utf-8 -*-
2 | """
3 | 桌宠主窗口 —— 透明无边框置顶窗口 + 动画链状态机 + 移动驱动 + 交互。
4 |
5 | 状态机（对应原插件 dsh-pet lib/client.js 的链式模型，行为 1:1 移植）：
6 | - 每个动画一次性播放，播完按概率选下一个：30% 待机 / 10% 转向 / 40% 动作 / 20% 移动；
7 | - 转向（东张西望）播完翻转朝向：facing=right 时水平镜像；
8 | - 点击回应 / 拖拽动画播完先回待缓冲，待机播完再进随机链；
9 | - 移动：动画只提供“走路姿态”（3 选 1），位置由 QTimer 驱动，
10 | 开头/结尾各 2s 不动，中间按播放进度插值；
11 | - 透明区域鼠标穿透：每帧用当前帧 alpha 生成窗口 mask（等效原版命中层设计）。
12 | """
13 |
14 | from __future__ import annotations
15 |
16 | import logging
17 | import math
18 | import random
19 | import sys
20 | import time
21 |
22 | from PySide6.QtCore import QElapsedTimer, QPoint, Qt, QTimer, Signal
23 | from PySide6.QtGui import QBitmap, QCursor, QImage, QPainter, QPixmap
24 | from PySide6.QtWidgets import QApplication, QMenu, QWidget
25 |
26 | from . import autostart as autostart_mod
27 | from . import catalog
28 | from .action_dialog import ActionSetDialog
29 | from .config import Config
30 | from .interaction import classify_tap_burst, cursor_facing, edge_contacts
31 | from .library import MovieLibrary
32 | from .performance import LoadGovernor, system_load_ratio
33 | from .sound import DuckScream
34 | from .webm_clip import frame_canvas_image
35 |
36 |
37 | def _mac_set_window_level(view_id: int, level: int) -> bool:
38 |     """macOS 原生：把 NSWindow 层级设为指定值（3=置顶浮动，0=普通）。
39 |
40 |     Qt 的 WindowStaysOnTopHint 在 macOS 上对无边框 Tool 窗口/运行时切换不可靠，
41 |     这里用 objc runtime 直接调 [NSWindow setLevel:] 强制生效（ctypes 零依赖）。
42 |     """
43 |     app = QApplication.instance()
44 |     if (
45 |         sys.platform != 'darwin'
46 |         or app is None
47 |         or app.platformName().lower() != 'cocoa'
48 |     ):
49 |         return False

```

```

50 |     try:
51 |         import ctypes
52 |         import ctypes.util
53 |
54 |         lib_path = ctypes.util.find_library('objc') or '/usr/lib/libobjc.A.dylib'
55 |         objc = ctypes.cdll.LoadLibrary(lib_path)
56 |
57 |         # 关键: sel_registerName 返回 SEL (64 位指针)。ctypes 默认按 c_int(32 位)
58 |         # 截断返回值, 损坏的 SEL 会让 ObjC runtime 段错误 (SIGSEGV), 必须显式声明
59 |         objc.sel_registerName.restype = ctypes.c_void_p
60 |         objc.sel_registerName.argtypes = [ctypes.c_char_p]
61 |
62 |         msg = objc.objc_msgSend
63 |         msg.restype = ctypes.c_void_p
64 |
65 |         sel_window = objc.sel_registerName(b'window')
66 |         sel_set_level = objc.sel_registerName(b'setLevel:')
67 |
68 |         # [view window] —— 无参, 返回 NSWindow*
69 |         msg.argtypes = [ctypes.c_void_p, ctypes.c_void_p]
70 |         window = msg(ctypes.c_void_p(view_id), sel_window)
71 |         if not window:
72 |             return False
73 |
74 |         # [window setLevel:level] —— 一个 NSInteger 参数
75 |         msg.argtypes = [ctypes.c_void_p, ctypes.c_void_p, ctypes.c_long]
76 |         msg(ctypes.c_void_p(window), sel_set_level, level)
77 |         return True
78 |     except Exception:
79 |         return False
80 |
81 |
82 | class PetWindow(QWidget):
83 |     """桌宠窗口本体。"""
84 |
85 |     softEdgesChanged = Signal(bool)
86 |     duckSoundChanged = Signal(bool)
87 |     proactiveGreetingsChanged = Signal(bool)
88 |     playlistChanged = Signal(str)
89 |     personalityChanged = Signal(str)
90 |     pausedChanged = Signal(bool)
91 |
92 |     def __init__(self, lib: MovieLibrary, config: Config) -> None:
93 |         super().__init__()
94 |         self.lib = lib
95 |         # 让窗口成为媒体库的生命周期所有者: 角色切换/退出时不留下孤立 reader。
96 |         if lib.parent() is None:
97 |             lib.setParent(self)
98 |         self.cfg = config
99 |         self.on_switch_character = None # 由 app 注入, 用于运行时切换角色
100 |         self.soft_edges: bool = bool(config.get('soft_edges', True))
101 |         self.lib.set_soft_edges(self.soft_edges)
102 |
103 |         # 根据当前形象实际拥有的动画动态计算分类, 支持不同角色动作不一致
104 |         self.cats = catalog.build_categories(lib.names(), getattr(lib, 'manifest', None), getattr(lib, 'folder_map', None), getattr(lib, 'folder_files', None))
105 |         self.idle = self.cats['idle']
106 |         self.turn = self.cats['turn']
107 |         self.idles = self.cats['idles']
108 |         self.turns = self.cats['turns']
109 |         self.moves = self.cats['moves']
110 |         self.clicks = self.cats['clicks']
111 |         self.drag = self.cats['drag']
112 |         self.acts = self.cats['acts']
113 |         available = set(lib.names())
114 |         self.favorites = [
115 |             name for name in self._config_names('favorites') if name in available
116 |         ]
117 |         self.playlist = [
118 |             name for name in self._config_names('playlist') if name in available
119 |         ]
120 |         self.playlist_mode = str(config.get('playlist_mode', 'off'))
121 |         if self.playlist_mode not in ('off', 'loop', 'random'):
122 |             self.playlist_mode = 'off'
123 |         self._playlist_index = -1
124 |         self.personality = self._normalize_personality(
125 |             config.get('personality', 'lively')
126 |         )
127 |
128 |         # 预载拖拽动画首帧, 避免第一次进入拖拽状态时同步解码卡顿

```



```

129 |         if self.drag:
130 |             self.lib.movie(self.drag).jumpToFrame(0)
131 |
132 |         self.playback_speed: float = float(config.get('playback_speed', 1.0))
133 |         self.mouse_through: bool = bool(config.get('mouse_through', False))
134 |         self.drag_physics: bool = bool(config.get('drag_physics', False))
135 |         self.duck_sound_enabled: bool = bool(config.get('duck_sound', True))
136 |         self.proactive_greetings: bool = bool(
137 |             config.get('proactive_greetings', True)
138 |         )
139 |         self._duck_sound = DuckScream(config.dir)
140 |         self._duck_sound.enabled = self.duck_sound_enabled
141 |
142 |         # ---- 窗口属性: 无边框 + 透明 + 不进任务栏: 置顶可配置 ----
143 |         flags = Qt.WindowType.FramelessWindowHint | Qt.WindowType.Tool
144 |         if config.get('on_top', True):
145 |             flags |= Qt.WindowType.WindowStaysOnTopHint
146 |         self.setWindowFlags(flags)
147 |         self.setAttribute(Qt.WidgetAttribute.WA_TranslucentBackground)
148 |         if self.mouse_through:
149 |             self.setWindowFlag(Qt.WindowType.WindowTransparentForInput, True)
150 |         if sys.platform == 'darwin' and config.get('on_top', True):
151 |             # macOS 上 Tool 窗口的置顶由 WA_MacAlwaysShowToolWindow 控制。
152 |             # WindowStaysOnTopHint 对 Tool 窗口不可靠 (Qt 官方已知问题 QTBUG-38580)
153 |             self.setAttribute(Qt.WidgetAttribute.WA_MacAlwaysShowToolWindow, True)
154 |
155 |         # ---- 状态 ----
156 |         self.anim: str = self.idle
157 |         self.facing: str = config.get('facing', 'left') # left | right
158 |         self.scale: float = float(config.get('scale', catalog.DEFAULT_SCALE))
159 |         self.no_move: bool = bool(config.get('no_move', False)) # 不移动: 禁用自动移动
160 |         self.movie = None
161 |         self._bound_movie = None
162 |         self._bound_movie_name: str | None = None
163 |         self._frame_pixmap: QPixmap | None = None
164 |         self._frame_logical_size = (0, 0)
165 |         self._frame_logical_rect = (0, 0, 0, 0)
166 |         self._mask_frame_counter = 0
167 |         self._ended_fired = False
168 |         self._shutting_down = False
169 |         self._suspended = False
170 |         self._paused = False
171 |
172 |         # ---- 交互状态 ----
173 |         self._press_global: QPoint | None = None
174 |         self._grab_offset: QPoint | None = None # 按下时 鼠标全局坐标 - 窗口左上角
175 |         self._dragging = False
176 |         self._just_dragged = False # 抑制拖拽结束后的幽灵点击
177 |         self._long_press_fired = False
178 |         self._tap_count = 0
179 |
180 |         self._long_press_timer = QTimer(self)
181 |         self._long_press_timer.setSingleShot(True)
182 |         self._long_press_timer.timeout.connect(self._on_long_press)
183 |         self._tap_timer = QTimer(self)
184 |         self._tap_timer.setSingleShot(True)
185 |         self._tap_timer.timeout.connect(self._flush_tap_burst)
186 |
187 |         # ---- 移动驱动 ----
188 |         self._move_plan: dict | None = None
189 |         self._move_timer = QTimer(self)
190 |         self._move_timer.setInterval(33) # ~30fps 位置插值
191 |         self._move_timer.timeout.connect(self._on_move_tick)
192 |
193 |         # ---- 点击 Q 弹效果 ----
194 |         self._squash_timer = QTimer(self)
195 |         self._squash_timer.setInterval(16)
196 |         self._squash_timer.timeout.connect(self._on_squash_tick)
197 |         self._squash_clock = QElapsedTimer()
198 |         self._squash_active = False
199 |         self._squash_duration_ms = 220
200 |         self._squash_progress = 1.0
201 |
202 |         # ---- 自适应省资源: 系统持续繁忙时降低动作/解码频率 ----
203 |         self._load_governor = LoadGovernor()
204 |         self._resource_constrained = False
205 |         self._resource_timer = QTimer(self)
206 |         self._resource_timer.setInterval(catalog.LOAD_SAMPLE_MS)
207 |         self._resource_timer.timeout.connect(self._check_system_load)

```

```

208 |         self._idle_pause_timer = QTimer(self)
209 |         self._idle_pause_timer.setSingleShot(True)
210 |         self._idle_pause_timer.timeout.connect(self._resume_busy_idle)
211 |
212 |         # ---- 拖动物理 ----
213 |         self._physics_timer = QTimer(self)
214 |         self._physics_timer.setInterval(16)
215 |         self._physics_timer.timeout.connect(self._on_physics_tick)
216 |         self._physics_mode: str | None = None # None / 'drag' / 'throw'
217 |         self._phys_pos = [0.0, 0.0]
218 |         self._phys_vel = [0.0, 0.0]
219 |         self._drag_target: QPoint | None = None
220 |         self._last_global: QPoint | None = None
221 |         self._last_move_time = 0.0
222 |
223 |         # ---- 鼠标靠近与主动问候 ----
224 |         self._cursor_timer = QTimer(self)
225 |         self._cursor_timer.setInterval(catalog.CURSOR_POLL_MS)
226 |         self._cursor_timer.timeout.connect(self._on_cursor_tick)
227 |         self._greeting_timer = QTimer(self)
228 |         self._greeting_timer.setSingleShot(True)
229 |         self._greeting_timer.timeout.connect(self._on_greeting_timer)
230 |
231 |         # ---- 尺寸与初始状态 ----
232 |         self._apply_scale()
233 |         self._restore_position()
234 |         self._switch(self.idle)
235 |         self._resource_timer.start()
236 |         self._check_system_load()
237 |         self._cursor_timer.start()
238 |         self._schedule_next_greeting()
239 |
240 |     def _config_names(self, key: str) -> list[str]:
241 |         """读取配置中的动作名列表，过滤异常值并保持原顺序。"""
242 |         value = self.cfg.get(key, [])
243 |         return [name for name in value if isinstance(name, str)] if isinstance(value, list) else []
244 |
245 |     @staticmethod
246 |     def _normalize_personality(value: str) -> str:
247 |         return value if value in catalog.PERSONALITY_PRESETS else 'lively'
248 |
249 |     # ===== 尺寸
250 |     def _apply_scale(self) -> None:
251 |         """按逻辑画布缩放窗口：素材分辨率不会改变桌宠大小。"""
252 |         self._w = max(1, int(round(catalog.CANVAS_W * self.scale)))
253 |         self._h = max(1, int(round((catalog.CANVAS_H + catalog.PAD) * self.scale)))
254 |         self.setFixedSize(self._w, self._h)
255 |
256 |     def change_scale(self, scale: float) -> None:
257 |         """切换缩放：保持窗口底边不动（脚踩的地面不变）。"""
258 |         if abs(scale - self.scale) < 1e-6:
259 |             return
260 |         old_bottom = self.geometry().bottom()
261 |         self.scale = scale
262 |         self.lib.set_decode_size(*catalog.decode_size_for_scale(scale))
263 |         self._apply_scale()
264 |         self.move(self.x(), old_bottom - self._h + 1)
265 |         self._switch(self.anim)
266 |         self.update()
267 |         self._save_position()
268 |
269 |     # ===== 位置
270 |     def _screen_available(self):
271 |         """窗口所在屏幕：macOS 上 self.screen() 可能失效，兜底主屏。"""
272 |         from PySide6.QtGui import QGuiApplication
273 |         scr = self.screen()
274 |         if scr is None:
275 |             scr = QGuiApplication.primaryScreen()
276 |         return scr
277 |
278 |     def _restore_position(self) -> None:
279 |         """恢复上次位置（按屏幕比例），无记录则落右下角。"""
280 |         scr = self._screen_available()
281 |         avail = scr.availableGeometry()
282 |         rx, ry = self.cfg.get('rx'), self.cfg.get('ry')
283 |         if rx is None or ry is None:
284 |             x = avail.right() - self._w - catalog.CORNER_MARGIN
285 |             y = avail.bottom() - self._h
286 |         else:

```

```

287 |         x = int(round(avail.left() + rx * avail.width())) - self._w // 2
288 |         y = int(round(avail.top() + ry * avail.height())) - self._h // 2
289 |         x = min(max(x, avail.left()), avail.right() - self._w)
290 |         y = min(max(y, avail.top()), avail.bottom() - self._h)
291 |         logging.info('恢复位置 screen=%s avail=(%d,%d,%d,%d) dpr=%s -> (%d,%d)',
292 |                     scr.name(), avail.left(), avail.top(), avail.right(),
293 |                     avail.bottom(), scr.devicePixelRatio(), x, y)
294 |         self.move(x, y)
295 |
296 |     def _save_position(self) -> None:
297 |         """以"窗口中心相对屏幕可用区"的比例"持久化位置（分辨率变化后仍正确）。"""
298 |         scr = self._screen_available()
299 |         avail = scr.availableGeometry()
300 |         if avail.width() <= 0 or avail.height() <= 0:
301 |             return
302 |         cx = self.x() + self._w / 2
303 |         cy = self.y() + self._h / 2
304 |         self.cfg.set('rx', (cx - avail.left()) / avail.width())
305 |         self.cfg.set('ry', (cy - avail.top()) / avail.height())
306 |         self.cfg.set('facing', self.facing)
307 |         self.cfg.set('scale', self.scale)
308 |         self.cfg.save()
309 |
310 |     def _go_default_corner(self) -> None:
311 |         scr = self._screen_available()
312 |         avail = scr.availableGeometry()
313 |         x = avail.right() - self._w - catalog.CORNER_MARGIN
314 |         y = avail.bottom() - self._h
315 |         logging.info('回到右下角 screen=%s avail=(%d,%d,%d,%d) dpr=%s -> (%d,%d)',
316 |                     scr.name(), avail.left(), avail.top(), avail.right(),
317 |                     avail.bottom(), scr.devicePixelRatio(), x, y)
318 |         self.move(x, y)
319 |         self._save_position()
320 |
321 |     def set_on_top(self, on: bool) -> None:
322 |         if sys.platform == 'darwin':
323 |             # 先设属性再改 flag: setWindowFlag 触发窗口重建时一并应用
324 |             self.setAttribute(Qt.WidgetAttribute.WA_MacAlwaysShowToolWindow, on)
325 |             self.setWindowFlag(Qt.WindowType.WindowStaysOnTopHint, on)
326 |             self.cfg.set('on_top', on)
327 |             self.cfg.save()
328 |             self.show()
329 |         if sys.platform == 'darwin':
330 |             # 延迟到 Qt 窗口重建完成后强制原生层级，避免被 Qt 覆盖
331 |             QTimer.singleShot(0, lambda: _mac_set_window_level(int(self.winId()), 3 if on else 0))
332 |         if on:
333 |             self.raise_()
334 |
335 |     def showEvent(self, event) -> None: # noqa: N802 (Qt 命名)
336 |         """窗口显示时校正层级（延迟执行，避免被 Qt 窗口重建覆盖）。"""
337 |         super().showEvent(event)
338 |         if sys.platform == 'darwin':
339 |             on = bool(self.cfg.get('on_top', True))
340 |             QTimer.singleShot(0, lambda: _mac_set_window_level(int(self.winId()), 3 if on else 0))
341 |
342 |     def set_no_move(self, on: bool) -> None:
343 |         """切换「不移动」：禁用自动移动；勾选瞬间若正在移动则立即停下回待机。"""
344 |         self.no_move = bool(on)
345 |         self.cfg.set('no_move', self.no_move)
346 |         self.cfg.save()
347 |         if self.no_move and self._move_plan is not None:
348 |             if self.idles:
349 |                 self._switch(self._pick(self.idles)) # 打断进行中的移动
350 |
351 | # ===== 播放
352 |     def _bind_movie(self, name: str, movie) -> None:
353 |         """只绑定当前播放器；LRU 重建同名动作时也会重新绑定。"""
354 |         if movie is self._bound_movie:
355 |             self._bound_movie_name = name
356 |             return
357 |         self._unbind_movie()
358 |         movie.frameChanged.connect(self._on_frame)
359 |         movie.finished.connect(self._on_movie_finished)
360 |         movie.errorOccurred.connect(self._on_movie_error)
361 |         self._bound_movie = movie
362 |         self._bound_movie_name = name
363 |
364 |     def _unbind_movie(self) -> None:
365 |         movie = self._bound_movie

```

```

366 |         if movie is not None:
367 |             for signal, slot in (
368 |                 (movie.frameChanged, self._on_frame),
369 |                 (movie.finished, self._on_movie_finished),
370 |                 (movie.errorOccurred, self._on_movie_error),
371 |             ):
372 |                 try:
373 |                     signal.disconnect(slot)
374 |                 except (RuntimeError, TypeError):
375 |                     pass
376 |             self._bound_movie = None
377 |             self._bound_movie_name = None
378 |
379 |     def _switch(self, name: str) -> None:
380 |         """切换到指定动画（链式模型：全部一次性播放）。"""
381 |         self._idle_pause_timer.stop()
382 |         self._cancel_move()
383 |         previous = self.movie
384 |         movie = self.lib.activate(name)
385 |         if previous is not None and previous is not movie:
386 |             previous.stop()
387 |         self._bind_movie(name, movie)
388 |         self.anim = name
389 |         if name in self.playlist:
390 |             self._playlist_index = self.playlist.index(name)
391 |         self.movie = movie
392 |         movie.stop()
393 |         movie.jumpToFrame(0)
394 |         if hasattr(movie, 'set_playback_speed'):
395 |             movie.set_playback_speed(self._effective_playback_speed(name))
396 |         self._ended_fired = False
397 |         self._mask_frame_counter = 0
398 |         self._rebuild_frame(force_mask=True)
399 |         if not self._suspended and not self._paused:
400 |             movie.start()
401 |
402 |     def _on_frame(self, n: int) -> None:
403 |         """媒体帧推进回调：重建画面：最后一帧触发播完处理。"""
404 |         name = self._bound_movie_name
405 |         if name != self.anim or self.movie is None or self.sender() is not self.movie:
406 |             return
407 |         self._rebuild_frame()
408 |         self.update()
409 |         if n >= self.lib.frames(name) - 1 and not self._ended_fired:
410 |             self._ended_fired = True
411 |             self.movie.stop() # 停在最后一帧，等 _on_anim_ended 切走
412 |             self._on_anim_ended(name)
413 |
414 |     def _on_movie_finished(self) -> None:
415 |         name = self._bound_movie_name
416 |         if (
417 |             name != self.anim
418 |             or self.movie is None
419 |             or self.sender() is not self.movie
420 |             or self._ended_fired
421 |         ):
422 |             return
423 |         self._ended_fired = True
424 |         self._on_anim_ended(name)
425 |
426 |     def _on_movie_error(self, message: str) -> None:
427 |         logging.error('动画解码失败 %s: %s', self._bound_movie_name, message)
428 |
429 |     def _rebuild_frame(self, *, force_mask: bool = False) -> None:
430 |         """按固定逻辑画布生成 Retina pixmap，并同步窗口 mask。
431 |
432 |         裁边图只作为解码优化：显示时还原到稳定画布，避免每帧透明边界
433 |         变化带来尺寸/位置抖动。
434 |         """
435 |         if self.movie is None:
436 |             return
437 |         frame = self.movie.currentFrame()
438 |         if frame is None or frame.image.isNull():
439 |             return
440 |         img = frame.canvas_image(frame)
441 |         if self.facing == 'right':
442 |             img = img.mirrored(True, False)
443 |         w_c = max(1, int(round(catalog.CANVAS_W * self.scale)))
444 |         h_c = max(1, int(round(catalog.CANVAS_H * self.scale)))

```

```

445 |     dpr = min(catalog.RENDER_DPR_CAP, max(1.0, float(self.devicePixelRatioF())))
446 |     pixel_w = max(1, int(round(w_c * dpr)))
447 |     pixel_h = max(1, int(round(h_c * dpr)))
448 |     img = img.scaled(pixel_w, pixel_h,
449 |                     Qt.AspectRatioMode.IgnoreAspectRatio,
450 |                     Qt.TransformationMode.SmoothTransformation)
451 |     pixmap = QPixmap.fromImage(img)
452 |     pixmap.setDevicePixelRatio(dpr)
453 |     self._frame_pixmap = pixmap
454 |     self._frame_logical_size = (w_c, h_c)
455 |     self._frame_logical_rect = (0, 0, w_c, h_c)
456 |     self._mask_frame_counter += 1
457 |     mask_interval = (
458 |         catalog.BUSY_MASK_FRAME_INTERVAL if self._resource_constrained else 1
459 |     )
460 |     if force_mask or self._mask_frame_counter >= mask_interval:
461 |         self._sync_mask()
462 |         self._mask_frame_counter = 0
463 |
464 | def _sync_mask(self) -> None:
465 |     """按当前帧 alpha 设置窗口 mask: 透明区域鼠标穿透到下层窗口。"""
466 |     canvas = QImage(self._w, self._h, QImage.Format.Format_ARGB32)
467 |     canvas.fill(Qt.GlobalColor.transparent)
468 |     p = QPainter(canvas)
469 |     p.translate(0, int(round(catalog.PAD * self.scale)))
470 |     if self._frame_pixmap is not None:
471 |         x, y, _, _ = self._frame_logical_rect
472 |         p.drawPixmap(x, y, self._frame_pixmap)
473 |     p.end()
474 |     self.setMask(QBitmap.fromImage(canvas.createAlphaMask()))
475 |
476 | def paintEvent(self, event) -> None: # noqa: N802 (Qt 命名)
477 |     painter = QPainter(self)
478 |     painter.setRenderHint(QPainter.RenderHint.SmoothPixmapTransform)
479 |     if self._frame_pixmap is not None:
480 |         if self._squash_active:
481 |             # Q 弹: 垂直变矮 + 水平微微变宽, 脚底保持不动
482 |             sy = 1.0 - 0.15 * math.sin(math.pi * self._squash_progress)
483 |             sx = 1.0 + 0.10 * math.sin(math.pi * self._squash_progress)
484 |             canvas_w, canvas_h = self._frame_logical_size
485 |             crop_x, crop_y, crop_w, crop_h = self._frame_logical_rect
486 |             scaled_canvas_w = canvas_w * sx
487 |             scaled_canvas_h = canvas_h * sy
488 |             base_x = (self._w - scaled_canvas_w) / 2.0
489 |             base_y = self._h - scaled_canvas_h
490 |             x = int(round(base_x + crop_x * sx))
491 |             y = int(round(base_y + crop_y * sy))
492 |             w = max(1, int(round(crop_w * sx)))
493 |             h = max(1, int(round(crop_h * sy)))
494 |             painter.drawPixmap(x, y, w, h, self._frame_pixmap)
495 |         else:
496 |             # 落地对齐: 整帧下移 PAD×scale, 让人物脚底踩在窗口底线
497 |             painter.translate(0, int(round(catalog.PAD * self.scale)))
498 |             x, y, _, _ = self._frame_logical_rect
499 |             painter.drawPixmap(x, y, self._frame_pixmap)
500 |     painter.end()
501 |
502 | def _start_squash(self) -> None:
503 |     """点击时启动 Q 弹效果: 画面先变矮再恢复。"""
504 |     self._squash_active = True
505 |     self._squash_progress = 0.0
506 |     self._squash_clock.start()
507 |     self._squash_timer.start()
508 |     self.update()
509 |
510 | def _on_squash_tick(self) -> None:
511 |     elapsed = self._squash_clock.elapsed()
512 |     self._squash_progress = min(1.0, elapsed / self._squash_duration_ms)
513 |     if self._squash_progress >= 1.0:
514 |         self._squash_active = False
515 |         self._squash_timer.stop()
516 |     self.update()
517 |
518 | def icon_pixmap(self, size: int = 64) -> QPixmap:
519 |     """托盘图标: 取当前帧 (无则待机首帧) 缩放。"""
520 |     if self._frame_pixmap is not None:
521 |         canvas_w, canvas_h = self._frame_logical_size
522 |         image = QImage(canvas_w, canvas_h, QImage.Format.Format_ARGB32)
523 |         image.fill(Qt.GlobalColor.transparent)

```

```

524 |         painter = QPainter(image)
525 |         x, y, _, _ = self._frame_logical_rect
526 |         painter.drawPixmap(x, y, self._frame_pixmap)
527 |         painter.end()
528 |     else:
529 |         frame = self.lib.movie(self.idle).currentFrame()
530 |         image = QImage(
531 |             frame.canvas_width,
532 |             frame.canvas_height,
533 |             QImage.Format.Format_ARGB32,
534 |         )
535 |         image.fill(Qt.GlobalColor.transparent)
536 |         painter = QPainter(image)
537 |         painter.drawImage(frame.x, frame.y, frame.image)
538 |         painter.end()
539 |     image = image.scaled(
540 |         size,
541 |         size,
542 |         Qt.AspectRatioMode.KeepAspectRatio,
543 |         Qt.TransformationMode.SMOOTHTransformation,
544 |     )
545 |     return QPixmap.fromImage(image)
546 |
547 | # ===== 动画链
548 | def _on_anim_ended(self, name: str) -> None:
549 |     if name == self.drag and self._dragging:
550 |         # 超长拖拽: 拖拽动画循环重播, 继续跟手
551 |         self.movie.jumpToFrame(0)
552 |         self._ended_fired = False
553 |         self.movie.start()
554 |         return
555 |     if name in self.turns:
556 |         # 转向动画播完 -> 翻转朝向
557 |         self.facing = 'right' if self.facing == 'left' else 'left'
558 |     if self.playlist_mode != 'off' and name in self.playlist:
559 |         self._play_next_playlist()
560 |         return
561 |     if name == self.drag or name in self.clicks:
562 |         # 交互打断的动画播完 -> 待机缓冲 (一次性), 待机播完再进链
563 |         if self.idles:
564 |             self._switch(self._pick(self.idles))
565 |             return
566 |     if self._resource_constrained and name in self.idles:
567 |         # 保留最后一帧短暂停顿, 不启动解码器: 交互会通过 _switch 立即取消等待。
568 |         self._idle_pause_timer.start(catalog.BUSY_IDLE_PAUSE_MS)
569 |         return
570 |     self._pick_next()
571 |
572 | def _pick_next(self) -> None:
573 |     """按播放列表或当前性格模式选择下一个动画。"""
574 |     if self.playlist_mode != 'off' and self.playlist:
575 |         self._play_next_playlist()
576 |         return
577 |     roll = random.random()
578 |     if self._resource_constrained:
579 |         if roll < catalog.BUSY_IDLE_PROBABILITY and self.idles:
580 |             self._switch(self._pick(self.idles, exclude=self.anim))
581 |         elif roll < catalog.BUSY_TURN_PROBABILITY and self.turns:
582 |             self._switch(self._pick(self.turns, exclude=self.anim))
583 |         else:
584 |             self._switch(self._pick(self.acts, exclude=self.anim))
585 |         return
586 |     profile = catalog.PERSONALITY_PRESETS[self.personality]
587 |     idle_edge = float(profile['idle'])
588 |     turn_edge = idle_edge + float(profile['turn'])
589 |     acts_edge = turn_edge + float(profile['acts'])
590 |     if self.no_move:
591 |         acts_edge += float(profile['move'])
592 |     if roll < idle_edge:
593 |         if self.idles:
594 |             self._switch(self._pick(self.idles, exclude=self.anim))
595 |         else:
596 |             self._switch(self._pick(self.acts, exclude=self.anim))
597 |     elif roll < turn_edge:
598 |         if self.turns:
599 |             self._switch(self._pick(self.turns, exclude=self.anim))
600 |         else:
601 |             self._switch(self._pick(self.acts, exclude=self.anim))
602 |     elif roll < acts_edge:

```

```

603 |         self._switch(self._pick(self.acts, exclude=self.anim))
604 |     else:
605 |         if self.no_move or not self._try_move():
606 |             self._switch(self._pick(self.acts, exclude=self.anim))
607 |
608 | @staticmethod
609 | def _pick(pool: list[str], exclude: str | None = None) -> str:
610 |     entries = [n for n in pool if n != exclude] or pool
611 |     return random.choice(entries)
612 |
613 | def _save_action_names(self, key: str, names: list[str]) -> None:
614 |     self.cfg.set(key, list(names))
615 |     self.cfg.save()
616 |
617 | def _edit_action_set(self, key: str, title: str) -> None:
618 |     current = list(self.favorites if key == 'favorites' else self.playlist)
619 |     dialog = ActionSetDialog(title, list(self.lib.names()), current, self)
620 |     if dialog.exec() != dialog.DialogCode.Accepted:
621 |         return
622 |     selected = [name for name in dialog.selected_names() if name in self.lib.names()]
623 |     if key == 'favorites':
624 |         self.favorites = selected
625 |     else:
626 |         self.playlist = selected
627 |         self._playlist_index = -1
628 |         if not self.playlist and self.playlist_mode != 'off':
629 |             self.playlist_mode = 'off'
630 |             self.cfg.set('playlist_mode', 'off')
631 |             self.playlistChanged.emit('off')
632 |         self._save_action_names(key, selected)
633 |         if key == 'playlist' and self.playlist_mode != 'off':
634 |             self._cancel_move()
635 |             self._play_next_playlist()
636 |
637 | def _toggle_current_favorite(self) -> None:
638 |     if self.anim in self.favorites:
639 |         self.favorites.remove(self.anim)
640 |     else:
641 |         self.favorites.append(self.anim)
642 |     self._save_action_names('favorites', self.favorites)
643 |
644 | def _set_playlist_from_favorites(self) -> None:
645 |     self.playlist = list(self.favorites)
646 |     self._playlist_index = -1
647 |     self._save_action_names('playlist', self.playlist)
648 |     if self.playlist_mode != 'off':
649 |         self._play_next_playlist()
650 |
651 | def _play_next_playlist(self) -> None:
652 |     if not self.playlist:
653 |         self.set_playlist_mode('off')
654 |         return
655 |     if self.playlist_mode == 'random':
656 |         name = self._pick(self.playlist, exclude=self.anim)
657 |     else:
658 |         self._playlist_index = (self._playlist_index + 1) % len(self.playlist)
659 |         name = self.playlist[self._playlist_index]
660 |     self._switch(name)
661 |
662 | def set_playlist_mode(self, mode: str) -> None:
663 |     if mode not in ('off', 'loop', 'random'):
664 |         return
665 |     if mode != 'off' and not self.playlist:
666 |         return
667 |     self.playlist_mode = mode
668 |     self.cfg.set('playlist_mode', mode)
669 |     self.cfg.save()
670 |     self.playlistChanged.emit(mode)
671 |     if mode != 'off':
672 |         self._cancel_move()
673 |         self._play_next_playlist()
674 |
675 | def add_action_menu(self, parent_menu: QMenu) -> QMenu:
676 |     """给桌宠右键菜单和状态栏菜单共用动作管理入口。"""
677 |     menu = parent_menu.addMenu('动作管理')
678 |     menu.aboutToShow.connect(lambda menu=menu: self._populate_action_menu(menu))
679 |     self._populate_action_menu(menu)
680 |     return menu
681 |

```

```

682 | def _populate_action_menu(self, menu: QMenu) -> None:
683 |     menu.clear()
684 |     current_label = (
685 |         '取消收藏当前动作' if self.anim in self.favorites else '收藏当前动作'
686 |     )
687 |     menu.addAction(current_label, self._toggle_current_favorite)
688 |     menu.addAction(
689 |         '编辑收藏夹...',
690 |         lambda: self._edit_action_set('favorites', '编辑收藏夹'),
691 |     )
692 |     menu.addAction(
693 |         '编辑播放列表...',
694 |         lambda: self._edit_action_set('playlist', '编辑播放列表'),
695 |     )
696 |
697 |     favorites = menu.addMenu('收藏夹')
698 |     if self.favorites:
699 |         for name in self.favorites:
700 |             favorites.addAction(name, lambda checked=False, name=name: self._switch(name))
701 |     else:
702 |         empty = favorites.addAction('暂无收藏')
703 |         empty.setEnabled(False)
704 |
705 |     playlist = menu.addMenu('播放列表')
706 |     if self.favorites:
707 |         playlist.addAction('用收藏夹覆盖播放列表', self._set_playlist_from_favorites)
708 |     if self.playlist:
709 |         playlist.addAction(f'当前列表: {len(self.playlist)} 段').setEnabled(False)
710 |     else:
711 |         empty = playlist.addAction('暂无动作, 请先编辑列表')
712 |         empty.setEnabled(False)
713 |     for mode, label in (
714 |         ('off', '关闭播放列表'),
715 |         ('loop', '循环播放'),
716 |         ('random', '随机播放'),
717 |     ):
718 |         action = playlist.addAction(label)
719 |         action.setCheckable(True)
720 |         action.setChecked(self.playlist_mode == mode)
721 |         action.setEnabled(mode == 'off' or bool(self.playlist))
722 |         action.triggered.connect(lambda checked=False, mode=mode: self._set_playlist_mode(mode))
723 |
724 | def set_personality(self, personality: str) -> None:
725 |     personality = self._normalize_personality(personality)
726 |     if personality == self.personality:
727 |         return
728 |     self.personality = personality
729 |     self.cfg.set('personality', personality)
730 |     self.cfg.save()
731 |     self._schedule_next_greeting()
732 |     self.personalityChanged.emit(personality)
733 |
734 | def add_personality_menu(self, parent_menu: QMenu) -> QMenu:
735 |     menu = parent_menu.addMenu('性格模式')
736 |     for key, profile in catalog.PERSONALITY_PRESETS.items():
737 |         action = menu.addAction(str(profile['label']))
738 |         action.setCheckable(True)
739 |         action.setChecked(self.personality == key)
740 |         action.triggered.connect(
741 |             lambda checked=False, key=key: self._set_personality(key)
742 |         )
743 |     return menu
744 |
745 | def _special_animation(self, preferred: str, fallback: list[str]) -> str | None:
746 |     """优先使用有明确语义的动作, 没有时回退到当前角色已有动作。"""
747 |     if preferred in self.lib.names():
748 |         return preferred
749 |     pool = [name for name in fallback if name in self.lib.names()]
750 |     return self._pick(pool, exclude=self.anim) if pool else None
751 |
752 | def _schedule_next_greeting(self) -> None:
753 |     self._greeting_timer.stop()
754 |     if (
755 |         not self.proactive_greetings
756 |         or self._suspended
757 |         or self._paused
758 |         or self._shutting_down
759 |     ):
760 |         return

```



```

761 |     profile = catalog.PERSONALITY_PRESETS[self.personality]
762 |     delay = random.randint(
763 |         int(profile['greeting_min_ms']),
764 |         int(profile['greeting_max_ms']),
765 |     )
766 |     self._greeting_timer.start(delay)
767 |
768 | def _on_greeting_timer(self) -> None:
769 |     if (
770 |         self.proactive_greetings
771 |         and not self._suspended
772 |         and not self._paused
773 |         and not self._shutting_down
774 |         and not self._dragging
775 |         and self._move_plan is None
776 |         and random.random() <= float(
777 |             catalog.PERSONALITY_PRESETS[self.personality]['greeting_chance']
778 |         )
779 |     ):
780 |         name = self._special_animation(
781 |             '点击回应-元气挥手', self.clicks or self.acts
782 |         )
783 |         if name:
784 |             self._switch(name)
785 |     self._schedule_next_greeting()
786 |
787 | def _on_cursor_tick(self) -> None:
788 |     """鼠标进入反应半径时镜像当前帧，让角色自然看向鼠标。"""
789 |     if self._suspended or self._paused or self._shutting_down or self._dragging:
790 |         return
791 |     center = (self.x() + self._w / 2.0, self.y() + self._h / 2.0)
792 |     cursor = QCursor.pos()
793 |     target = cursor_facing(
794 |         center,
795 |         (float(cursor.x()), float(cursor.y())),
796 |         catalog.CURSOR_REACTION_RADIUS,
797 |         catalog.CURSOR_DEAD_ZONE,
798 |     )
799 |     if target is None or target == self.facing:
800 |         return
801 |     self.facing = target
802 |     self._rebuild_frame(force_mask=True)
803 |     self.update()
804 |
805 | # ===== 移动
806 | def _try_move(self, name: str | None = None) -> bool:
807 |     """计划一次朝 facing 方向的移动；屏幕空间不够返回 False。
808 |
809 |     name 给定时使用指定动画（手动触发），否则随机选一个移动姿态。
810 |
811 |     if self._move_plan is not None:
812 |         return True # 已在移动/已计划
813 |     avail = self.screen().availableGeometry()
814 |     dir_sign = 1 if self.facing == 'right' else -1
815 |     cx = self.x() + self._w / 2
816 |     distance = random.randint(catalog.MOVE_MIN_PX, catalog.MOVE_MAX_PX)
817 |     target_cx = cx + dir_sign * distance
818 |     half_w = self._w / 2
819 |     left_bound = avail.left() + catalog.MOVE_MARGIN + half_w
820 |     right_bound = avail.right() - catalog.MOVE_MARGIN - half_w
821 |     if target_cx < left_bound or target_cx > right_bound:
822 |         return False
823 |     if not self.moves:
824 |         return False
825 |     move_name = name or self._pick(self.moves)
826 |     duration = self.lib.duration(move_name)
827 |     self._switch(move_name)
828 |     self._move_plan = {
829 |         'start_x': self.x(),
830 |         'target_x': int(round(target_cx - half_w)),
831 |         'y': self.y(),
832 |         'duration': duration,
833 |     }
834 |     self._move_timer.start()
835 |     return True
836 |
837 | def _trigger_move(self, name: str) -> None:
838 |     """手动触发移动（右键菜单）：先打断当前移动，再朝 facing 方向走动；
839 |     屏幕空间不足则原地播放走路姿态（不位移）。"""

```

```

840 |         self._cancel_move()
841 |     if not self._try_move(name):
842 |         self._switch(name) # 贴边放不下：原地播放走路姿态，不位移
843 |
844 | def _on_move_tick(self) -> None:
845 |     """位置驱动：跟随动画播放进度插值（前后各 2s 不动，中间走全程）。"""
846 |     plan = self._move_plan
847 |     if not plan or self.movie is None:
848 |         self._move_timer.stop()
849 |         return
850 |     t = self.movie.currentTimeSeconds()
851 |     lead, tail = catalog.MOVE_LEAD_SEC, catalog.MOVE_TAIL_SEC
852 |     dur = plan['duration']
853 |     if t <= lead:
854 |         x = plan['start_x']
855 |     elif t >= dur - tail:
856 |         x = plan['target_x']
857 |     else:
858 |         progress = (t - lead) / max(0.1, dur - lead - tail)
859 |         x = plan['start_x'] + (plan['target_x'] - plan['start_x']) * progress
860 |     self.move(int(round(x)), plan['y'])
861 |     if t >= dur - tail:
862 |         # 到位：提交终点，动画自然播完后续链
863 |         self._move_timer.stop()
864 |         self._move_plan = None
865 |         self._save_position()
866 |
867 | def _cancel_move(self) -> None:
868 |     self._move_timer.stop()
869 |     self._move_plan = None
870 |
871 | # ===== 交互
872 | def _is_in_interactive_area(self, local_pos) -> bool:
873 |     """由于动画左右有留白，只把窗口中间 1/3 宽度作为可交互区域。"""
874 |     return self._w / 3.0 <= local_pos.x() <= self._w * 2.0 / 3.0
875 |
876 | def mousePressEvent(self, event) -> None: # noqa: N802
877 |     if event.button() == Qt.MouseButton.LeftButton:
878 |         if not self._is_in_interactive_area(event.position().toPoint()):
879 |             return # 左右留白区域不参与点击/拖拽
880 |         self._press_global = event.globalPosition().toPoint()
881 |         self._grab_offset = self._press_global - self.pos()
882 |         self._dragging = False
883 |         self._long_press_fired = False
884 |         self._long_press_timer.start(catalog.LONG_PRESS_MS)
885 |         self._cancel_move() # 按下即打断移动
886 |         self._last_global = self._press_global
887 |         self._last_move_time = time.monotonic()
888 |         self._phys_vel = [0.0, 0.0]
889 |         self._phys_pos = [float(self.x()), float(self.y())]
890 |         self._stop_physics()
891 |         event.accept()
892 |     else:
893 |         super().mousePressEvent(event)
894 |
895 | def mouseMoveEvent(self, event) -> None: # noqa: N802
896 |     if self._press_global is None or not (event.buttons() & Qt.MouseButton.LeftButton):
897 |         return
898 |     g = event.globalPosition().toPoint()
899 |     delta = g - self._press_global
900 |     if not self._dragging:
901 |         if math.hypot(delta.x(), delta.y()) < catalog.DRAG_THRESHOLD * self.scale:
902 |             return # 未超阈值：仍是点击候选
903 |         self._dragging = True
904 |         self._long_press_timer.stop()
905 |         if self.drag:
906 |             self._switch(self.drag) # 进入拖拽：播放悬空反馈动画
907 |         if self.drag_physics:
908 |             self._phys_pos = [float(self.x()), float(self.y())]
909 |             self._drag_target = g - self._grab_offset
910 |             self._physics_mode = 'drag'
911 |             self._physics_timer.start()
912 |         else:
913 |             self.move(g - self._grab_offset)
914 |             self._last_global = g
915 |             self._last_move_time = time.monotonic()
916 |             event.accept()
917 |         return
918 |

```

```

919 | # 已经处于拖拽中
920 | if self.drag_physics:
921 |     now = time.monotonic()
922 |     dt = now - self._last_move_time
923 |     if dt > 0 and self._last_global is not None:
924 |         inst_vx = (g.x() - self._last_global.x()) / dt
925 |         inst_vy = (g.y() - self._last_global.y()) / dt
926 |         self._phys_vel[0] = self._phys_vel[0] * 0.6 + inst_vx * 0.4
927 |         self._phys_vel[1] = self._phys_vel[1] * 0.6 + inst_vy * 0.4
928 |         self._last_global = g
929 |         self._last_move_time = now
930 |         self._drag_target = g - self._grab_offset
931 |         if self._physics_mode != 'drag':
932 |             self._physics_mode = 'drag'
933 |             self._physics_timer.start()
934 |     else:
935 |         self.move(g - self._grab_offset) # 跟手（保持抓起时的偏移）
936 |     event.accept()
937 |
938 | def mouseReleaseEvent(self, event) -> None: # noqa: N802
939 |     if event.button() != Qt.MouseButton.LeftButton:
940 |         super().mouseReleaseEvent(event)
941 |         return
942 |     was_dragging = self._dragging
943 |     self._long_press_timer.stop()
944 |     g = event.globalPosition().toPoint()
945 |     dist = 0.0
946 |     if self._press_global is not None:
947 |         d = g - self._press_global
948 |         dist = math.hypot(d.x(), d.y())
949 |     if was_dragging:
950 |         self._just_dragged = True # 抑制拖拽结束后的幽灵点击
951 |         QTimer.singleShot(150, self._clear_just_dragged)
952 |         if self.drag_physics:
953 |             # 松手后进入抛掷物理：保留当前速度，重力 + 反弹 + 衰减
954 |             self._physics_mode = 'throw'
955 |             self._physics_timer.start()
956 |         else:
957 |             if self._grab_offset is not None:
958 |                 self.move(g - self._grab_offset) # 停在松手处
959 |             edge_bounced = self._trigger_edge_feedback()
960 |             if not edge_bounced and not self.drag_physics:
961 |                 self._save_position()
962 |             if self.idles and not edge_bounced:
963 |                 self._switch(self._pick(self.idles)) # 回待机缓冲
964 |     elif dist < catalog.DRAG_THRESHOLD * self.scale and not self._long_press_fired:
965 |         self._queue_tap()
966 |         self._dragging = False
967 |         self._press_global = None
968 |         self._grab_offset = None
969 |         self._long_press_fired = False
970 |         event.accept()
971 |
972 | def _clear_just_dragged(self) -> None:
973 |     self._just_dragged = False
974 |
975 | def _on_long_press(self) -> None:
976 |     if self._press_global is None or self._dragging or self._suspended:
977 |         return
978 |     self._long_press_fired = True
979 |     self._tap_timer.stop()
980 |     self._tap_count = 0
981 |     self._cancel_move()
982 |     self._start_squash()
983 |     name = self._special_animation('被吓一跳', self.acts or self.clicks)
984 |     if name:
985 |         self._switch(name)
986 |     self._duck_sound.play()
987 |
988 | def _queue_tap(self) -> None:
989 |     """延迟一个短窗口，区分单击、双击和快速连续点击。"""
990 |     self._tap_count += 1
991 |     self._tap_timer.start(catalog.TAP_BURST_WINDOW_MS)
992 |
993 | def _flush_tap_burst(self) -> None:
994 |     count = self._tap_count
995 |     self._tap_count = 0
996 |     action = classify_tap_burst(count)
997 |     if action == 'single':

```

```

998 |         self._on_click()
999 |     elif action == 'double':
1000 |         self._on_double_click()
1001 |     else:
1002 |         self._on_rapid_click(count)
1003 |
1004 | def _play_click_response(self, pool: list[str]) -> None:
1005 |     if not pool:
1006 |         return
1007 |     self._cancel_move()
1008 |     self._start_squash()
1009 |     self._switch(self._pick(pool, exclude=self.anim))
1010 |
1011 | def _on_click(self) -> None:
1012 |     """单击：播放一个普通点击响应。"""
1013 |     if self._just_dragged:
1014 |         return
1015 |     self._play_click_response(self.clicks)
1016 |
1017 | def _on_double_click(self) -> None:
1018 |     """双击：播放更强的响应，并触发尖叫鸭。"""
1019 |     self._play_click_response(self.clicks or self.acts)
1020 |     self._duck_sound.play()
1021 |
1022 | def _on_rapid_click(self, count: int) -> None:
1023 |     """快速连续点击：切换到随机动作，次数越多越像被连续戳醒。"""
1024 |     self._play_click_response(self.acts or self.clicks)
1025 |     self._duck_sound.play()
1026 |
1027 | def _trigger_edge_feedback(self) -> bool:
1028 |     """拖到屏幕边缘时做一次短暂压扁，并用物理反弹离开边缘。"""
1029 |     scr = self._screen_available()
1030 |     avail = scr.availableGeometry()
1031 |     contacts = edge_contacts(
1032 |         (float(self.x()), float(self.y()), float(self._w), float(self._h)),
1033 |         (
1034 |             float(avail.left()),
1035 |             float(avail.top()),
1036 |             float(avail.width()),
1037 |             float(avail.height()),
1038 |         ),
1039 |         catalog.EDGE_FEEDBACK_MARGIN,
1040 |     )
1041 |     if not contacts:
1042 |         return False
1043 |
1044 |     if 'left' in contacts:
1045 |         self.facing = 'right'
1046 |         self._phys_vel[0] = max(abs(self._phys_vel[0]), catalog.EDGE_BOUNCE_SPEED)
1047 |     elif 'right' in contacts:
1048 |         self.facing = 'left'
1049 |         self._phys_vel[0] = -max(abs(self._phys_vel[0]), catalog.EDGE_BOUNCE_SPEED)
1050 |     if 'top' in contacts:
1051 |         self._phys_vel[1] = max(abs(self._phys_vel[1]), catalog.EDGE_BOUNCE_SPEED)
1052 |     elif 'bottom' in contacts:
1053 |         self._phys_vel[1] = -max(abs(self._phys_vel[1]), catalog.EDGE_BOUNCE_SPEED)
1054 |
1055 |     self._phys_pos = [float(self.x()), float(self.y())]
1056 |     self._physics_mode = 'throw'
1057 |     self._physics_timer.start()
1058 |     self._start_squash()
1059 |     self._rebuild_frame(force_mask=True)
1060 |     name = self._special_animation('被吓一跳', self.acts or self.clicks)
1061 |     if name:
1062 |         self._switch(name)
1063 |     return True
1064 |
1065 | def contextMenuEvent(self, event) -> None: # noqa: N802
1066 |     if not self._is_in_interactive_area(event.pos()):
1067 |         return
1068 |     menu = QMenu(self)
1069 |
1070 |     if self.idles:
1071 |         m_idle = menu.addMenu('动画 · 待机')
1072 |         for n in self.idles:
1073 |             m_idle.addAction(n, lambda n=n: self._switch(n))
1074 |     if self.turns:
1075 |         m_turn = menu.addMenu('动画 · 转向')
1076 |         for n in self.turns:

```

```

1077 |         m_turn.addAction(n, lambda n=n: self._switch(n))
1078 |
1079 |     m_moves = menu.addMenu(' 动画 • 移动')
1080 |     for n in self.moves:
1081 |         m_moves.addAction(n, lambda n=n: self._trigger_move(n))
1082 |
1083 |     m_clicks = menu.addMenu(' 动画 • 点击回应')
1084 |     for n in self.clicks:
1085 |         m_clicks.addAction(n, lambda n=n: self._switch(n))
1086 |
1087 |     m_acts = menu.addMenu(' 动画 • 随机动作')
1088 |     for n in self.acts:
1089 |         m_acts.addAction(n, lambda n=n: self._switch(n))
1090 |
1091 |     self.add_action_menu(menu)
1092 |     self.add_personality_menu(menu)
1093 |
1094 |     m_speed = menu.addMenu(' 播放速率')
1095 |     for i in range(10, 21):
1096 |         v = i / 10.0
1097 |         act = m_speed.addAction(f' {v:.1f}x')
1098 |         act.setCheckable(True)
1099 |         act.setChecked(abs(self.playback_speed - v) < 0.01)
1100 |         act.triggered.connect(lambda checked=False, v=v: self.set_playback_speed(v))
1101 |
1102 |     drag_physics_act = menu.addAction(' 拖动物理')
1103 |     drag_physics_act.setCheckable(True)
1104 |     drag_physics_act.setChecked(self.drag_physics)
1105 |     drag_physics_act.toggled.connect(self.set_drag_physics)
1106 |
1107 |     interaction = menu.addMenu(' 互动反馈')
1108 |     duck_sound = interaction.addAction(' 尖叫鸭音效')
1109 |     duck_sound.setCheckable(True)
1110 |     duck_sound.setChecked(self.duck_sound_enabled)
1111 |     duck_sound.toggled.connect(self.set_duck_sound)
1112 |     greetings = interaction.addAction(' 偶尔主动打招呼')
1113 |     greetings.setCheckable(True)
1114 |     greetings.setChecked(self.proactive_greetings)
1115 |     greetings.toggled.connect(self.set_proactive_greetings)
1116 |
1117 |     m_display = menu.addMenu(' 画面调整')
1118 |     soft_edges_act = m_display.addAction(' 清理透明底噪 (Alpha=1)')
1119 |     soft_edges_act.setCheckable(True)
1120 |     soft_edges_act.setChecked(self.soft_edges)
1121 |     soft_edges_act.toggled.connect(self.set_soft_edges)
1122 |
1123 |     m_char = menu.addMenu(' 切换角色')
1124 |     current = str(self.cfg.get('character', catalog.DEFAULT_CHARACTER))
1125 |     for cid in catalog.list_available_characters():
1126 |         act = m_char.addAction(cid)
1127 |         act.setCheckable(True)
1128 |         act.setChecked(cid == current)
1129 |         act.triggered.connect(lambda checked=False, cid=cid: self._request_switch_character(cid))
1130 |
1131 |     menu.addSeparator()
1132 |     menu.addAction(' 回到右下角', self._go_default_corner)
1133 |
1134 |     on_top = menu.addAction(' 窗口置顶')
1135 |     on_top.setCheckable(True)
1136 |     on_top.setChecked(bool(self.cfg.get('on_top', True)))
1137 |     on_top.toggled.connect(self.set_on_top)
1138 |
1139 |     no_move = menu.addAction(' 不移动')
1140 |     no_move.setCheckable(True)
1141 |     no_move.setChecked(self.no_move)
1142 |     no_move.toggled.connect(self.set_no_move)
1143 |
1144 |     auto = menu.addAction(' 开机自启')
1145 |     auto.setCheckable(True)
1146 |     auto.setChecked(autostart_mod.is_enabled())
1147 |     auto.toggled.connect(autostart_mod.set_enabled)
1148 |
1149 |     m_scale = menu.addMenu(' 大小')
1150 |     for s in catalog.SCALE_STEPS:
1151 |         px = int(round(catalog.CANVAS_W * s))
1152 |         act = m_scale.addAction(f' {px}px')
1153 |         act.setCheckable(True)
1154 |         act.setChecked(abs(self.scale - s) < 0.02)
1155 |         act.triggered.connect(lambda checked=False, s=s: self.change_scale(s))

```

```

1156 |
1157 |         menu.addSeparator()
1158 |         menu.addAction('退出', self._request_quit)
1159 |         menu.exec(event.globalPos())
1160 |
1161 |     def _request_switch_character(self, character_id: str) -> None:
1162 |         """请求切换角色：优先交给 app 做热切换，否则只保存配置。"""
1163 |         if self.on_switch_character is not None:
1164 |             self.on_switch_character(character_id)
1165 |         else:
1166 |             self.cfg.set('character', character_id)
1167 |             self.cfg.save()
1168 |
1169 |     def set_playback_speed(self, speed: float) -> None:
1170 |         """设置动画播放速率并持久化。"""
1171 |         self.playback_speed = max(0.1, float(speed))
1172 |         self.cfg.set('playback_speed', self.playback_speed)
1173 |         self.cfg.save()
1174 |         if self.movie is not None and hasattr(self.movie, 'set_playback_speed'):
1175 |             self.movie.set_playback_speed(self._effective_playback_speed(self.anim))
1176 |
1177 |     def set_soft_edges(self, enabled: bool) -> None:
1178 |         """切换 Alpha=1 底噪清理并持久化；重建当前播放器以即时生效。"""
1179 |         enabled = bool(enabled)
1180 |         if enabled == self.soft_edges:
1181 |             return
1182 |         self.soft_edges = enabled
1183 |         self.cfg.set('soft_edges', enabled)
1184 |         self.cfg.save()
1185 |
1186 |         current = self.anim
1187 |         self._unbind_movie()
1188 |         if self.movie is not None:
1189 |             self.movie.stop()
1190 |             self.movie = None
1191 |         self.lib.set_soft_edges(enabled)
1192 |         if not self._suspended and not self._shutting_down:
1193 |             self._switch(current if current in self.lib.names() else self.idle)
1194 |         self.softEdgesChanged.emit(enabled)
1195 |
1196 |     def _effective_playback_speed(self, name: str) -> float:
1197 |         if self._resource_constrained and name in self.idles:
1198 |             return self.playback_speed * catalog.BUSY_IDLE_SPEED_FACTOR
1199 |         return self.playback_speed
1200 |
1201 |     def _check_system_load(self) -> None:
1202 |         ratio = system_load_ratio()
1203 |         if not self._load_governor.observe(ratio):
1204 |             return
1205 |         self._resource_constrained = self._load_governor.constrained
1206 |         logging.info(
1207 |             '资源模式=%s load_ratio=%.2f',
1208 |             '省资源' if self._resource_constrained else '正常',
1209 |             ratio,
1210 |         )
1211 |         interaction_active = self.anim in self.clicks or self.anim == self.drag
1212 |         if (
1213 |             self._resource_constrained
1214 |             and not self._dragging
1215 |             and not interaction_active
1216 |             and self.idles
1217 |         ):
1218 |             self._switch(self._pick(self.idles))
1219 |         elif self.movie is not None and hasattr(self.movie, 'set_playback_speed'):
1220 |             self.movie.set_playback_speed(self._effective_playback_speed(self.anim))
1221 |         if not self._resource_constrained and self._frame_pixmap is not None:
1222 |             self._sync_mask()
1223 |
1224 |     def _resume_busy_idle(self) -> None:
1225 |         if self._resource_constrained and self.idles and not self._dragging:
1226 |             self._switch(self._pick(self.idles, exclude=self.anim))
1227 |         else:
1228 |             self._pick_next()
1229 |
1230 |     def set_paused(self, on: bool) -> None:
1231 |         """暂停/继续可见动画，不隐藏窗口，也不释放素材库。"""
1232 |         on = bool(on)
1233 |         if on == self._paused or self._suspended or self._shutting_down:
1234 |             return

```

```

1235 |     self._paused = on
1236 |     if on:
1237 |         self._cancel_move()
1238 |         self._long_press_timer.stop()
1239 |         self._tap_timer.stop()
1240 |         self._cursor_timer.stop()
1241 |         self._greeting_timer.stop()
1242 |         self._resource_timer.stop()
1243 |         self._physics_timer.stop()
1244 |         if self.movie is not None:
1245 |             self.movie.stop()
1246 |     else:
1247 |         self._cursor_timer.start()
1248 |         self._schedule_next_greeting()
1249 |         self._resource_timer.start()
1250 |         if self.movie is not None:
1251 |             self.movie.start()
1252 |     else:
1253 |         self._switch(self.anim if self.anim in self.lib.names() else self.idle)
1254 |     self.pausedChanged.emit(self._paused)
1255 |
1256 | def toggle_pause(self) -> None:
1257 |     self.set_paused(not self._paused)
1258 |
1259 | def play_random_action(self) -> None:
1260 |     pool = self.acts or self.clicks or self.idles
1261 |     if pool:
1262 |         self._cancel_move()
1263 |         self._switch(self._pick(pool, exclude=self.anim))
1264 |
1265 | def trigger_duck_sound(self) -> None:
1266 |     self._duck_sound.play()
1267 |
1268 | def suspend_animation(self) -> None:
1269 |     """托盘隐藏时停止解码与高频计时器，保留当前画面和动作名。"""
1270 |     if self._suspended or self._shutting_down:
1271 |         return
1272 |     self._suspended = True
1273 |     self._long_press_timer.stop()
1274 |     self._tap_timer.stop()
1275 |     self._cursor_timer.stop()
1276 |     self._greeting_timer.stop()
1277 |     self._cancel_move()
1278 |     self._squash_timer.stop()
1279 |     self._physics_timer.stop()
1280 |     self._idle_pause_timer.stop()
1281 |     self._resource_timer.stop()
1282 |     self._unbind_movie()
1283 |     if self.movie is not None:
1284 |         self.movie.stop()
1285 |     self.movie = None
1286 |     self.lib.close()
1287 |
1288 | def resume_animation(self) -> None:
1289 |     """从托盘重新显示时恢复当前动作并重新启用负载采样。"""
1290 |     if not self._suspended or self._shutting_down:
1291 |         return
1292 |     self._suspended = False
1293 |     if not self._paused:
1294 |         self._cursor_timer.start()
1295 |         self._schedule_next_greeting()
1296 |         self._resource_timer.start()
1297 |         self._check_system_load()
1298 |     self._switch(self.anim if self.anim in self.lib.names() else self.idle)
1299 |
1300 | def set_mouse_through(self, on: bool) -> None:
1301 |     """鼠标穿透：开启后桌宠不接收鼠标事件，点击会穿透到下层。"""
1302 |     self.mouse_through = bool(on)
1303 |     self.cfg.set('mouse_through', self.mouse_through)
1304 |     self.cfg.save()
1305 |     self.setWindowFlag(Qt.WindowType.WindowTransparentForInput, self.mouse_through)
1306 |     self.show()
1307 |
1308 | def set_drag_physics(self, on: bool) -> None:
1309 |     """拖动物理开关。"""
1310 |     self.drag_physics = bool(on)
1311 |     self.cfg.set('drag_physics', self.drag_physics)
1312 |     self.cfg.save()
1313 |     if not self.drag_physics:

```

```

1314 |         self._stop_physics()
1315 |
1316 |     def set_duck_sound(self, on: bool) -> None:
1317 |         """尖叫鸭音效开关。"""
1318 |         self.duck_sound_enabled = bool(on)
1319 |         self._duck_sound.enabled = self.duck_sound_enabled
1320 |         self.cfg.set('duck_sound', self.duck_sound_enabled)
1321 |         self.cfg.save()
1322 |         if not self.duck_sound_enabled:
1323 |             self._duck_sound.close()
1324 |         self.duckSoundChanged.emit(self.duck_sound_enabled)
1325 |
1326 |     def set_proactive_greetings(self, on: bool) -> None:
1327 |         """偶尔主动挥手问候开关。"""
1328 |         self.proactive_greetings = bool(on)
1329 |         self.cfg.set('proactive_greetings', self.proactive_greetings)
1330 |         self.cfg.save()
1331 |         if self.proactive_greetings:
1332 |             self._schedule_next_greeting()
1333 |         else:
1334 |             self._greeting_timer.stop()
1335 |         self.proactiveGreetingsChanged.emit(self.proactive_greetings)
1336 |
1337 |     def _stop_physics(self) -> None:
1338 |         self._physics_timer.stop()
1339 |         self._physics_mode = None
1340 |
1341 |     def _on_physics_tick(self) -> None:
1342 |         if self._physics_mode == 'drag':
1343 |             self._tick_drag_physics()
1344 |         elif self._physics_mode == 'throw':
1345 |             self._tick_throw_physics()
1346 |
1347 |     def _tick_drag_physics(self) -> None:
1348 |         if self._drag_target is None:
1349 |             return
1350 |         dt = 0.016
1351 |         tx, ty = self._drag_target.x(), self._drag_target.y()
1352 |         px, py = self._phys_pos
1353 |         # 弹簧跟随 + 阻尼, 产生惯性/离心感
1354 |         ax = (tx - px) * 80.0 - self._phys_vel[0] * 10.0
1355 |         ay = (ty - py) * 80.0 - self._phys_vel[1] * 10.0
1356 |         self._phys_vel[0] += ax * dt
1357 |         self._phys_vel[1] += ay * dt
1358 |         self._phys_pos[0] += self._phys_vel[0] * dt
1359 |         self._phys_pos[1] += self._phys_vel[1] * dt
1360 |         self.move(int(round(self._phys_pos[0])), int(round(self._phys_pos[1])))
1361 |
1362 |     def _tick_throw_physics(self) -> None:
1363 |         dt = 0.016
1364 |         self._phys_vel[1] += 1400.0 * dt # 重力
1365 |         self._phys_pos[0] += self._phys_vel[0] * dt
1366 |         self._phys_pos[1] += self._phys_vel[1] * dt
1367 |         scr = self._screen_available()
1368 |         avail = scr.availableGeometry()
1369 |         # 忽略左右留白: 角色实际可视区域约为窗口中间 1/3,
1370 |         # 允许窗口略微超出屏幕边界, 让角色形象真正碰到边缘才反弹。
1371 |         margin = self._w / 3.0
1372 |         left = avail.left() - margin
1373 |         top = avail.top()
1374 |         right = avail.right() - self._w + margin
1375 |         bottom = avail.bottom() - self._h
1376 |         bounced = False
1377 |         if self._phys_pos[0] < left:
1378 |             self._phys_pos[0] = left
1379 |             self._phys_vel[0] = abs(self._phys_vel[0]) * 0.78
1380 |             bounced = True
1381 |         elif self._phys_pos[0] > right:
1382 |             self._phys_pos[0] = right
1383 |             self._phys_vel[0] = -abs(self._phys_vel[0]) * 0.78
1384 |             bounced = True
1385 |         if self._phys_pos[1] < top:
1386 |             self._phys_pos[1] = top
1387 |             self._phys_vel[1] = abs(self._phys_vel[1]) * 0.78
1388 |             bounced = True
1389 |         elif self._phys_pos[1] >= bottom:
1390 |             self._phys_pos[1] = bottom
1391 |             # 地面摩擦力: 水平速度逐渐衰减, 避免一直在地面滑/弹
1392 |             friction = 2.5 * dt

```



```

1393 |         self._phys_vel[0] *= max(0.0, 1.0 - friction)
1394 |         if abs(self._phys_vel[1]) < 40:
1395 |             self._phys_vel[1] = 0.0
1396 |         else:
1397 |             self._phys_vel[1] = -abs(self._phys_vel[1]) * 0.78
1398 |         bounced = True
1399 |         self.move(int(round(self._phys_pos[0])), int(round(self._phys_pos[1])))
1400 |         speed = math.hypot(self._phys_vel[0], self._phys_vel[1])
1401 |         # 在地面上且水平速度也很低时, 彻底停下
1402 |         if self._phys_pos[1] >= bottom - 1 and abs(self._phys_vel[1]) < 1 and abs(self._phys_vel[0]) < 15:
1403 |             self._stop_physics()
1404 |             self._save_position()
1405 |         elif bounced and speed < 40 and abs(self._phys_vel[1]) < 1:
1406 |             self._stop_physics()
1407 |             self._save_position()
1408 |
1409 |     def _request_quit(self) -> None:
1410 |         self._save_position()
1411 |         QApplication.instance().quit()
1412 |
1413 |     def shutdown(self) -> None:
1414 |         """停止窗口计时器和媒体 reader, 可安全重复调用。"""
1415 |         if self._shutting_down:
1416 |             return
1417 |         self._shutting_down = True
1418 |         for timer in (
1419 |             self._move_timer,
1420 |             self._squash_timer,
1421 |             self._physics_timer,
1422 |             self._resource_timer,
1423 |             self._idle_pause_timer,
1424 |             self._long_press_timer,
1425 |             self._tap_timer,
1426 |             self._cursor_timer,
1427 |             self._greeting_timer,
1428 |         ):
1429 |             timer.stop()
1430 |         self._unbind_movie()
1431 |         if self.movie is not None:
1432 |             self.movie.stop()
1433 |             self.movie = None
1434 |         self.lib.close()
1435 |         self._duck_sound.close()
1436 |
1437 |     def closeEvent(self, event) -> None: # noqa: N802
1438 |         self._save_position()
1439 |         self.shutdown()
1440 |         super().closeEvent(event)

```

17. run-mac.command (9 lines)

```

1 | #!/bin/zsh
2 | set -e
3 |
4 | ROOT="$(cd -- "$(dirname -- "$0")" && pwd)"
5 | export QT_PLUGIN_PATH="$ROOT/qt-plugins"
6 | export QT_QPA_PLATFORM=cocoa
7 |
8 | cd "$ROOT"
9 | exec "$ROOT/.venv/bin/python" -m pet "$@"

```

18. scripts/convert_to_gif.py (71 lines)

```

1 | # -*- coding: utf-8 -*-
2 | """将 assets/characters 下的 webm 角色动画转换为 assets/characters_gif 下的 GIF。
3 |
4 | 保持相同的子目录结构:
5 |     assets/characters/<角色ID>/videos/**/*.*.webm
6 |     -> assets/characters_gif/<角色ID>/videos/**/*.*.gif
7 | """
8 | from __future__ import annotations
9 |
10 | import shutil

```

```

11 | import subprocess
12 | import sys
13 | from pathlib import Path
14 |
15 | ROOT = Path(__file__).resolve().parent.parent
16 | SRC = ROOT / 'assets' / 'characters'
17 | DST = ROOT / 'assets' / 'characters_gif'
18 |
19 | try:
20 |     import imageio_ffmpeg
21 |     FFMPEG = imageio_ffmpeg.get_ffmpeg_exe()
22 | except Exception as exc:
23 |     print(f'无法获取 ffmpeg: {exc}')
24 |     sys.exit(1)
25 |
26 |
27 | def convert_one(src: Path, dst: Path) -> bool:
28 |     if dst.exists() and dst.stat().st_size > 0:
29 |         return True # 已存在则跳过
30 |     dst.parent.mkdir(parents=True, exist_ok=True)
31 |     # 关键: 解码端必须指定 libvpx-vp9, 否则会丢失 alpha
32 |     cmd = [
33 |         FFMPEG, '-y', '-hide_banner', '-loglevel', 'error',
34 |         '-c:v', 'libvpx-vp9', '-i', str(src),
35 |         '-vf', 'fps=24,split[s0][s1];'
36 |             '[s0]palettegen=reserve_transparent=1:stats_mode=diff[p];'
37 |             '[s1][p]paletteuse=alpha_threshold=128:'
38 |             'dither=sierra2_4a:diff_mode=rectangle',
39 |         '-gifflags', '+transdiff', '-loop', '0', str(dst),
40 |     ]
41 |     r = subprocess.run(cmd, capture_output=True)
42 |     if r.returncode != 0:
43 |         print(f'失败: {src.name}: {r.stderr.decode("utf-8", "replace")[:200]}', flush=True)
44 |         return False
45 |     return True
46 |
47 |
48 | def main() -> int:
49 |     if not SRC.is_dir():
50 |         print(f'未找到 {SRC}')
51 |         return 1
52 |     webm_files = sorted(SRC.rglob('*.*webm'))
53 |     if not webm_files:
54 |         print('没有 webm 文件')
55 |         return 1
56 |     ok = 0
57 |     fail = 0
58 |     for src in webm_files:
59 |         rel = src.relative_to(SRC)
60 |         dst = DST / rel.with_suffix('.gif')
61 |         if convert_one(src, dst):
62 |             ok += 1
63 |         else:
64 |             fail += 1
65 |         print(f'[{ok+fail}/{len(webm_files)}] {rel}', flush=True)
66 |     print(f'完成: 成功 {ok}, 失败 {fail}')
67 |     return 0 if fail == 0 else 1
68 |
69 |
70 | if __name__ == '__main__':
71 |     sys.exit(main())

```

19. tests/diagnose.py (76 lines)

```

1 | # -*- coding: utf-8 -*-
2 | """诊断: 验证高清晰素材下窗口渲染的每个环节。"""
3 |
4 | from __future__ import annotations
5 |
6 | import os
7 | import sys
8 |
9 | os.environ.setdefault('QT_QPA_PLATFORM', 'offscreen')
10 |
11 | ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
12 | sys.path.insert(0, ROOT)

```

```

13 |
14 | from PySide6.QtWidgets import QApplication # noqa: E402
15 |
16 | from pet import catalog # noqa: E402
17 | from pet.config import Config # noqa: E402
18 | from pet.library import MovieLibrary # noqa: E402
19 | from pet.window import PetWindow # noqa: E402
20 |
21 | app = QApplication([])
22 | lib = MovieLibrary()
23 | tmp = os.path.join(ROOT, 'tests', '_tmp_diag_fresh')
24 | cfg = Config(base=tmp)
25 | win = PetWindow(lib, cfg)
26 | win.show()
27 | app.processEvents()
28 |
29 | print('scale:', win.scale, '(DEFAULT_SCALE=', catalog.DEFAULT_SCALE, ')')
30 | print('窗口 size:', win.width(), 'x', win.height())
31 | print('窗口 isVisible:', win.isVisible())
32 | print('窗口 pos:', win.x(), win.y())
33 | region = win.mask()
34 | br = region.boundingRect()
35 | print('mask boundingRect:', br.x(), br.y(), br.width(), 'x', br.height())
36 |
37 | # 当前帧
38 | pm = win._frame_pixmap
39 | print('当前帧 pixmap:', None if pm is None else f'{pm.width()}x{pm.height()}')
40 |
41 | if pm is not None:
42 |     img = pm.toImage()
43 |     opaque = 0
44 |     for y in range(0, img.height(), 5):
45 |         for x in range(0, img.width(), 5):
46 |             if img.pixelColor(x, y).alpha() > 0:
47 |                 opaque += 1
48 |     print('当前帧非透明采样点:', opaque)
49 |
50 | # mask 非空像素
51 | mask_img = win.mask().toImage() if hasattr(win.mask(), 'toImage') else None
52 | if mask_img is None:
53 |     # QRegion 无 toImage, 从窗口 grab 判断
54 |     grab = win.grab()
55 |     gimg = grab.toImage()
56 |     mask_opaque = 0
57 |     for y in range(0, gimg.height(), 4):
58 |         for x in range(0, gimg.width(), 4):
59 |             if gimg.pixelColor(x, y).alpha() > 0:
60 |                 mask_opaque += 1
61 |     print('窗口 grab 非透明采样点:', mask_opaque, '(grab 尺寸', grab.width(), 'x', grab.height(), ')')
62 | else:
63 |     mask_opaque = 0
64 |     for y in range(0, mask_img.height(), 4):
65 |         for x in range(0, mask_img.width(), 4):
66 |             if mask_img.pixelColor(x, y).alpha() > 0:
67 |                 mask_opaque += 1
68 |     print('mask 非透明采样点:', mask_opaque)
69 |
70 | # 当前 QMovie 状态
71 | mv = win.movie
72 | print('当前动画:', win.anim)
73 | print('QMovie 帧数:', mv.frameCount() if mv else None)
74 | print('QMovie 当前帧号:', mv.currentFrameNumber() if mv else None)
75 | cp = mv.currentPixmap() if mv else None
76 | print('QMovie currentPixmap:', None if cp is None or cp.isNull() else f'{cp.width()}x{cp.height()}')

```

20. tests/grab_screen.py (37 lines)

```

1 | # -*- coding: utf-8 -*-
2 | """Qt QScreen.grabWindow 抓屏: 验证真实屏幕上透明桌宠是否可见。"""
3 |
4 | from __future__ import annotations
5 |
6 | import os
7 | import sys
8 |
9 | ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))

```

```

10 | sys.path.insert(0, ROOT)
11 |
12 | from PySide6.QtWidgets import QApplication # noqa: E402
13 |
14 | app = QApplication([])
15 | screen = app.primaryScreen()
16 | # 抓右下角区域 (桌宠预期位置屏幕 x1050-1511, y534-815)
17 | x, y, w, h = 900, 470, 636, 394
18 | pix = screen.grabWindow(0, x, y, w, h)
19 | out = os.path.join(ROOT, 'grab_qt.png')
20 | pix.save(out)
21 | print('saved', out, pix.width(), 'x', pix.height())
22 |
23 | # 分析非桌面背景/肤色像素
24 | img = pix.toImage()
25 | skin = 0
26 | colorful = 0
27 | for yy in range(0, img.height(), 2):
28 |     for xx in range(0, img.width(), 2):
29 |         c = img.pixelColor(xx, yy)
30 |         if c.alpha() == 0:
31 |             continue
32 |         r, g, b = c.red(), c.green(), c.blue()
33 |         if r > 150 and g > 110 and b > 110 and r > g > b:
34 |             skin += 1
35 |         if max(r, g, b) - min(r, g, b) > 30:
36 |             colorful += 1
37 | print('肤色采样:', skin, '彩色采样:', colorful)

```

21. tests/pos_diag.py (40 lines)

```

1 | # -*- coding: utf-8 -*-
2 | """真实 Windows 下诊断 screen/availableGeometry 与窗口定位。"""
3 |
4 | from __future__ import annotations
5 |
6 | import os
7 | import sys
8 |
9 | ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
10 | sys.path.insert(0, ROOT)
11 |
12 | from PySide6.QtGui import QGuiApplication # noqa: E402
13 | from PySide6.QtWidgets import QApplication # noqa: E402
14 |
15 | from pet import catalog # noqa: E402
16 | from pet.config import Config # noqa: E402
17 | from pet.library import MovieLibrary # noqa: E402
18 | from pet.window import PetWindow # noqa: E402
19 |
20 | app = QApplication([])
21 | screens = QGuiApplication.screens()
22 | for i, s in enumerate(screens):
23 |     print(f'screen[{i}]: geometry={s.geometry()} available={s.availableGeometry()} '
24 |           f'name={s.name()}')
25 |
26 | lib = MovieLibrary()
27 | cfg = Config(base=os.path.join(ROOT, 'tests', '_tmp_pos_fresh'))
28 | win = PetWindow(lib, cfg)
29 |
30 | print('窗口 size:', win.width(), 'x', win.height())
31 | print('win.screen() availableGeometry:', win.screen().availableGeometry())
32 | print('move 前 pos:', win.x(), win.y())
33 |
34 | avail = win.screen().availableGeometry()
35 | x = avail.right() - win.width() - catalog.CORNER_MARGIN
36 | y = avail.bottom() - win.height()
37 | print(f'期望右下角: x={x} y={y}')
38 | win.show()
39 | app.processEvents()
40 | print('show 后 pos:', win.x(), win.y())

```

22. tests/smoke.py (138 lines)

```

1 | # -*- coding: utf-8 -*-
2 | """
3 | Smoke test for the webm-backed media layer + window behavior (真实 webm 素材)。
4 |
5 | Run: python tests/smoke.py
6 | """
7 |
8 | from __future__ import annotations
9 |
10 | import os
11 | import sys
12 | import time
13 |
14 | os.environ.setdefault("QT_QPA_PLATFORM", "offscreen")
15 |
16 | ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
17 | if ROOT not in sys.path:
18 |     sys.path.insert(0, ROOT)
19 | os.environ.setdefault("QT_PLUGIN_PATH", os.path.join(ROOT, "qt-plugins"))
20 |
21 | from PySide6.QtWidgets import QApplication # noqa: E402
22 |
23 | from pet import catalog # noqa: E402
24 | from pet.config import Config # noqa: E402
25 | from pet.library import MovieLibrary # noqa: E402
26 | from pet.window import PetWindow # noqa: E402
27 |
28 |
29 | def _wait_until(app: QApplication, predicate, timeout: float = 5.0) -> None:
30 |     deadline = time.monotonic() + timeout
31 |     while time.monotonic() < deadline:
32 |         app.processEvents()
33 |         if predicate():
34 |             return
35 |         time.sleep(0.01)
36 |     raise AssertionError("timeout waiting for condition")
37 |
38 |
39 | def main() -> int:
40 |     app = QApplication([])
41 |     lib = MovieLibrary() # 真实 webm: assets/videos
42 |
43 |     # 1. 91 段素材全量可加载, 帧数/时长有效
44 |     names = lib.names()
45 |     assert len(names) == 91, len(names)
46 |     # 启动只预热移动动画; 其余播放器按需创建, 避免 91 个 QTimer/reader 同时常驻。
47 |     assert lib.loaded_count() <= len(catalog.MOVES)
48 |     for name in names:
49 |         assert lib.frames(name) >= 1, (name, lib.frames(name))
50 |         assert lib.duration(name) > 0, (name, lib.duration(name))
51 |
52 |     # 2. 透明通道: 待机首帧同时含透明与不透明像素
53 |     idle = lib.movie(catalog.IDLE)
54 |     idle.jumpToFrame(0)
55 |     assert idle.sourceSize() == (1280, 720), idle.sourceSize()
56 |     decoded_w, decoded_h = idle.decodedSize()
57 |     limit_w, limit_h = catalog.decode_size_for_scale(catalog.DEFAULT_SCALE)
58 |     assert 0 < decoded_w <= limit_w and 0 < decoded_h <= limit_h
59 |     assert limit_w - decoded_w <= 2 and limit_h - decoded_h <= 2
60 |     img = idle.currentPixmap().toImage()
61 |     alphas = set()
62 |     for x in range(0, img.width(), 20):
63 |         for y in range(0, img.height(), 10):
64 |             alphas.add(img.pixelColor(x, y).alpha())
65 |     assert len(alphas) >= 2, sorted(alphas)
66 |
67 |     # 3. 播放推进
68 |     idle.start()
69 |     _wait_until(app, lambda: idle.currentFrameNumber() >= 1)
70 |     assert idle.currentFrameNumber() >= 1
71 |     idle.stop()
72 |
73 |     # 4. 窗口实例化: 尺寸/初始动画/透明 mask
74 |     cfg = Config(base=os.path.join(os.path.dirname(os.path.abspath(__file__)), "_tmp_cfg"))
75 |     win = PetWindow(lib, cfg)
76 |     win._resource_timer.stop() # 冒烟测试固定为普通模式: 迟滞策略由纯逻辑测试覆盖
77 |     win._resource_constrained = False
78 |     win.show()
79 |     assert win.anim == catalog.IDLE

```

```

80 |     assert win.mask() is not None and not win.mask().isNull()
81 |     assert win.width() == int(round(catalog.CANVAS_W * win.scale))
82 |     assert win.height() == int(round((catalog.CANVAS_H + catalog.PAD) * win.scale))
83 |
84 |     # 5. 缩放: 底边不动
85 |     bottom = win.geometry().bottom()
86 |     win.change_scale(1.25)
87 |     assert win.geometry().bottom() == bottom
88 |     assert win.width() == int(round(catalog.CANVAS_W * 1.25))
89 |     win.change_scale(1.0)
90 |
91 |     # 6. 点击回应: 仅待机时可点; 播完回待机缓冲
92 |     win._on_click()
93 |     assert win.anim in catalog.CLICKS, win.anim
94 |     win._on_anim_ended(win.anim)
95 |     assert win.anim == catalog.IDLE
96 |
97 |     # 7. 转向: 东张西望播完翻转朝向
98 |     facing_before = win.facing
99 |     win._switch(catalog.TURN)
100 |     win._on_anim_ended(catalog.TURN)
101 |     assert win.facing != facing_before
102 |
103 |     # 8. 移动: 空间足够则生成移动计划并推进插值
104 |     win._cancel_move()
105 |     ok = win._try_move()
106 |     assert isinstance(ok, bool)
107 |     if ok:
108 |         assert win._move_plan is not None
109 |         x0 = win.x()
110 |         win._on_move_tick()
111 |         assert win.x() in (x0, win._move_plan["target_x"]) # 前后 2s 内位置不动或已到位
112 |         win._cancel_move()
113 |
114 |     # 9. 「不移动」: 状态机不再进入移动动画; 手动移动仍可走动; 开关持久化
115 |     win.set_no_move(True)
116 |     assert win.no_move is True and cfg.get("no_move") is True
117 |     selected = []
118 |     real_switch = win._switch
119 |     win._switch = selected.append
120 |     for _ in range(200):
121 |         win._pick_next()
122 |     assert selected and all(name not in catalog.MOVES for name in selected)
123 |     win._switch = real_switch
124 |     win._cancel_move()
125 |     win._trigger_move(catalog.MOVES[0])
126 |     assert win.anim in catalog.MOVES, win.anim
127 |     win._cancel_move()
128 |     win.set_no_move(False)
129 |     assert win.no_move is False and cfg.get("no_move") is False
130 |
131 |     win.shutdown()
132 |     win.close()
133 |     print("\n=== ALL SMOKE TESTS PASSED ===")
134 |     return 0
135 |
136 |
137 | if __name__ == "__main__":
138 |     raise SystemExit(main())

```

23. tests/test_media_runtime.py (252 lines)

```

1 | # -*- coding: utf-8 -*-
2 | """媒体热路径的行为回归测试。"""
3 |
4 | from __future__ import annotations
5 |
6 | import os
7 | import sys
8 | import tempfile
9 | import unittest
10 | from pathlib import Path
11 |
12 | os.environ.setdefault("QT_QPA_PLATFORM", "offscreen")
13 | ROOT = Path(__file__).resolve().parent.parent
14 | if str(ROOT) not in sys.path:

```

```

15 |     sys.path.insert(0, str(ROOT))
16 |
17 | from PySide6.QtCore import QObject, Signal # noqa: E402
18 | from PySide6.QtGui import QColor, QImage # noqa: E402
19 | from PySide6.QtWidgets import QApplication # noqa: E402
20 |
21 | from pet.config import Config # noqa: E402
22 | from pet.library import MovieLibrary # noqa: E402
23 | from pet.window import PetWindow # noqa: E402
24 | from pet.webm_clip import trim_transparent_frame # noqa: E402
25 |
26 |
27 | def _app() -> QApplication:
28 |     return QApplication.instance() or QApplication([])
29 |
30 |
31 | class _FrameOnlyClip(QObject):
32 |     frameChanged = Signal(int)
33 |     finished = Signal()
34 |     errorOccurred = Signal(str)
35 |
36 |     def __init__(self, frame) -> None:
37 |         super().__init__()
38 |         self._frame = frame
39 |
40 |     def currentFrame(self):
41 |         return self._frame
42 |
43 |     def currentPixmap(self):
44 |         raise AssertionError("运行时不应再走 QPixmap -> QImage 回转")
45 |
46 |     def start(self) -> None:
47 |         return
48 |
49 |     def stop(self) -> None:
50 |         return
51 |
52 |     def jumpToFrame(self, frame_index: int) -> bool:
53 |         return frame_index == 0
54 |
55 |     def frameCount(self) -> int:
56 |         return 100
57 |
58 |     def currentTimeSeconds(self) -> float:
59 |         return 0.0
60 |
61 |     def set_playback_speed(self, speed: float) -> None:
62 |         return
63 |
64 |
65 | class _FrameOnlyLibrary(QObject):
66 |     manifest = None
67 |     folder_map = {"idle": "idle"}
68 |     folder_files = {"idle": ["idle"]}
69 |
70 |     def __init__(self, clip: _FrameOnlyClip) -> None:
71 |         super().__init__()
72 |         self.clip = clip
73 |         self.soft_edges = True
74 |
75 |     def names(self):
76 |         return ["idle"]
77 |
78 |     def movie(self, name: str):
79 |         if name != "idle":
80 |             raise KeyError(name)
81 |         return self.clip
82 |
83 |     activate = movie
84 |
85 |     def frames(self, name: str) -> int:
86 |         return self.clip.frameCount()
87 |
88 |     def set_decode_size(self, width: int, height: int) -> None:
89 |         return
90 |
91 |     def set_soft_edges(self, enabled: bool) -> None:
92 |         self.soft_edges = bool(enabled)
93 |

```

```

94 |     def stop_all(self) -> None:
95 |         self.clip.stop()
96 |
97 |     def close(self) -> None:
98 |         self.clip.stop()
99 |
100 |
101 | class _RecreatingLibrary(_FrameOnlyLibrary):
102 |     def __init__(self, frame) -> None:
103 |         self._frame = frame
104 |         self._clip = None
105 |         super().__init__(self._new_clip())
106 |
107 |     def _new_clip(self) -> _FrameOnlyClip:
108 |         self._clip = _FrameOnlyClip(self._frame)
109 |         return self._clip
110 |
111 |     def movie(self, name: str):
112 |         if name != "idle":
113 |             raise KeyError(name)
114 |         return self._clip or self._new_clip()
115 |
116 |     def activate(self, name: str):
117 |         return self.movie(name)
118 |
119 |     def close(self) -> None:
120 |         if self._clip is not None:
121 |             self._clip.stop()
122 |             self._clip = None
123 |
124 |
125 | class MediaRuntimeTests(unittest.TestCase):
126 |     @classmethod
127 |     def setUpClass(cls) -> None:
128 |         cls.app = _app()
129 |
130 |     def test_transparent_crop_preserves_canvas_coordinates(self) -> None:
131 |         image = QImage(100, 80, QImage.Format.Format_RGBA8888)
132 |         image.fill(QColor(0, 0, 0, 0))
133 |         for y in range(20, 61):
134 |             for x in range(10, 41):
135 |                 image.setPixelColor(x, y, QColor(255, 255, 255, 255))
136 |
137 |         frame = trim_transparent_frame(image)
138 |
139 |         self.assertEqual(frame.canvas_size, (100, 80))
140 |         self.assertEqual(frame.offset, (2, 12))
141 |         self.assertEqual((frame.image.width(), frame.image.height()), (47, 57))
142 |
143 |     def test_soft_edges_clear_only_alpha_one_floor(self) -> None:
144 |         image = QImage(12, 12, QImage.Format.Format_RGBA8888)
145 |         image.fill(QColor(255, 0, 255, 0))
146 |         for y in range(3, 9):
147 |             for x in range(3, 9):
148 |                 image.setPixelColor(x, y, QColor(255, 255, 255, 255))
149 |         image.setPixelColor(3, 5, QColor(255, 255, 255, 1))
150 |         image.setPixelColor(4, 5, QColor(255, 255, 255, 2))
151 |         image.setPixelColor(0, 0, QColor(255, 255, 255, 1))
152 |
153 |         soft = trim_transparent_frame(image, padding=2, soften_edges=True)
154 |         floor_x, floor_y = 3 - soft.x, 5 - soft.y
155 |         low_alpha_x, low_alpha_y = 4 - soft.x, 5 - soft.y
156 |         center_x, center_y = 5 - soft.x, 5 - soft.y
157 |
158 |         self.assertEqual(
159 |             soft.image.format(),
160 |             QImage.Format.Format_ARGB32_Premultiplied,
161 |         )
162 |         self.assertEqual(soft.offset, (1, 1))
163 |         self.assertEqual(soft.image.pixelColor(floor_x, floor_y).alpha(), 0)
164 |         self.assertEqual(soft.image.pixelColor(low_alpha_x, low_alpha_y).alpha(), 2)
165 |         self.assertEqual(soft.image.pixelColor(center_x, center_y).alpha(), 240)
166 |
167 |     def test_window_renders_directly_from_cropped_qimage(self) -> None:
168 |         image = QImage(100, 80, QImage.Format.Format_RGBA8888)
169 |         image.fill(QColor(0, 0, 0, 0))
170 |         for y in range(20, 61):
171 |             for x in range(10, 41):
172 |                 image.setPixelColor(x, y, QColor(255, 255, 255, 255))

```



```

173 |         frame = trim_transparent_frame(image)
174 |         lib = _FrameOnlyLibrary(_FrameOnlyClip(frame))
175 |
176 |         with tempfile.TemporaryDirectory() as tmp:
177 |             window = PetWindow(lib, Config(base=tmp))
178 |             window._resource_timer.stop()
179 |             self.assertIsNotNone(window._frame_pixmap)
180 |             self.assertEqual(window._frame_logical_rect[:2], (0, 0))
181 |             self.assertEqual(window._frame_logical_rect[2:], window._frame_logical_size)
182 |             window.shutdown()
183 |             window.close()
184 |
185 |     def test_movie_library_lru_keeps_active_clip(self) -> None:
186 |         with tempfile.TemporaryDirectory() as tmp:
187 |             root = Path(tmp)
188 |             one_pixel_gif = (
189 |                 b"GIF89a\x01\x00\x01\x00\x80\x00\x00\x00\x00\xff\xff\xff"
190 |                 b"! \xf9\x04\x01\x00\x00\x00\x00,\x00\x00\x00\x00\x01\x00\x01\x00"
191 |                 b"\x00\x02\x02D\x01\x00;"
192 |             )
193 |             for name in ("a", "b", "c"):
194 |                 (root / f"{name}.gif").write_bytes(one_pixel_gif)
195 |
196 |             library = MovieLibrary(asset_dir=root, cache_limit=2)
197 |             active = library.activate("a")
198 |             library.movie("b")
199 |             library.movie("c")
200 |
201 |             self.assertIs(library.movie("a"), active)
202 |             self.assertEqual(set(library.movies()), {"a", "c"})
203 |             self.assertEqual(library.loaded_count(), 2)
204 |             library.close()
205 |
206 |     def test_hidden_window_rebinds_recreated_clip(self) -> None:
207 |         red = QImage(20, 20, QImage.Format.Format_RGBA8888)
208 |         red.fill(QColor(255, 0, 0, 255))
209 |         library = _RecreatingLibrary(trim_transparent_frame(red))
210 |
211 |         with tempfile.TemporaryDirectory() as tmp:
212 |             window = PetWindow(library, Config(base=tmp))
213 |             window._resource_timer.stop()
214 |             first = window.movie
215 |
216 |             window.suspend_animation()
217 |             window.resume_animation()
218 |             second = window.movie
219 |             self.assertNotEqual(first, second)
220 |
221 |             green = QImage(20, 20, QImage.Format.Format_RGBA8888)
222 |             green.fill(QColor(0, 255, 0, 255))
223 |             second._frame = trim_transparent_frame(green)
224 |             second.frameChanged.emit(1)
225 |             self.app.processEvents()
226 |
227 |             color = window._frame_pixmap.toImage().pixelColor(0, 0)
228 |             self.assertEqual((color.red(), color.green(), color.blue()), (0, 255, 0))
229 |             window.shutdown()
230 |             window.close()
231 |
232 |     def test_window_soft_edges_toggle_persists_and_reloads_clip(self) -> None:
233 |         image = QImage(20, 20, QImage.Format.Format_RGBA8888)
234 |         image.fill(QColor(255, 255, 255, 255))
235 |         library = _FrameOnlyLibrary(_FrameOnlyClip(trim_transparent_frame(image)))
236 |
237 |         with tempfile.TemporaryDirectory() as tmp:
238 |             config = Config(base=tmp)
239 |             window = PetWindow(library, config)
240 |             window._resource_timer.stop()
241 |
242 |             window.set_soft_edges(False)
243 |
244 |             self.assertFalse(window.soft_edges)
245 |             self.assertFalse(library.soft_edges)
246 |             self.assertFalse(config.get("soft_edges"))
247 |             window.shutdown()
248 |             window.close()
249 |
250 |
251 | if __name__ == "__main__":

```

24. tests/test_runtime.py (77 lines)

```

1 | # -*- coding: utf-8 -*-
2 | """目录/常量完整性测试（无需 GUI）."""
3 |
4 | from pet import catalog
5 | from pet.interaction import classify_tap_burst, cursor_facing, edge_contacts
6 | from pet.hotkeys import GlobalHotkeys
7 | from pet.performance import LoadGovernor
8 | from pet.sound import DuckScream
9 |
10 |
11 | def test_catalog_integrity():
12 |     assert len(catalog.ANIM_FILES) == 91
13 |     assert len(catalog.ACTS) == 80
14 |     assert len(catalog.CLICKS) == 5
15 |     assert len(catalog.MOVES) == 3
16 |     assert catalog.IDLE in catalog.ANIM_FILES
17 |     assert catalog.TURN in catalog.ANIM_FILES
18 |     assert catalog.DRAG in catalog.ANIM_FILES
19 |     assert all(n in catalog.ANIM_FILES for n in catalog.CLICKS + catalog.MOVES)
20 |     assert catalog.FRAME_MS > 0
21 |     assert (catalog.DECODE_MAX_W, catalog.DECODE_MAX_H) == (1280, 720)
22 |     assert catalog.decode_size_for_scale(0.72) == (922, 520)
23 |     assert catalog.decode_size_for_scale(1.0) == (1280, 720)
24 |     assert catalog.BUSY_MASK_FRAME_INTERVAL == 2
25 |
26 |
27 | def test_load_governor_hysteresis():
28 |     governor = LoadGovernor()
29 |     assert governor.observe(0.80) is False
30 |     assert governor.observe(0.80) is True
31 |     assert governor.constrained is True
32 |     assert governor.observe(0.40) is False
33 |     assert governor.observe(0.40) is False
34 |     assert governor.observe(0.40) is True
35 |     assert governor.constrained is False
36 |
37 |
38 | def test_interaction_gesture_classification():
39 |     assert classify_tap_burst(1) == "single"
40 |     assert classify_tap_burst(2) == "double"
41 |     assert classify_tap_burst(5) == "rapid"
42 |     assert cursor_facing((100, 100), (250, 110), 200) == "right"
43 |     assert cursor_facing((100, 100), (-50, 110), 200) == "left"
44 |     assert cursor_facing((100, 100), (105, 100), 200) is None
45 |     assert cursor_facing((100, 100), (400, 100), 200) is None
46 |
47 |
48 | def test_interaction_edge_contacts():
49 |     assert edge_contacts((0, 20, 100, 100), (0, 0, 800, 600), 18) == ["left"]
50 |     assert edge_contacts((700, 500, 100, 100), (0, 0, 800, 600), 18) == {
51 |         "right",
52 |         "bottom",
53 |     }
54 |
55 |
56 | def test_duck_sound_generates_small_wav(tmp_path):
57 |     sound = DuckScream(tmp_path)
58 |     sound._ensure_wave()
59 |     assert sound.path.is_file()
60 |     assert sound.path.stat().st_size > 1_000
61 |     sound.close()
62 |
63 |
64 | def test_personality_presets_and_global_hotkeys_are_stable():
65 |     assert set(catalog.PERSONALITY_PRESETS) == {'quiet', 'lively', 'mischievous'}
66 |     for profile in catalog.PERSONALITY_PRESETS.values():
67 |         assert abs(
68 |             profile['idle'] + profile['turn'] + profile['acts'] + profile['move']
69 |             - 1.0
70 |         ) < 1e-6
71 |     assert [spec.action for spec in GlobalHotkeys.DEFAULTS] == [
72 |         'toggle_visible',

```

```

73 |         'toggle_pause',
74 |         'random_action',
75 |         'toggle_mouse_through',
76 |         'duck_sound',
77 |     ]

```

25. tests/win_diag.py (61 lines)

```

1 | # -*- coding: utf-8 -*-
2 | """真实 Windows 窗口诊断：枚举指定 PID 的顶层窗口，查位置/可见性/样式。"""
3 |
4 | from __future__ import annotations
5 |
6 | import ctypes
7 | import sys
8 | import time
9 | from ctypes import windypes
10 |
11 | user32 = ctypes.windll.user32
12 |
13 | WNDENUMPROC = ctypes.WINFUNCTYPE(wintypes.BOOL, wintypes.HWND, wintypes.LPARAM)
14 |
15 |
16 | def enum_windows(pid: int):
17 |     results = []
18 |
19 |     def cb(hwnd, lparam):
20 |         proc = wintypes.DWORD()
21 |         user32.GetWindowThreadProcessId(hwnd, ctypes.byref(proc))
22 |         if proc.value == pid:
23 |             rect = wintypes.RECT()
24 |             user32.GetWindowRect(hwnd, ctypes.byref(rect))
25 |             visible = user32.IsWindowVisible(hwnd)
26 |             style = user32.GetWindowLongW(hwnd, -16) # GWL_STYLE
27 |             exstyle = user32.GetWindowLongW(hwnd, -20) # GWL_EXSTYLE
28 |             results.append({
29 |                 'hwnd': hwnd,
30 |                 'rect': (rect.left, rect.top, rect.right, rect.bottom),
31 |                 'visible': bool(visible),
32 |                 'exstyle': exstyle,
33 |                 'style': style,
34 |             })
35 |             return True
36 |
37 |     user32.EnumWindows(WNDENUMPROC(cb), 0)
38 |     return results
39 |
40 |
41 | def main() -> int:
42 |     pid = int(sys.argv[1]) if len(sys.argv) > 1 else 0
43 |     if not pid:
44 |         print('用法: python tests/win_diag.py <pid>')
45 |         return 1
46 |     wins = enum_windows(pid)
47 |     print(f'PID {pid} 的顶层窗口数: {len(wins)}')
48 |     for w in wins:
49 |         l, t, r, b = w['rect']
50 |         print(f" hwnd={w['hwnd']:#x} rect=({l},{t},{r},{b}) 尺寸={r-l}x{b-t} "
51 |               f"visible={w['visible']} exstyle={w['exstyle']:#x}")
52 |         # WS_EX_LAYERED=0x80000, WS_EX_TOOLWINDOW=0x80, WS_EX_TRANSPARENT=0x20
53 |         if w['exstyle'] & 0x80000:
54 |             print('    含 WS_EX_LAYERED (分层/透明窗口)')
55 |         if w['exstyle'] & 0x80:
56 |             print('    含 WS_EX_TOOLWINDOW')
57 |     return 0
58 |
59 |
60 | if __name__ == '__main__':
61 |     raise SystemExit(main())

```

26. tools/build_hq_webm.py (330 lines)

```

1 | #!/usr/bin/env python3

```

```

2 | """从 dsh-pet 源 MP4 生成高质量透明 WebM。
3 |
4 | 处理链:
5 | 1. 用原项目 watermark_mask_v5.mkv 填充水印区域;
6 | 2. 在 1280x720 原始分辨率上做 YCbCr 色差抠图和边缘连通清理;
7 | 3. 沿背景色差方向做绿色溢色抑制;
8 | 4. 以 1280x720、24fps、VP9 alpha 输出, 运行时再缩放到窗口尺寸。
9 |
10 | 用法示例:
11 |     python tools/build_hq_webm.py \
12 |         --src-dir /private/tmp/dsh-quality/sources \
13 |         --mask /private/tmp/dsh-quality/sources/watermark_mask_v5.mkv \
14 |         --out-dir /private/tmp/dsh-quality/step-hq
15 | """
16 |
17 | from __future__ import annotations
18 |
19 | import argparse
20 | import os
21 | import subprocess
22 | from concurrent.futures import ThreadPoolExecutor
23 | from pathlib import Path
24 |
25 | import numpy as np
26 | from scipy.ndimage import convolve, distance_transform_edt, label
27 |
28 | W, H, FPS = 1280, 720, 24
29 | CRF = 24
30 | WATERMARK_CLEAR_W, WATERMARK_CLEAR_H = 240, 130
31 | CHROMA_IN, CHROMA_OUT = 8.0, 28.0
32 | FLOOD_TOL, DESPILL_BAND = 16.0, 42.0
33 |
34 | # Release 中的源视频使用拼音名, 运行时素材使用中文名。
35 | SOURCE_TO_ANIM = [
36 |     'beiluoye-yanmo': '被落叶淹没',
37 |     'beishubiao-tuozhuai-xuankong-fankui': '被鼠标拖拽悬空反饋',
38 |     'beixiayitiao-zhamao': '被吓一跳(炸毛)',
39 |     'chaoda-shenlanyao': '超大伸懒腰',
40 |     'chi-baifan': '吃白饭',
41 |     'chi-bingqilin-ronghua': '吃冰淇淋融化',
42 |     'chi-token': '吃Token',
43 |     'chi-wancan': '吃晚餐',
44 |     'chi-wucan': '吃午餐',
45 |     'chi-zaocan': '吃早餐',
46 |     'chui-qiqiu': '吹气球',
47 |     'da-keshui-bei-jingxing': '打瞌睡被惊醒',
48 |     'daiji-huxi-xiuxian': '待机呼吸休闲',
49 |     'dakou-chi-lingshi': '大口吃零食',
50 |     'dianji-huiying-aojiao-shengqi-ceshen-zhanshi': '点击回应 - 傲娇生气(侧身展示)',
51 |     'dianji-huiying-haixiu-jingya': '点击回应 - 害羞惊讶',
52 |     'dianji-huiying-kaixin-yuedong': '点击回应 - 开心跃动',
53 |     'dongwu-huanrao': '动物环绕',
54 |     'dongzhangxiwang': '东张西望',
55 |     'duixueren': '堆雪人',
56 |     'fang-fengzheng': '放风筝',
57 |     'haqian-liantian': '哈欠连天',
58 |     'jingyu-tu-paopao-texiao': '鲸鱼吐泡泡特效',
59 |     'keai-zhaiwu': '可爱宅男',
60 |     'lanjing-xianshi': '蓝鲸现世',
61 |     'nvpu-quxi-liyi': '女仆屈膝礼仪',
62 |     'pangxie-zoulu': '螃蟹走路',
63 |     'qingkuai-jilu': '轻快记录',
64 |     'qingkuai-yaobaiwu': '轻快摇摆舞',
65 |     'shendu-sikao-suisuonian': '深度思考碎碎念',
66 |     'touchi-lingshi-bei-zhuazhu': '偷吃零食被抓住',
67 |     'wan-shuiqiang': '玩水枪',
68 |     'wan-youxi-qijibaituai': '玩游戏气急败坏',
69 |     'xiaofudu-yuandi-360du-xuanzhuan-zhanshi': '小幅度原地 360 度旋转展示',
70 |     'xiaotiqin-yanzou': '小提琴演奏',
71 |     'xie-daima': '写代码',
72 |     'yaoshan-naliang': '摇扇纳凉',
73 |     'yong-jingyu-weiba-paidadi': '用鲸鱼尾巴拍打地面',
74 |     'youxian-hengga': '悠闲哼歌',
75 |     'youya-nvpuwu': '优雅女仆舞',
76 |     'yuandi-dunxia-wan-wanju-qiche': '原地蹲下玩玩具汽车',
77 |     'yuandi-piaofu-tabu': '原地漂浮踏步',
78 |     'yuandi-qiaoji-zhuomian-hudong': '原地敲击桌面互动',
79 |     'yuandi-tiaoyue-zhuasui-touding-wupin': '原地跳跃抓碎头顶物品',
80 |     'yuandi-xiaoqi-chenmian': '原地小憩沉眠',

```

```

81 |     'yuandi-zhongli-xiadun-yasuo': '原地重力下蹲压缩',
82 |     'yuandi-zhuanxin-wan-mofang': '原地专心玩魔方',
83 |     'yuandi-zuozhuan-benpao': '原地左转奔跑',
84 |     'zhao-jingzi': '照镜子',
85 |     'zhengti-huanzhuang-shise': '整体换装试色',
86 |     'zhongqiu-shangyue-chi-yuebing': '中秋赏月吃月饼',
87 | }
88 |
89 |
90 | def _read_exact(stream, size: int) -> bytes:
91 |     chunks: list[bytes] = []
92 |     remaining = size
93 |     while remaining:
94 |         chunk = stream.read(remaining)
95 |         if not chunk:
96 |             break
97 |         chunks.append(chunk)
98 |         remaining -= len(chunk)
99 |     return b''.join(chunks)
100 |
101 |
102 | def _kth_nearest(mask: np.ndarray, k: int = 5) -> tuple[np.ndarray, np.ndarray]:
103 |     current = mask.copy()
104 |     iy = ix = None
105 |     for _ in range(k):
106 |         _, indices = distance_transform_edt(current, return_indices=True)
107 |         iy, ix = indices
108 |         used = np.zeros_like(current, dtype=bool)
109 |         used[iy[mask], ix[mask]] = True
110 |         current |= used
111 |     assert iy is not None and ix is not None
112 |     return iy, ix
113 |
114 |
115 | def _fill_watermark(src: Path, mask_path: Path, dst: Path, ffmpeg: str) -> None:
116 |     raw = W * H * 3
117 |     mask_raw = W * H
118 |     reader = subprocess.Popen(
119 |         [ffmpeg, '-hide_banner', '-loglevel', 'error', '-i', str(src),
120 |          '-f', 'rawvideo', '-pix_fmt', 'rgb24', '-'],
121 |         stdout=subprocess.PIPE,
122 |     )
123 |     mask_reader = subprocess.Popen(
124 |         [ffmpeg, '-hide_banner', '-loglevel', 'error', '-i', str(mask_path),
125 |          '-f', 'rawvideo', '-pix_fmt', 'gray', '-'],
126 |         stdout=subprocess.PIPE,
127 |     )
128 |     writer = subprocess.Popen(
129 |         [ffmpeg, '-y', '-hide_banner', '-loglevel', 'error',
130 |          '-f', 'rawvideo', '-pix_fmt', 'rgb24', '-s', f'[W]x[H]', '-r', str(FPS), '-i', '-',
131 |          '-c:v', 'libx264', '-pix_fmt', 'yuv420p', '-crf', '18', '-preset', 'fast', str(dst)],
132 |         stdin=subprocess.PIPE,
133 |     )
134 |     frames = 0
135 |     try:
136 |         while True:
137 |             buf = _read_exact(reader.stdout, raw)
138 |             if len(buf) < raw:
139 |                 break
140 |             mbuf = _read_exact(mask_reader.stdout, mask_raw)
141 |             if len(mbuf) < mask_raw:
142 |                 raise RuntimeError(f'mask 帧数不足: {mask_path}')
143 |             frame = np.frombuffer(buf, np.uint8).reshape(H, W, 3).copy()
144 |             wm = np.frombuffer(mbuf, np.uint8).reshape(H, W) == 255
145 |             if wm.any():
146 |                 iy, ix = _kth_nearest(wm)
147 |                 frame[wm] = frame[np.clip(iy[wm], 0, H - 1), np.clip(ix[wm], 0, W - 1)]
148 |                 writer.stdin.write(frame.tobytes())
149 |                 frames += 1
150 |     finally:
151 |         reader.stdout.close()
152 |         mask_reader.stdout.close()
153 |         writer.stdin.close()
154 |         rc = (reader.wait(), mask_reader.wait(), writer.wait())
155 |     if rc != (0, 0, 0):
156 |         raise RuntimeError(f'去水印失败 {src.name}: returncodes={rc}')
157 |     if frames < 2:
158 |         raise RuntimeError(f'去水印没有生成有效帧: {src.name}')
159 |

```

```

160 |
161 | def _rgb_to_yuv(frame: np.ndarray) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
162 |     r = frame[..., 0].astype(np.float32)
163 |     g = frame[..., 1].astype(np.float32)
164 |     b = frame[..., 2].astype(np.float32)
165 |     y = np.clip(16 + 0.257 * r + 0.504 * g + 0.098 * b, 16, 235).astype(np.uint8)
166 |     u = np.clip(128 - 0.148 * r - 0.291 * g + 0.439 * b, 16, 240)
167 |     v = np.clip(128 + 0.439 * r - 0.368 * g - 0.071 * b, 16, 240)
168 |     u = u.reshape(H // 2, 2, W // 2, 2).mean(axis=(1, 3)).astype(np.uint8)
169 |     v = v.reshape(H // 2, 2, W // 2, 2).mean(axis=(1, 3)).astype(np.uint8)
170 |     return y, u, v
171 |
172 |
173 | def _rgb_to_ycbcr(frame: np.ndarray) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
174 |     rgb = frame.astype(np.float32)
175 |     r, g, b = rgb[..., 0], rgb[..., 1], rgb[..., 2]
176 |     y = 0.299 * r + 0.587 * g + 0.114 * b
177 |     cb = (b - y) * 0.564 + 128.0
178 |     cr = (r - y) * 0.713 + 128.0
179 |     return y, cb, cr
180 |
181 |
182 | def _ycbcr_to_rgb(y: np.ndarray, cb: np.ndarray, cr: np.ndarray) -> np.ndarray:
183 |     r = y + 1.402 * (cr - 128.0)
184 |     g = y - 0.344136 * (cb - 128.0) - 0.714136 * (cr - 128.0)
185 |     b = y + 1.772 * (cb - 128.0)
186 |     return np.clip(np.stack((r, g, b), axis=-1), 0, 255).astype(np.uint8)
187 |
188 |
189 | def _detect_background_chroma(frame: np.ndarray) -> tuple[float, float]:
190 |     """用四角色差直方图估计低饱和绿幕的 Cb/Cr 键值。"""
191 |     h, w = frame.shape[:2]
192 |     cw, ch = max(2, w // 5), max(2, h // 5)
193 |     step = 4
194 |     sample = np.concatenate((
195 |         frame[ch:step, :cw:step].reshape(-1, 3),
196 |         frame[ch:step, -cw:step].reshape(-1, 3),
197 |         frame[-ch:step, :cw:step].reshape(-1, 3),
198 |         frame[-ch:step, -cw:step].reshape(-1, 3),
199 |     ), axis=0)
200 |     _, cb, cr = _rgb_to_ycbcr(sample)
201 |     cb_bin = np.clip(cb.astype(np.int16) // 8, 0, 31)
202 |     cr_bin = np.clip(cr.astype(np.int16) // 8, 0, 31)
203 |     bins = cb_bin * 32 + cr_bin
204 |     mode = int(np.bincount(bins, minlength=1024).argmax())
205 |     selected = bins == mode
206 |     return float(cb[selected].mean()), float(cr[selected].mean())
207 |
208 |
209 | def _encode_alpha(src: Path, dst: Path, ffmpeg: str) -> None:
210 |     raw = W * H * 3
211 |     reader = subprocess.Popen(
212 |         [ffmpeg, '-hide_banner', '-loglevel', 'error', '-i', str(src),
213 |          '-f', 'rawvideo', '-pix_fmt', 'rgb24', '-'],
214 |         stdout=subprocess.PIPE,
215 |     )
216 |     writer = subprocess.Popen(
217 |         [ffmpeg, '-y', '-hide_banner', '-loglevel', 'error',
218 |          '-f', 'rawvideo', '-pix_fmt', 'yuva420p', '-s', f'[W]x[H]', '-r', str(FPS), '-i', '-',
219 |          '-c:v', 'libvpx-vp9', '-pix_fmt', 'yuva420p', '-crf', str(CRF), '-b:v', '0',
220 |          '-auto-alt-ref', '0', '-row-mt', '1', '-deadline', 'good', '-cpu-used', '4', '-an', str(dst)],
221 |         stdin=subprocess.PIPE,
222 |     )
223 |     frames = 0
224 |     try:
225 |         while True:
226 |             buf = _read_exact(reader.stdout, raw)
227 |             if len(buf) < raw:
228 |                 break
229 |             frame = np.frombuffer(buf, np.uint8).reshape(H, W, 3).copy()
230 |             y, cb, cr = _rgb_to_ycbcr(frame)
231 |             key_cb, key_cr = _detect_background_chroma(frame)
232 |             dist = np.hypot(cb - key_cb, cr - key_cr)
233 |             t = np.clip((dist - CHROMA_IN) / (CHROMA_OUT - CHROMA_IN), 0, 1)
234 |             alpha = (t * t * (3.0 - 2.0 * t) * 255.0).astype(np.uint8)
235 |
236 |             # PerfectPixel 的色差空间 despill: 只沿绿幕的 Cb/Cr 方向回收边缘绿, 不压暗角色。
237 |             key_vec_cb, key_vec_cr = key_cb - 128.0, key_cr - 128.0
238 |             key_len = max(float(np.hypot(key_vec_cb, key_vec_cr)), 1.0)

```

```

239 |         pcb, pcr = cb - 128.0, cr - 128.0
240 |         proj = (pcb * key_vec_cb + pcr * key_vec_cr) / key_len
241 |         band = np.clip((DESPILL_BAND - dist) / DESPILL_BAND, 0, 1)
242 |         weight = band * band * (3.0 - 2.0 * band) * 0.92
243 |         spill = (proj > 0) & (dist < DESPILL_BAND)
244 |         amount = proj * weight * spill
245 |         pcb -= (key_vec_cb / key_len) * amount
246 |         pcr -= (key_vec_cr / key_len) * amount
247 |         frame = _ycbcr_to_rgb(y, pcb + 128.0, pcr + 128.0)
248 |
249 |         # 从四边连通的背景色做 flood fill, 清除地面阴影和压缩噪声, 但保留主体内部同色细节。
250 |         key_connected = dist <= FL000_TOL
251 |         components, component_count = label(key_connected, structure=np.array(
252 |             [[0, 1, 0], [1, 1, 1], [0, 1, 0]], dtype=np.uint8,
253 |         ))
254 |         border_labels = np.unique(np.concatenate((
255 |             components[0, :], components[-1, :], components[:, 0], components[:, -1],
256 |         )))
257 |         border_labels = border_labels[border_labels > 0]
258 |         background = np.isin(components, border_labels)
259 |         alpha[background] = 0
260 |
261 |         # 清掉孤立压缩点, 并填补被噪声包围的 1px 小孔。
262 |         opaque = alpha > 20
263 |         neighbors = convolve(opaque.astype(np.uint8), np.ones((3, 3), np.uint8), mode='constant') - opaque
264 |         alpha[(alpha > 20) & (neighbors == 0)] = 0
265 |         alpha[(alpha <= 20) & (neighbors >= 7)] = 255
266 |
267 |         # 源片左上角固定水印在少数帧会留下 1px 残点: 该区域不可能出现角色主体。
268 |         alpha[:WATERMARK_CLEAR_H, :WATERMARK_CLEAR_W] = 0
269 |         frame[:WATERMARK_CLEAR_H, :WATERMARK_CLEAR_W] = 0
270 |
271 |         y, u, v = _rgb_to_yuv(frame)
272 |         writer.stdin.write(y.tobytes() + u.tobytes() + v.tobytes() + alpha.tobytes())
273 |         frames += 1
274 |     finally:
275 |         reader.stdout.close()
276 |         writer.stdin.close()
277 |         rc = (reader.wait(), writer.wait())
278 |     if rc != (0, 0):
279 |         raise RuntimeError(f'抠图编码失败 {src.name}: returncodes={rc}')
280 |     if frames < 2:
281 |         raise RuntimeError(f'抠图没有生成有效帧: {src.name}')
282 |
283 |
284 | def _process_one(job: tuple[Path, Path, Path, str, bool]) -> str:
285 |     source, mask, out_dir, ffmpeg, force = job
286 |     name = SOURCE_TO_ANIM[source.stem]
287 |     output = out_dir / f'{name}.webm'
288 |     if not force and output.exists() and output.stat().st_size > 100_000:
289 |         probe = subprocess.run(
290 |             [ffmpeg.replace('ffmpeg', 'ffprobe'), '-v', 'error', '-show_entries',
291 |              'stream=codec_name,width,height', '-of', 'csv=p=0', str(output)],
292 |             capture_output=True, text=True,
293 |         )
294 |         if probe.returncode == 0 and 'vp9,1280,720' in probe.stdout:
295 |             return f'{name}: SKIP'
296 |
297 |     step01 = out_dir / f'{name}.step01.mp4'
298 |     _fill_watermark(source, mask, step01, ffmpeg)
299 |     _encode_alpha(step01, output, ffmpeg)
300 |     step01.unlink(missing_ok=True)
301 |     return f'{name}: {output.stat().st_size / 1e6:.2f} MB'
302 |
303 |
304 | def main() -> int:
305 |     parser = argparse.ArgumentParser()
306 |     parser.add_argument('--src-dir', type=Path, required=True)
307 |     parser.add_argument('--mask', type=Path, required=True)
308 |     parser.add_argument('--out-dir', type=Path, required=True)
309 |     parser.add_argument('--ffmpeg', default='ffmpeg')
310 |     parser.add_argument('--workers', type=int, default=min(4, os.cpu_count() or 1))
311 |     parser.add_argument('--force', action='store_true', help='覆盖已有输出')
312 |     args = parser.parse_args()
313 |     args.out_dir.mkdir(parents=True, exist_ok=True)
314 |
315 |     sources = sorted(args.src_dir.glob('*.*mp4'))
316 |     if len(sources) != len(SOURCE_TO_ANIM):
317 |         raise SystemExit(f'源视频数量不正确: {len(sources)}, 期望 {len(SOURCE_TO_ANIM)}')

```

```

318 |     missing = [p.stem for p in sources if p.stem not in SOURCE_TO_ANIM]
319 |     if missing:
320 |         raise SystemExit(f'缺少名称映射: {missing}')
321 |
322 |     jobs = [(source, args.mask, args.out_dir, args.ffmpeg, args.force) for source in sources]
323 |     with ThreadPoolExecutor(max_workers=max(1, args.workers)) as executor:
324 |         for index, result in enumerate(executor.map(_process_one, jobs), start=1):
325 |             print(f'[{index}/{len(sources)}] {result}', flush=True)
326 |     return 0
327 |
328 |
329 | if __name__ == '__main__':
330 |     raise SystemExit(main())

```

27. tools/build_project_pdf.py (253 lines)

```

1 | from __future__ import annotations
2 |
3 | import os
4 | from datetime import date
5 | from pathlib import Path
6 |
7 | from reportlab.lib import colors
8 | from reportlab.lib.enums import TA_CENTER, TA_LEFT
9 | from reportlab.lib.pagesizes import A4
10 | from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
11 | from reportlab.lib.units import mm
12 | from reportlab.pdfbase import pdfmetrics
13 | from reportlab.pdfbase.ttfonts import TTFont
14 | from reportlab.platypus import (
15 |     Image,
16 |     KeepTogether,
17 |     PageBreak,
18 |     Paragraph,
19 |     SimpleDocTemplate,
20 |     Spacer,
21 |     Table,
22 |     TableStyle,
23 | )
24 |
25 |
26 | ROOT = Path(__file__).resolve().parents[1]
27 | OUT = ROOT / "output" / "pdf" / "dsh-pet-indesktop-project-report.pdf"
28 | FONT = Path("/Users/ray/Library/Fonts/SimHei.ttf")
29 | ICON = ROOT / "assets" / "app-icon.png"
30 |
31 |
32 | def register_fonts() -> str:
33 |     if FONT.exists():
34 |         pdfmetrics.registerFont(TTFont("ProjectSans", str(FONT)))
35 |         return "ProjectSans"
36 |     pdfmetrics.registerFont(TTFont("ProjectSans", "/System/Library/Fonts/STHeiti Light.ttc"))
37 |     return "ProjectSans"
38 |
39 |
40 | def p(text: str, style: ParagraphStyle) -> Paragraph:
41 |     return Paragraph(text, style)
42 |
43 |
44 | def footer(canvas, doc) -> None:
45 |     canvas.saveState()
46 |     canvas.setStrokeColor(colors.HexColor("#D7E5F2"))
47 |     canvas.line(18 * mm, 15 * mm, 192 * mm, 15 * mm)
48 |     canvas.setFillColor(colors.HexColor("#8C8298"))
49 |     canvas.setFont("ProjectSans", 8)
50 |     canvas.drawString(18 * mm, 10 * mm, "DSH-Pet project report")
51 |     canvas.drawRightString(192 * mm, 10 * mm, f"[doc.page]")
52 |     canvas.restoreState()
53 |
54 |
55 | def build() -> Path:
56 |     font_name = register_fonts()
57 |     OUT.parent.mkdir(parents=True, exist_ok=True)
58 |
59 |     doc = SimpleDocTemplate(
60 |         str(OUT),

```



```

61 |         pageSize=A4,
62 |         rightMargin=18 * mm,
63 |         leftMargin=18 * mm,
64 |         topMargin=17 * mm,
65 |         bottomMargin=22 * mm,
66 |         title="DSH-Pet 项目交付报告",
67 |         author="Codex",
68 |     )
69 |
70 |     styles = getSampleStyleSheet()
71 |     title = ParagraphStyle(
72 |         "TitleZH", parent=styles["Title"], fontName=font_name, fontSize=25,
73 |         leading=33, textColor=colors.HexColor("#12385A"), alignment=TA_CENTER,
74 |         spaceAfter=6 * mm,
75 |     )
76 |     subtitle = ParagraphStyle(
77 |         "SubtitleZH", parent=styles["Normal"], fontName=font_name, fontSize=11,
78 |         leading=18, textColor=colors.HexColor("#54728E"), alignment=TA_CENTER,
79 |         spaceAfter=8 * mm,
80 |     )
81 |     h1 = ParagraphStyle(
82 |         "Heading1ZH", parent=styles["Heading1"], fontName=font_name, fontSize=16,
83 |         leading=23, textColor=colors.HexColor("#12385A"), spaceBefore=5 * mm,
84 |         spaceAfter=3 * mm,
85 |     )
86 |     h2 = ParagraphStyle(
87 |         "Heading2ZH", parent=styles["Heading2"], fontName=font_name, fontSize=11.5,
88 |         leading=17, textColor=colors.HexColor("#1B6085"), spaceBefore=3 * mm,
89 |         spaceAfter=1.5 * mm,
90 |     )
91 |     body = ParagraphStyle(
92 |         "BodyZH", parent=styles["BodyText"], fontName=font_name, fontSize=9.4,
93 |         leading=16, textColor=colors.HexColor("#243746"), alignment=TA_LEFT,
94 |         spaceAfter=2.2 * mm,
95 |     )
96 |     small = ParagraphStyle(
97 |         "SmallZH", parent=body, fontSize=8.3, leading=13, textColor=colors.HexColor("#53697B"),
98 |     )
99 |     cell = ParagraphStyle(
100 |         "CellZH", parent=body, fontSize=8.4, leading=13, spaceAfter=0,
101 |     )
102 |     cell_bold = ParagraphStyle(
103 |         "CellBoldZH", parent=cell, textColor=colors.HexColor("#12385A"),
104 |     )
105 |     cell_header = ParagraphStyle(
106 |         "CellHeaderZH", parent=cell, textColor=colors.white,
107 |     )
108 |     bullet = ParagraphStyle(
109 |         "BulletZH", parent=body, leftIndent=5 * mm, firstLineIndent=3 * mm,
110 |         bulletIndent=0, spaceAfter=1.1 * mm,
111 |     )
112 |
113 |     story = []
114 |     if ICON.exists():
115 |         icon = Image(str(ICON), width=32 * mm, height=32 * mm)
116 |         icon.hAlign = "CENTER"
117 |         story.extend([Spacer(1, 5 * mm), icon, Spacer(1, 4 * mm)])
118 |     story.extend([
119 |         p("DSH-Pet", title),
120 |         p("桌面动态宠物项目交付报告", subtitle),
121 |         p("版本范围: 当前工作区代码与素材 • 生成日期: " + date.today().isoformat(), subtitle),
122 |     ])
123 |
124 |     summary_data = [
125 |         [p("项目", cell_bold), p("DSH-Pet 独立桌面宠物", cell)],
126 |         [p("代码仓库", cell_bold), p("https://github.com/MerZlin/dsh-pet-indesktop", cell)],
127 |         [p("上传分支", cell_bold), p("codex/upload-interaction-pdf", cell)],
128 |         [p("运行形态", cell_bold), p("macOS 独立应用, 状态栏菜单控制, 桌面透明窗口展示", cell)],
129 |         [p("角色素材", cell_bold), p("shenshen, 91 段 WebM 动画, 包含互动、移动、待机、随机和转身分类", cell)],
130 |     ]
131 |     summary = Table(summary_data, colWidths=[30 * mm, 144 * mm], hAlign="CENTER")
132 |     summary.setStyle(TableStyle([
133 |         ("BACKGROUND", (0, 0), (0, -1), colors.HexColor("#EAF4FB")),
134 |         ("BACKGROUND", (1, 0), (1, -1), colors.HexColor("#F8BFE")),
135 |         ("BOX", (0, 0), (-1, -1), 0.6, colors.HexColor("#B9D4E8")),
136 |         ("INNERGRID", (0, 0), (-1, -1), 0.35, colors.HexColor("#D7E5F2")),
137 |         ("VALIGN", (0, 0), (-1, -1), "TOP"),
138 |         ("LEFTPADDING", (0, 0), (-1, -1), 3 * mm),
139 |         ("RIGHTPADDING", (0, 0), (-1, -1), 3 * mm),

```

```

140 |         ("TOPPADDING", (0, 0), (-1, -1), 2.2 * mm),
141 |         ("BOTTPADDING", (0, 0), (-1, -1), 2.2 * mm),
142 |     ])
143 |     story.extend([summary, PageBreak()])
144 |
145 |     story.append(p("1. 本次交付内容", h1))
146 |     for item in [
147 |         "桌面宠物基础运行：透明窗口、状态栏菜单、动画播放和退出控制。",
148 |         "互动反馈：双击、长按、拖拽、快速连续点击、鼠标靠近注视、边缘反弹和主动问候。",
149 |         "动作收藏夹与播放列表：91 段动画可收藏，可按收藏列表循环、顺序或随机播放。",
150 |         "全局快捷键：显示或隐藏、暂停或继续、随机动作、鼠标穿透、尖叫鸭。",
151 |         "情绪与性格：安静、活泼、调皮三种模式，分别调整待机、转身、主动动作和问候概率。",
152 |         "媒体质量处理：保留高画质 WebM 资产，并在透明区执行 Alpha=1 清理，降低透明噪声和边缘残留。",
153 |         "发布准备：保留 macOS 应用构建方式、超分处理脚本、测试脚本和图标资源。",
154 |     ]:
155 |         story.append(p(" " + item, bullet))
156 |
157 |     story.append(p("2. 架构说明", h1))
158 |     architecture = [
159 |         [p("层", cell_header), p("主要文件", cell_header), p("职责", cell_header)],
160 |         [p("应用入口", cell_bold), p("pet/app.py", cell)], p("创建 QApplication、状态栏菜单、全局快捷键和生命周期管理。", cell)],
161 |         [p("桌面窗口", cell_bold), p("pet/window.py", cell)], p("处理窗口展示、动画状态、鼠标交互、播放列表、性格模式和菜单动作。", cell)],
162 |         [p("素材目录", cell_bold), p("pet/catalog.py\npet/library.py", cell)], p("发现角色、分类动作、收藏集合和动画元数据，减少窗口层对文件结构的耦合。", cell)],
163 |         [p("媒体播放", cell_bold), p("pet/webm_clip.py", cell)], p("读取 WebM 帧、处理透明通道、固定画布绘制，避免窗口内容随动画帧抖动。", cell)],
164 |         [p("交互与声音", cell_bold), p("pet/interaction.py\npet/sound.py", cell)], p("集中管理点击、拖拽、边缘状态、靠近注视和尖叫鸭反馈。", cell)],
165 |         [p("可扩展控制", cell_bold), p("pet/action_dialog.py\npet/hotkeys.py", cell)], p("动作选择对话框与 macOS Carbon 全局快捷键注册，后续可扩展更多控制项。", cell)],
166 |     ]
167 |     arch_table = Table(architecture, colWidths=[25 * mm, 45 * mm, 104 * mm], repeatRows=1)
168 |     arch_table.setStyle(TableStyle([
169 |         ("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#1B6085")),
170 |         ("TEXTCOLOR", (0, 0), (-1, 0), colors.white),
171 |         ("BACKGROUND", (0, 1), (-1, -1), colors.HexColor("#F8FBFE")),
172 |         ("ROWBACKGROUNDS", (0, 1), (-1, -1), [colors.HexColor("#F8FBFE"), colors.HexColor("#EEF6FB")]),
173 |         ("BOX", (0, 0), (-1, -1), 0.5, colors.HexColor("#B9D4E8")),
174 |         ("INNERGRID", (0, 0), (-1, -1), 0.3, colors.HexColor("#CFE0EC")),
175 |         ("VALIGN", (0, 0), (-1, -1), "TOP"),
176 |         ("LEFTPADDING", (0, 0), (-1, -1), 2.2 * mm),
177 |         ("RIGHTPADDING", (0, 0), (-1, -1), 2.2 * mm),
178 |         ("TOPPADDING", (0, 0), (-1, -1), 2 * mm),
179 |         ("BOTTPADDING", (0, 0), (-1, -1), 2 * mm),
180 |     ]))
181 |     story.extend([arch_table, Spacer(1, 3 * mm)])
182 |     story.append(p("设计原则：高频动画播放逻辑与低频控制面板分离；配置写入用户目录；素材代际保持独立，便于后续替换超分版本或新增角色。", body))
183 |
184 |     story.append(p("3. 操作说明", h1))
185 |     for item in [
186 |         "右键状态栏图标：打开通作管理、收藏夹、播放列表、性格模式和显示控制。",
187 |         "动作收藏：在动作管理中勾选需要的动画，确认后可将收藏集设为播放列表。",
188 |         "播放模式：关闭、顺序循环、随机循环三种状态；播放列表为空时自动回退到普通动作逻辑。",
189 |         "性格模式：安静减少主动动作，活泼保持平衡，调皮提高随机动作和主动问候频率。",
190 |         "默认全局快捷键：Control + Option + Command + H 显示或隐藏；P 暂停；R 随机动作；M 鼠标穿透；D 尖叫鸭。",
191 |         "鼠标穿透开启后，窗口不接收鼠标命中；再次使用快捷键 M 或状态栏菜单即可恢复。",
192 |     ]:
193 |         story.append(p(" " + item, bullet))
194 |
195 |     story.append(p("4. 质量验证", h1))
196 |     validation = [
197 |         [p("检查项", cell_header), p("结果", cell_header), p("证据", cell_header)],
198 |         [p("运行时单元测试", cell)], p("通过", cell_bold), p("tests.test_media_runtime: 6 tests OK.", cell)],
199 |         [p("交互辅助测试", cell)], p("通过", cell_bold), p("offscreen 交互、收藏夹、性格和播放列表 harness 均返回 OK.", cell)],
200 |         [p("全局快捷键", cell)], p("通过", cell_bold), p("注册测试返回 HOTKEY_START True True None, 停止后 HOTKEY_STOP False.", cell)],
201 |         [p("动画素材", cell)], p("通过", cell_bold), p("应用日志记录 shenshen 91 段动画加载完成。", cell)],
202 |         [p("macOS 应用", cell)], p("通过", cell_bold), p("/Applications/DSH-Pet.app 已验证签名并可启动.", cell)],
203 |         [p("交付压缩包", cell)], p("通过", cell_bold), p("DSH-Pet-macos-arm64-collections-hotkeys.zip 已通过 unzip -tq.", cell)],
204 |     ]
205 |     validation_table = Table(validation, colWidths=[35 * mm, 18 * mm, 121 * mm], repeatRows=1)
206 |     validation_table.setStyle(TableStyle([
207 |         ("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#1B6085")),
208 |         ("TEXTCOLOR", (0, 0), (-1, 0), colors.white),
209 |         ("ROWBACKGROUNDS", (0, 1), (-1, -1), [colors.HexColor("#F8FBFE"), colors.HexColor("#EEF6FB")]),
210 |         ("BOX", (0, 0), (-1, -1), 0.5, colors.HexColor("#B9D4E8")),
211 |         ("INNERGRID", (0, 0), (-1, -1), 0.3, colors.HexColor("#CFE0EC")),
212 |         ("VALIGN", (0, 0), (-1, -1), "TOP"),
213 |         ("LEFTPADDING", (0, 0), (-1, -1), 2.2 * mm),
214 |         ("RIGHTPADDING", (0, 0), (-1, -1), 2.2 * mm),
215 |         ("TOPPADDING", (0, 0), (-1, -1), 2 * mm),
216 |         ("BOTTPADDING", (0, 0), (-1, -1), 2 * mm),
217 |     ]))
218 |     story.append(validation_table)

```

```

219 |
220 |     story.append(p("5. 文件交付位置", h1))
221 |     paths = [
222 |         ("源代码", str(ROOT)),
223 |         ("macOS 应用", "/Applications/DSH-Pet.app"),
224 |         ("应用压缩包", "/Users/ray/Downloads/DSH-Pet-macos-arm64-collections-hotkeys.zip"),
225 |         ("本 PDF", str(OUT)),
226 |         ("GitHub 分支", "https://github.com/MerZlin/dsh-pet-indesktop/tree/codex/upload-interaction-pdf"),
227 |     ]
228 |     path_rows = [[p("交付物", cell_header), p("路径或地址", cell_header)]]
229 |     path_rows.extend([[p(name, cell), p(value, small)] for name, value in paths])
230 |     path_table = Table(path_rows, colWidths=[34 * mm, 140 * mm], repeatRows=1)
231 |     path_table.setStyle(TableStyle([
232 |         ("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#1B6085")),
233 |         ("TEXTCOLOR", (0, 0), (-1, 0), colors.white),
234 |         ("ROWBACKGROUNDS", (0, 1), (-1, -1), [colors.HexColor("#F8FBFE"), colors.HexColor("#EEF6FB")]),
235 |         ("BOX", (0, 0), (-1, -1), 0.5, colors.HexColor("#B9D4E8")),
236 |         ("INNERGRID", (0, 0), (-1, -1), 0.3, colors.HexColor("#CFE0EC")),
237 |         ("ALIGN", (0, 0), (-1, -1), "TOP"),
238 |         ("LEFTPADDING", (0, 0), (-1, -1), 2.2 * mm),
239 |         ("RIGHTPADDING", (0, 0), (-1, -1), 2.2 * mm),
240 |         ("TOPPADDING", (0, 0), (-1, -1), 2 * mm),
241 |         ("BOTTOMPADDING", (0, 0), (-1, -1), 2 * mm),
242 |     ]))
243 |     story.extend([path_table, Spacer(1, 5 * mm)])
244 |     story.append(p("备注：本报告对应当前工作区交付范围。原始视频、高画质视频、超分工具、应用包和历史备份保持在各自目录中，未在本次上传中覆盖或删除。", body))
245 |     story.append(p("后续升级建议：继续将角色动作元数据和行为概率配置化；新增角色时复用同一素材目录协议和播放管线。", body))
246 |
247 |     doc.build(story, onFirstPage=footer, onLaterPages=footer)
248 |     return OUT
249 |
250 |
251 | if __name__ == "__main__":
252 |     path = build()
253 |     print(path)

```

28. tools/build_runtime_webm_assets.py (167 lines)

```

1 | #!/usr/bin/env python3
2 | """Build lightweight transparent WebM playback assets from high-resolution masters."""
3 |
4 | from __future__ import annotations
5 |
6 | import argparse
7 | from concurrent.futures import ThreadPoolExecutor, as_completed
8 | from pathlib import Path
9 |
10 | from superres_webm_assets import alpha_range, probe_video, run
11 |
12 |
13 | def target_size(source_meta: dict, max_width: int, max_height: int) -> tuple[int, int]:
14 |     scale = min(max_width / source_meta["width"], max_height / source_meta["height"], 1.0)
15 |     width = max(2, int(source_meta["width"] * scale) // 2 * 2)
16 |     height = max(2, int(source_meta["height"] * scale) // 2 * 2)
17 |     return width, height
18 |
19 |
20 | def valid_output(
21 |     source: Path,
22 |     destination: Path,
23 |     *,
24 |     max_width: int,
25 |     max_height: int,
26 |     ffmpeg: Path,
27 |     ffprobe: Path,
28 | ) -> tuple[bool, str]:
29 |     if not destination.is_file() or destination.stat().st_size <= 0:
30 |         return False, "missing"
31 |     try:
32 |         source_meta = probe_video(source, ffprobe)
33 |         output_meta = probe_video(destination, ffprobe)
34 |     except Exception as exc:
35 |         return False, f"probe failed: {exc}"
36 |     expected = target_size(source_meta, max_width, max_height)
37 |     actual = output_meta["width"], output_meta["height"]
38 |     if actual != expected:

```

```

39 |         return False, f"size {actual} != {expected}"
40 |     if abs(output_meta["fps"] - source_meta["fps"]) > 0.01:
41 |         return False, f"fps {output_meta['fps']} != {source_meta['fps']}"
42 |     tolerance = max(0.2, 2.0 / max(1.0, source_meta["fps"]))
43 |     if abs(output_meta["duration"] - source_meta["duration"]) > tolerance:
44 |         return False, "duration mismatch"
45 |     if output_meta["alpha_mode"] != "1":
46 |         return False, "alpha_mode missing"
47 |     alpha = alpha_range(destination, ffmpeg)
48 |     if alpha is None or alpha[0] >= 32 or alpha[1] <= 223:
49 |         return False, f"invalid alpha range {alpha}"
50 |     return True, "ok"
51 |
52 |
53 | def build_one(
54 |     source: Path,
55 |     destination: Path,
56 |     *,
57 |     max_width: int,
58 |     max_height: int,
59 |     crf: int,
60 |     ffmpeg: Path,
61 |     ffprobe: Path,
62 | ) -> str:
63 |     valid, _ = valid_output(
64 |         source,
65 |         destination,
66 |         max_width=max_width,
67 |         max_height=max_height,
68 |         ffmpeg=ffmpeg,
69 |         ffprobe=ffprobe,
70 |     )
71 |     if valid:
72 |         return "skip"
73 |     destination.parent.mkdir(parents=True, exist_ok=True)
74 |     partial = destination.with_name(f"{destination.stem}.partial.webm")
75 |     partial.unlink(missing_ok=True)
76 |     source_meta = probe_video(source, ffprobe)
77 |     width, height = target_size(source_meta, max_width, max_height)
78 |     run([
79 |         str(ffmpeg), "-y", "-hide_banner", "-loglevel", "error",
80 |         "-c:v", "libvpx-vp9", "-i", str(source),
81 |         "-vf", f"scale={width}:{height}:flags=lanczos",
82 |         "-c:v", "libvpx-vp9", "-pix_fmt", "yuva420p", "-auto-alt-ref", "0",
83 |         "-crf", str(crf), "-b:v", "0", "-row-mt", "1", "-cpu-used", "4",
84 |         "-metadata:s:v:0", "alpha_mode=1", "-an", str(partial),
85 |     ])
86 |     valid, reason = valid_output(
87 |         source,
88 |         partial,
89 |         max_width=max_width,
90 |         max_height=max_height,
91 |         ffmpeg=ffmpeg,
92 |         ffprobe=ffprobe,
93 |     )
94 |     if not valid:
95 |         partial.unlink(missing_ok=True)
96 |         raise RuntimeError(f"invalid output {source.name}: {reason}")
97 |     partial.replace(destination)
98 |     return "done"
99 |
100 |
101 | def main() -> None:
102 |     parser = argparse.ArgumentParser()
103 |     parser.add_argument("--input-root", type=Path, required=True)
104 |     parser.add_argument("--output-root", type=Path, required=True)
105 |     parser.add_argument("--ffmpeg", type=Path, default=Path("/opt/homebrew/bin/ffmpeg"))
106 |     parser.add_argument("--ffprobe", type=Path, default=Path("/opt/homebrew/bin/ffprobe"))
107 |     parser.add_argument("--max-width", type=int, default=1280)
108 |     parser.add_argument("--max-height", type=int, default=720)
109 |     parser.add_argument("--crf", type=int, default=18)
110 |     parser.add_argument("--workers", type=int, default=2)
111 |     parser.add_argument("--verify-only", action="store_true")
112 |     args = parser.parse_args()
113 |
114 |     input_root = args.input_root.resolve()
115 |     output_root = args.output_root.resolve()
116 |     ffmpeg = args.ffmpeg.resolve()
117 |     ffprobe = args.ffprobe.resolve()

```

```

118 | inputs = sorted(input_root.rglob("*.webm"))
119 | if not inputs:
120 |     raise SystemExit(f"no WebM assets found: {input_root}")
121 | output_root.mkdir(parents=True, exist_ok=True)
122 |
123 | if args.verify_only:
124 |     failures = []
125 |     for index, source in enumerate(inputs, 1):
126 |         relative = source.relative_to(input_root)
127 |         valid, reason = valid_output(
128 |             source,
129 |             output_root / relative,
130 |             max_width=args.max_width,
131 |             max_height=args.max_height,
132 |             ffmpeg=ffmpeg,
133 |             ffprobe=ffprobe,
134 |         )
135 |         print(f"[{index}/{len(inputs)}] {'ok' if valid else 'invalid'} [{relative}]", flush=True)
136 |         if not valid:
137 |             failures.append(f"[{relative}]: {reason}")
138 |     if failures:
139 |         raise SystemExit(f"verification failed for {len(failures)} assets")
140 |     print(f"verified {len(inputs)} runtime assets -> {output_root}")
141 |     return
142 |
143 | with ThreadPoolExecutor(max_workers=max(1, args.workers)) as executor:
144 |     futures = []
145 |     for source in inputs:
146 |         relative = source.relative_to(input_root)
147 |         future = executor.submit(
148 |             build_one,
149 |             source,
150 |             output_root / relative,
151 |             max_width=args.max_width,
152 |             max_height=args.max_height,
153 |             orf=args.orf,
154 |             ffmpeg=ffmpeg,
155 |             ffprobe=ffprobe,
156 |         )
157 |         futures[future] = relative
158 |     completed = 0
159 |     for future in as_completed(futures):
160 |         completed += 1
161 |         result = future.result()
162 |         print(f"[{completed}/{len(inputs)}] {result} {futures[future]}", flush=True)
163 |     print(f"completed {len(inputs)} runtime assets -> {output_root}")
164 |
165 |
166 | if __name__ == "__main__":
167 |     main()

```

29. tools/build_source_code_pdf.py (161 lines)

```

1 | from __future__ import annotations
2 |
3 | from datetime import date
4 | from pathlib import Path
5 |
6 | from reportlab.lib import colors
7 | from reportlab.lib.enums import TA_CENTER
8 | from reportlab.lib.pagesizes import A4, landscape
9 | from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
10 | from reportlab.lib.units import mm
11 | from reportlab.pdfbase import pdfmetrics
12 | from reportlab.pdfbase.ttfonts import TTFont
13 | from reportlab.platypus import PageBreak, Paragraph, Preformatted, SimpleDocTemplate, Spacer, Table, TableStyle
14 |
15 |
16 | ROOT = Path(__file__).resolve().parents[1]
17 | OUT = ROOT / "output" / "pdf" / "dsh-pet-indesktop-source-code.pdf"
18 | FONT = Path("/Users/ray/Library/Fonts/SimHei.ttf")
19 |
20 |
21 | def source_files() -> list[Path]:
22 |     roots = [ROOT / "pet", ROOT / "tests", ROOT / "scripts", ROOT / "tools"]
23 |     suffixes = [".py", ".command", ".md", ".toml", ".yml", ".yaml", ".json"]

```

```

24 | files = []
25 | for root in roots:
26 |     if not root.exists():
27 |         continue
28 |     for path in root.rglob("*"):
29 |         if not path.is_file() or path.suffix not in suffixes:
30 |             continue
31 |         if any(part.startswith("__tmp") for part in path.parts):
32 |             continue
33 |         if "__pycache__" in path.parts or ".venv" in path.parts:
34 |             continue
35 |         files.append(path)
36 | for name in (".gitignore", "README.md", "run-mac.command"):
37 |     path = ROOT / name
38 |     if path.exists():
39 |         files.append(path)
40 | return sorted(set(files), key=lambda path: str(path.relative_to(ROOT)))
41 |
42 |
43 | def register_font() -> str:
44 |     pdfmetrics.registerFont(TTFont("CodeZH", str(FONT)))
45 |     return "CodeZH"
46 |
47 |
48 | def footer(canvas, doc) -> None:
49 |     canvas.saveState()
50 |     canvas.setStrokeColor(colors.HexColor("#D7E5F2"))
51 |     canvas.line(12 * mm, 10 * mm, 285 * mm, 10 * mm)
52 |     canvas.setFillColor(colors.HexColor("#6C8298"))
53 |     canvas.setFont("CodeZH", 7)
54 |     canvas.drawString(12 * mm, 6 * mm, "DSH-Pet source code listing")
55 |     canvas.drawRightString(285 * mm, 6 * mm, f"[doc.page]")
56 |     canvas.restoreState()
57 |
58 |
59 | def build() -> Path:
60 |     font_name = register_font()
61 |     files = source_files()
62 |     OUT.parent.mkdir(parents=True, exist_ok=True)
63 |     doc = SimpleDocTemplate(
64 |         str(OUT),
65 |         pagesize=landscape(A4),
66 |         rightMargin=12 * mm,
67 |         leftMargin=12 * mm,
68 |         topMargin=12 * mm,
69 |         bottomMargin=14 * mm,
70 |         title="DSH-Pet 源代码全文",
71 |         author="Codex",
72 |     )
73 |
74 |     styles = getSampleStyleSheet()
75 |     cover = ParagraphStyle(
76 |         "Cover", parent=styles["Title"], fontName=font_name, fontSize=24,
77 |         leading=32, alignment=TA_CENTER, textColor=colors.HexColor("#12385A"),
78 |         spaceAfter=6 * mm,
79 |     )
80 |     sub = ParagraphStyle(
81 |         "Sub", parent=styles["Normal"], fontName=font_name, fontSize=11,
82 |         leading=18, alignment=TA_CENTER, textColor=colors.HexColor("#54728E"),
83 |     )
84 |     heading = ParagraphStyle(
85 |         "FileHeading", parent=styles["Heading2"], fontName=font_name, fontSize=11,
86 |         leading=16, textColor=colors.white, backgroundColor=colors.HexColor("#1B6095"),
87 |         leftIndent=2 * mm, borderPadding=2 * mm, spaceBefore=2 * mm, spaceAfter=2 * mm,
88 |     )
89 |     toc_heading = ParagraphStyle(
90 |         "TocHeading", parent=styles["Heading1"], fontName=font_name, fontSize=16,
91 |         leading=23, textColor=colors.HexColor("#12385A"), spaceAfter=3 * mm,
92 |     )
93 |     normal = ParagraphStyle(
94 |         "NormalZH", parent=styles["Normal"], fontName=font_name, fontSize=9,
95 |         leading=15, textColor=colors.HexColor("#243746"),
96 |     )
97 |     code = ParagraphStyle(
98 |         "CodeZH", parent=styles["Code"], fontName=font_name, fontSize=5.3,
99 |         leading=6.4, leftIndent=0, rightIndent=0, spaceBefore=0, spaceAfter=0,
100 |         textColor=colors.HexColor("#1F2933"), backgroundColor=colors.HexColor("#F7FAFC"),
101 |     )
102 |     toc_cell = ParagraphStyle(

```

```

103 |         "TocCell", parent=normal, fontSize=8.3, leading=12, spaceAfter=0,
104 |     )
105 |     toc_header = ParagraphStyle(
106 |         "TocHeader", parent=toc_cell, textColor=colors.white,
107 |     )
108 |
109 |     story = [
110 |         Spacer(1, 25 * mm),
111 |         Paragraph("DSH-Pet", cover),
112 |         Paragraph("源代码全文 PDF", cover),
113 |         Paragraph("逐文件收录当前工作区代码，带文件名与行号", sub),
114 |         Spacer(1, 8 * mm),
115 |         Paragraph("生成日期: " + date.today().isoformat(), sub),
116 |         Paragraph("范围: pet / tests / scripts / tools 及根目录代码配置", sub),
117 |         Spacer(1, 12 * mm),
118 |         Paragraph("说明: 行号和文件标题是 PDF 排版辅助信息; 代码正文按文件内容输出。视频、图片、模型和临时运行产物不属于代码，未写入本 PDF。", normal),
119 |         PageBreak(),
120 |         Paragraph("文件目录", toc_heading),
121 |     ]
122 |
123 |     toc_rows = [[Paragraph("序号", toc_header), Paragraph("文件", toc_header), Paragraph("行数", toc_header)]]
124 |     for index, path in enumerate(files, 1):
125 |         text = path.read_text(encoding="utf-8", errors="replace")
126 |         toc_rows.append([
127 |             Paragraph(str(index), toc_cell),
128 |             Paragraph(str(path.relative_to(ROOT)), toc_cell),
129 |             Paragraph(str(len(text.splitlines())), toc_cell),
130 |         ])
131 |     toc = Table(toc_rows, colWidths=[18 * mm, 210 * mm, 25 * mm], repeatRows=1)
132 |     toc.setStyle(TableStyle([
133 |         ("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#1B6085")),
134 |         ("ROWBACKGROUNDS", (0, 1), (-1, -1), [colors.HexColor("#F7FAFC"), colors.HexColor("#EAF4FB")]),
135 |         ("BOX", (0, 0), (-1, -1), 0.5, colors.HexColor("#B9D4E8")),
136 |         ("INNERGRID", (0, 0), (-1, -1), 0.3, colors.HexColor("#CFE0EC")),
137 |         ("ALIGN", (0, 0), (-1, -1), "TOP"),
138 |         ("LEFTPADDING", (0, 0), (-1, -1), 2 * mm),
139 |         ("RIGHTPADDING", (0, 0), (-1, -1), 2 * mm),
140 |         ("TOPPADDING", (0, 0), (-1, -1), 1.2 * mm),
141 |         ("BOTTOMPADDING", (0, 0), (-1, -1), 1.2 * mm),
142 |     ]))
143 |     story.extend([toc, PageBreak()])
144 |
145 |     for index, path in enumerate(files, 1):
146 |         relative = path.relative_to(ROOT)
147 |         text = path.read_text(encoding="utf-8", errors="replace")
148 |         lines = text.splitlines()
149 |         numbered = "\n".join(f"{line_no:5d} | {line}" for line_no, line in enumerate(lines, 1))
150 |         story.append(Paragraph(f"{index:02d}. {relative} ({len(lines)} lines)", heading))
151 |         story.append(Preformatted(numbered, code, maxLineLength=170))
152 |         story.append(Spacer(1, 4 * mm))
153 |
154 |     doc.build(story, onFirstPage=footer, onLaterPages=footer)
155 |     print(f"files={len(files)}")
156 |     print(f"output={OUT}")
157 |     return OUT
158 |
159 |
160 | if __name__ == "__main__":
161 |     build()

```

30. tools/superres_webm_assets.py (207 lines)

```

1 | #!/usr/bin/env python3
2 | """AI-super-resolve transparent WebM pet assets with Real-ESRGAN NCNN."""
3 |
4 | from __future__ import annotations
5 |
6 | import argparse
7 | import json
8 | import subprocess
9 | import tempfile
10 | from fractions import Fraction
11 | from pathlib import Path
12 |
13 |
14 | def run(command: list[str], *, cwd: Path | None = None) -> None:

```

```

15 |     completed = subprocess.run(
16 |         command,
17 |         cwd=cwd,
18 |         stdout=subprocess.DEVNULL,
19 |         stderr=subprocess.PIPE,
20 |         text=True,
21 |     )
22 |     if completed.returncode != 0:
23 |         detail = (completed.stderr or '').strip().splitlines()[-12:]
24 |         raise RuntimeError(
25 |             f"command failed ({completed.returncode}): {' '.join(command[:4])}\n"
26 |             + '\n'.join(detail)
27 |         )
28 |
29 |
30 | def probe_video(path: Path, ffprobe: Path) -> dict:
31 |     completed = subprocess.run(
32 |         [
33 |             str(ffprobe), "-v", "error", "-select_streams", "v:0",
34 |             "-show_entries",
35 |             "stream=width,height,avg_frame_rate:stream_tags=alpha_mode:format=duration",
36 |             "-of", "json", str(path),
37 |         ],
38 |         check=True,
39 |         capture_output=True,
40 |         text=True,
41 |     )
42 |     payload = json.loads(completed.stdout)
43 |     stream = payload["streams"][0]
44 |     tags = {str(k).lower(): str(v) for k, v in stream.get("tags", {}).items()}
45 |     fps_text = stream.get("avg_frame_rate", "0/1")
46 |     return {
47 |         "width": int(stream["width"]),
48 |         "height": int(stream["height"]),
49 |         "fps": float(Fraction(fps_text)) if fps_text != "0/0" else 0.0,
50 |         "fps_text": fps_text,
51 |         "duration": float(payload.get("format", {}).get("duration") or 0.0),
52 |         "alpha_mode": tags.get("alpha_mode"),
53 |     }
54 |
55 |
56 | def alpha_range(path: Path, ffmpeg: Path) -> tuple[int, int] | None:
57 |     completed = subprocess.run(
58 |         [
59 |             str(ffmpeg), "-v", "error", "-c:v", "libvpx-vp9", "-i", str(path),
60 |             "-vf", "alphaextract", "-frames:v", "1", "-pix_fmt", "gray",
61 |             "-f", "rawvideo", "-",
62 |         ],
63 |         stdout=subprocess.PIPE,
64 |         stderr=subprocess.PIPE,
65 |     )
66 |     if completed.returncode != 0 or not completed.stdout:
67 |         return None
68 |     return min(completed.stdout), max(completed.stdout)
69 |
70 |
71 | def valid_output(
72 |     source: Path,
73 |     destination: Path,
74 |     *,
75 |     scale: int,
76 |     ffmpeg: Path,
77 |     ffprobe: Path,
78 | ) -> tuple[bool, str]:
79 |     if not destination.is_file() or destination.stat().st_size <= 0:
80 |         return False, "missing"
81 |     try:
82 |         source_meta = probe_video(source, ffprobe)
83 |         output_meta = probe_video(destination, ffprobe)
84 |     except Exception as exc:
85 |         return False, f"probe failed: {exc}"
86 |
87 |     expected_size = (source_meta["width"] * scale, source_meta["height"] * scale)
88 |     actual_size = (output_meta["width"], output_meta["height"])
89 |     if actual_size != expected_size:
90 |         return False, f"size {actual_size} != {expected_size}"
91 |     if abs(output_meta["fps"] - source_meta["fps"]) > 0.01:
92 |         return False, f"fps {output_meta['fps']} != {source_meta['fps']}"
93 |     duration_tolerance = max(0.2, 2.0 / max(1.0, source_meta["fps"]))

```



```

94 |     if abs(output_meta["duration"] - source_meta["duration"]) > duration_tolerance:
95 |         return False, "duration mismatch"
96 |     if output_meta["alpha_mode"] != "1":
97 |         return False, "alpha_mode missing"
98 |     alpha = alpha_range(destination, ffmpeg)
99 |     if alpha is None or alpha[0] >= 32 or alpha[1] <= 223:
100 |         return False, f"invalid alpha range {alpha}"
101 |     return True, "ok"
102 |
103 |
104 | def main() -> None:
105 |     parser = argparse.ArgumentParser()
106 |     parser.add_argument("--input-root", type=Path, required=True)
107 |     parser.add_argument("--output-root", type=Path, required=True)
108 |     parser.add_argument(
109 |         "--engine-root",
110 |         type=Path,
111 |         default=Path(__file__).resolve().parent / "realesrgan-20220424",
112 |     )
113 |     parser.add_argument("--ffmpeg", type=Path, default=Path("/opt/homebrew/bin/ffmpeg"))
114 |     parser.add_argument("--ffprobe", type=Path, default=Path("/opt/homebrew/bin/ffprobe"))
115 |     parser.add_argument("--model-name", default="realesr-animevideov3")
116 |     parser.add_argument("--scale", type=int, choices=(2, 3, 4), default=2)
117 |     parser.add_argument("--workers", default="1:4:2")
118 |     parser.add_argument("--crf", type=int, default=18)
119 |     parser.add_argument("--limit", type=int, default=0)
120 |     parser.add_argument("--verify-only", action="store_true")
121 |     args = parser.parse_args()
122 |
123 |     input_root = args.input_root.resolve()
124 |     output_root = args.output_root.resolve()
125 |     engine_root = args.engine_root.resolve()
126 |     ffmpeg = args.ffmpeg.resolve()
127 |     ffprobe = args.ffprobe.resolve()
128 |     engine = engine_root / "realesrgan-ncnn-vulkan"
129 |     if not input_root.is_dir():
130 |         raise SystemExit(f"input root does not exist: {input_root}")
131 |     if not args.verify_only and not engine.is_file():
132 |         raise SystemExit(f"Real-ESRGAN executable does not exist: {engine}")
133 |
134 |     inputs = sorted(input_root.rglob("*.webm"))
135 |     if args.limit:
136 |         inputs = inputs[: args.limit]
137 |     output_root.mkdir(parents=True, exist_ok=True)
138 |
139 |     failures: list[str] = []
140 |
141 |     for index, source in enumerate(inputs, 1):
142 |         relative = source.relative_to(input_root)
143 |         destination = output_root / relative
144 |         destination.parent.mkdir(parents=True, exist_ok=True)
145 |         valid, reason = valid_output(
146 |             source,
147 |             destination,
148 |             scale=args.scale,
149 |             ffmpeg=ffmpeg,
150 |             ffprobe=ffprobe,
151 |         )
152 |         if valid:
153 |             print(f"[{index}/{len(inputs)}] skip {relative}", flush=True)
154 |             continue
155 |         if args.verify_only:
156 |             failures.append(f"[{relative}]: {reason}")
157 |             print(f"[{index}/{len(inputs)}] invalid {relative}: {reason}", flush=True)
158 |             continue
159 |
160 |         partial = destination.with_name(f"{destination.stem}.partial.webm")
161 |         partial.unlink(missing_ok=True)
162 |         source_meta = probe_video(source, ffprobe)
163 |
164 |         with tempfile.TemporaryDirectory(prefix="dsh-superres-") as temporary:
165 |             work = Path(temporary)
166 |             source_frames = work / "source"
167 |             output_frames = work / "output"
168 |             source_frames.mkdir()
169 |             output_frames.mkdir()
170 |             run([
171 |                 str(ffmpeg), "-y", "--hide_banner", "--loglevel", "error",
172 |                 "-c:v", "libvpx-vp9", "-i", str(source), "--vsync", "0",

```

```

173 |         str(source_frames / "frame%05d.png"),
174 |     ])
175 |     run([
176 |         str(engine), "-i", str(source_frames), "-o", str(output_frames),
177 |         "-n", args.model_name, "-s", str(args.scale), "-f", "png",
178 |         "-t", "256", "-j", args.workers,
179 |     ], cwd=engine_root)
180 |     run([
181 |         str(ffmpeg), "-y", "-hide_banner", "-loglevel", "error",
182 |         "-framerate", source_meta["fps_text"],
183 |         "-i", str(output_frames / "frame%05d.png"),
184 |         "-c:v", "libvpx-vp9", "-pix_fmt", "yuva420p", "-auto-alt-ref", "0",
185 |         "-crf", str(args.crf), "-b:v", "0", "-row-mt", "1", "-cpu-used", "4",
186 |         "-metadata:s:v:0", "alpha_mode=1", "-an", str(partial),
187 |     ])
188 |     valid, reason = valid_output(
189 |         source,
190 |         partial,
191 |         scale=args.scale,
192 |         ffmpeg=ffmpeg,
193 |         ffprobe=ffprobe,
194 |     )
195 |     if not valid:
196 |         partial.unlink(missing_ok=True)
197 |         raise RuntimeError(f"invalid output {relative}: {reason}")
198 |         partial.replace(destination)
199 |         print(f"[{index}/{len(inputs)}] done {relative}", flush=True)
200 |
201 |     if failures:
202 |         raise SystemExit(f"verification failed for {len(failures)} assets")
203 |     print(f"completed {len(inputs)} assets -> {output_root}")
204 |
205 |
206 | if __name__ == "__main__":
207 |     main()

```